

ΑΡΙΣΤΟΤΕΛΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΟΝΙΚΗΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ



Επιτάχυνση επιλογής top-k σε ετερογενή συστήματα CPU, GPU και NPU

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

Ιωάννης Μιχάλαϊνας

ioannisam@ece.auth.gr

Επιβλέπων: Νικόλαος Πιτσιάνης

Αναπληρωτής Καθηγητής, Τμήμα Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών

Θεσσαλονίκη, Σεπτέμβριος 2026

Περίληψη

Η επιλογή top-k εντοπίζει τα k μεγαλύτερα στοιχεία ενός συνόλου. Εμφανίζεται σε πλήθος εφαρμογών και επαναλαμβάνεται αρκετά συχνά ώστε το κόστος της να είναι αισθητό. Η πρόσφατη ενσωμάτωση μονάδων νευρωνικής επεξεργασίας στην ίδια ψηφίδα με τον επεξεργαστή εγείρει το εύλογο ερώτημα αν συμφέρει η ανάθεση τέτοιων υπολογισμών σε επιταχυντή.

Η εργασία υλοποιεί δύο αλγορίθμους με θεμελιωδώς διαφορετική συμπεριφορά: την περικομμένη διτονική ταξινόμηση, που είναι ανεξάρτητη από τα δεδομένα, και την επιλογή με τοπικούς σωρούς, που εξαρτάται από αυτά, σε τρεις αρχιτεκτονικές: επεξεργαστή με διανυσματικές επεκτάσεις AVX-512, μονάδα γραφικών με CUDA και μονάδα νευρωνικής επεξεργασίας AMD XDNA. Οι υλοποιήσεις αξιολογήθηκαν με ενιαία μεθοδολογία σε δύο πεδία μέτρησης, το αλγοριθμικό και το end-to-end, καταγράφοντας τους χρόνους εκτέλεσης, το ενεργειακό αποτύπωμα και τον όγκο διακινούμενων δεδομένων. Η τελική τους απόδοση αναλύθηκε σε σύγκριση με τις μέγιστες θεωρητικές δυνατότητες του συστήματος.

Τα αποτελέσματα δείχνουν ότι το πρόβλημα περιορίζεται από την κίνηση δεδομένων σε κάθε αρχιτεκτονική. Η κατάταξη των δύο αλγορίθμων ανατρέπεται με τον λόγο n/k και η επιλογή του πεδίου μέτρησης αλλάζει εντελώς την κατάταξη των αρχιτεκτονικών. Ενεργειακά, η μονάδα νευρωνικής επεξεργασίας εμφανίζει τη χαμηλότερη ισχύ και ταυτόχρονα την υψηλότερη κατανάλωση, καθώς η διάρκεια κυριαρχεί του γινομένου. Καμία εκτέλεση σε επιταχυντή δεν είναι βέλτιστη κατά Pareto και ειδικά για μικρό k η ανάθεση είναι μετρήσιμα χειρότερη από τον υπολογισμό στον επεξεργαστή.

Λέξεις-κλειδιά: επιλογή top-k, δίκτυα διτονικής ταξινόμησης, ετερογενής υπολογισμός, μονάδα νευρωνικής επεξεργασίας, MLIR-AIE, διανυσματοποίηση SIMD, ενεργειακή αποτίμηση, μοντέλο roofline

Abstract

Top-k selection identifies the k largest elements of a set. It appears across a wide range of applications and is repeated often enough that its cost is noticeable. The recent integration of neural processing units into the same die as the processor makes the question of whether offloading such computations to an accelerator is worthwhile a reasonable one.

This work implements two algorithms with fundamentally different behaviour: truncated bitonic sorting which is data-oblivious and map-reduce selection with local heaps which is data-dependent, on three architectures: a processor with AVX-512 vector extensions, a GPU programmed in CUDA, and an AMD XDNA neural processing unit. The implementations were evaluated with a unified methodology across two measurement scopes, algorithmic and end-to-end, recording execution time, energy footprint and the volume of data moved. Their resulting performance was analysed against the system's maximum theoretical capabilities.

The results show that the problem is limited by data movement on every architecture. The ranking of the two algorithms is overturned by the ratio n/k and the choice of measurement scope completely overturns the ranking of the architectures. In energy terms the neural processing unit exhibits the lowest power draw and simultaneously the highest consumption, since duration dominates the product. No accelerator run is Pareto-optimal. For small k , offloading is measurably worse than computing on the processor.

Keywords: top-k selection, bitonic sorting networks, heterogeneous computing, neural processing unit, MLIR-AIE, SIMD vectorisation, energy assessment, roofline model

Ευχαριστίες

Η εργασία αυτή αποτελεί το μεγαλύτερο προγραμματιστικό έργο που έχω αναλάβει μέχρι σήμερα. Θα ήθελα να ευχαριστήσω τον επιβλέποντα καθηγητή μου, κ. Νικόλαο Πιτσιάνη, στα μαθήματα του οποίου ανακάλυψα το ενδιαφέρον μου για το High Performance Computing και το low-level programming.

Επίσης, θα ήθελα να ευχαριστήσω την οικογένειά μου για την αμέριστη υποστήριξη που μου παρείχε καθ' όλη τη διάρκεια των σπουδών μου· ιδιαίτερα τον αδελφό μου, Μανώλη, ο οποίος, ως συνάδελφος, με βοήθησε με πολύτιμες συμβουλές και καθοδήγηση.

Ιωάννης Μιχάλαινας

Περιεχόμενα

Περίληψη	2
Abstract	3
Ευχαριστίες	4
1 Εισαγωγή	10
1.1 Αντικείμενο της εργασίας	10
1.2 Κίνητρο	10
1.3 Συνεισφορά	11
1.4 Δομή της εργασίας	11
2 Υπόβαθρο και σχετική εργασία	13
2.1 Το πρόβλημα της επιλογής top-k	13
2.2 Αλγόριθμοι επιλογής	14
2.2.1 Δίκτυα διτονικής ταξινόμησης	15
2.2.2 Περικομμένη διτονική ταξινόμηση	15
2.2.3 Επιλογή με σωρούς και map-reduce	17
2.2.4 Επιλογή με βάση την αναπαράσταση	18
2.3 Αρχιτεκτονικές επεξεργασίας	18
2.3.1 Διανυσματικές επεκτάσεις CPU	18
2.3.2 Μοντέλο SIMT και ιεραρχία μνήμης GPU	19
2.3.3 Αρχιτεκτονική ροής δεδομένων AIE	19
2.4 Υπάρχουσες υλοποιήσεις και βιβλιοθήκες	20
3 Μεθοδολογία και πειραματική διάταξη	22
3.1 Σύστημα αναφοράς	22
3.2 Δομή της υποδομής μετρήσεων	23
3.3 Μετρούμενα μεγέθη	24
3.3.1 Πεδία μέτρησης: algo και end-to-end	24
3.3.2 Μέτρηση χρόνου	25
3.3.3 Μέτρηση ενέργειας	26
3.3.4 Μέτρηση κίνησης δεδομένων	27
3.4 Μοντέλο roofline	27
3.5 Βιβλιοθήκες σύγκρισης	29
3.6 Έλεγχος ορθότητας	29
3.7 Παραμετρικός χώρος	30
3.7.1 Τύποι δεδομένων	30

3.7.2 Κατανομές εισόδου	31
3.8 Συνθήκες μέτρησης και επαναληψιμότητα	32
4 Υλοποίηση σε CPU	33
4.1 Σχεδιαστικές επιλογές	33
4.1.1 Διανυσματοποίηση	33
4.1.2 Αφαίρεση διανυσματικών τύπων	34
4.1.3 Υποστήριξη αριθμών κινητής υποδιαστολής μισής ακρίβειας	35
4.1.4 Παραλληλοποίηση σε πολλαπλά νήματα	35
4.2 Διτονική ταξινόμηση	36
4.3 Map-reduce με τοπικούς σωρούς	37
4.4 Διαχείριση μνήμης και κίνηση δεδομένων	39
4.5 Υλοποίηση αναφοράς	40
4.6 Περιοριστικοί παράγοντες	40
5 Υλοποίηση σε GPU	42
5.1 Σχεδιαστικές επιλογές	42
5.1.1 Αντιστοίχιση σε πλέγμα και ομάδες νημάτων	42
5.2 Διτονική ταξινόμηση	43
5.2.1 Οργάνωση σε πλακίδια και κοινή μνήμη	43
5.3 Map-reduce με τοπικούς σωρούς	44
5.3.1 Όριο παραλληλίας για μεγάλα k	45
5.4 Διαχείριση μνήμης και κίνηση δεδομένων	46
5.5 Υλοποίηση αναφοράς	47
5.5.1 Επιλογή με βάση την αναπαράσταση στη CUB	47
5.6 Περιοριστικοί παράγοντες	48
6 Υλοποίηση σε NPU	49
6.1 Σχεδιαστικές επιλογές	49
6.1.1 Ροή ανάπτυξης με MLIR-AIE	49
6.1.2 Τοπολογία πλακιδίων και object_fifo	50
6.2 Διτονική ταξινόμηση	51
6.2.1 Πυρήνας υπολογιστικού πλακιδίου	51
6.2.2 Κλιμάκωση σε πολλαπλά πλακίδια	51
6.3 Map-reduce με τοπικούς σωρούς	52
6.4 Διαχείριση μνήμης και κίνηση δεδομένων	52
6.4.1 Διοχέτευση και ρόλος του host	52
6.4.2 Κωδικοποίηση δεδομένων και τύποι	53
6.4.3 Τελική διόρθωση στον host	54
6.5 Υλοποίηση αναφοράς	54
6.6 Περιοριστικοί παράγοντες	54
7 Αποτελέσματα και αξιολόγηση	56
7.1 Ορθότητα	56
7.2 Χρόνος εκτέλεσης	56
7.2.1 Εξάρτηση από το μέγεθος εισόδου	56
7.2.2 Εξάρτηση από την παράμετρο k	58
7.3 Σύγκριση αρχιτεκτονικών	61
7.4 Ανάλυση με το μοντέλο roofline	63

7.4.1 Αξιοποίηση εύρους ζώνης μνήμης	65
7.5 Ενεργειακή αποτίμηση	66
7.6 Επίδραση του τύπου δεδομένων	68
7.7 Ευαισθησία στην κατανομή της εισόδου	70
8 Συμπεράσματα και μελλοντική εργασία	72
8.1 Συμπεράσματα	72
8.2 Περιορισμοί	73
8.3 Μελλοντική εργασία	74
Α΄ Ακρωνύμια	75
Β΄ Γλωσσάρι όρων	76
Γ΄ Αναπαραγωγή των μετρήσεων	77
Γ.1 Απαιτήσεις λογισμικού	77
Γ.2 Μεταγλώττιση	78
Γ.3 Εκτέλεση των πειραμάτων	78
Γ.3.1 Εκτέλεση επιμέρους βημάτων	79
Γ.3.2 Παράμετροι εκτέλεσης	79
Γ.4 Παραγωγή των διαγραμμάτων	80

Κατάλογος Σχημάτων

2.1	Επιλογή έναντι ταξινόμησης	14
2.2	Το πλήρες διτονικό δίκτυο και η περικοπή του	16
2.3	Επιλογή με σωρούς και σχήμα map-reduce	17
2.4	Επιλογή με βάση την αναπαράσταση	18
3.1	Τα τέσσερα επίπεδα της υποδομής μετρήσεων	24
3.2	Τα δύο πεδία μέτρησης ανά αρχιτεκτονική	25
3.3	Η ανάγνωση του μοντέλου roofline	28
3.4	Οι πέντε κατανομές εισόδου	31
4.1	Το θεμελιώδες μοτίβο σύγκρισης-ανταλλαγής	34
4.2	Η διαδρομή της μισής ακρίβειας	35
4.3	Συγχώνευση διαδοχικών επιπέδων σε ένα πέρασμα	37
4.4	Η μαζική απόρριψη του map-reduce	38
5.1	Ταξινομημένο μπλοκ και πλακίδιο κοινής μνήμης	44
5.2	Το όριο παραλληλίας του map-reduce	46
6.1	Τοπολογία πλακιδίων και ουρές object_fifo	50
6.2	Η διοχέτευση τριών σταδίων της εκφόρτωσης	53
7.1	Χρόνος συναρτήσεων του μεγέθους, στα δύο πεδία μέτρησης	56
7.2	Κάθε αλγόριθμος στις τρεις αρχιτεκτονικές	57
7.3	Χρόνος συναρτήσεων του μεγέθους, ανά τιμή του k	58
7.4	Οι τρεις αλγόριθμοι στη μονάδα γραφικών για k ίσο με 131072.	60
7.5	Επιτάχυνση έναντι της υλοποίησης αναφοράς	61
7.6	Τοποθέτηση των εκτελέσεων στα μετρημένα ταβάνια	63
7.7	Μετρημένο εύρος ζώνης ανάγνωσης συναρτήσεων του μεγέθους του συνόλου εργασίας, για τις τρεις αρχιτεκτονικές.	64
7.8	Επιτευχθέν εύρος ζώνης ανά αρχιτεκτονική	65
7.9	Αλγοριθμικός και συνολικός χρόνος στη μονάδα γραφικών	66
7.10	Καθαρή ενέργεια και ισχύς ανά αρχιτεκτονική και αλγόριθμο	67
7.11	Συνολικός χρόνος και ενέργεια ανά μετρημένη περίπτωση	67
7.12	Ευαισθησία του χρόνου στην κατανομή της εισόδου	70

Κατάλογος Πινάκων

3.1	Το σύστημα αναφοράς στο οποίο εκτελέστηκαν όλες οι μετρήσεις.	22
3.2	Το περιεχόμενο των δύο πεδίων μέτρησης ανά αρχιτεκτονική.	24
3.3	Οι τύποι δεδομένων που σαρώνονται.	30
3.4	Οι κατανομές εισόδου που σαρώνονται.	31
3.5	Οι συνθήκες στις οποίες καθλώνεται το σύστημα πριν από κάθε σάρωση.	32
6.1	Κατανομή του χρόνου της περιοχής εκφόρτωσης ανά στάδιο	53
7.1	Αλγοριθμικός χρόνος για τα δύο άκρα του k	59
7.2	Αλγοριθμικός χρόνος ανά τύπο δεδομένων	68
Β'.1	Αντιστοίχιση ελληνικών και αγγλικών όρων.	76
Γ'.1	Οι παράμετροι εκτέλεσης και οι προεπιλογές τους.	80

Κεφάλαιο 1

Εισαγωγή

1.1 Αντικείμενο της εργασίας

Η εργασία εξετάζει την επιτάχυνση της επιλογής top-k, δηλαδή του εντοπισμού των k μεγαλύτερων στοιχείων ενός συνόλου n τιμών, σε τρεις αρχιτεκτονικές: έναν επεξεργαστή γενικού σκοπού, μια μονάδα γραφικών και μια μονάδα νευρωνικής επεξεργασίας ενσωματωμένη στην ίδια ψηφίδα με τον επεξεργαστή.

Για την εκάστοτε αρχιτεκτονική αναπτύσσονται δύο αλγοριθμικές προσεγγίσεις με ριζικά διαφορετικά χαρακτηριστικά εκτέλεσης. Η περικομμένη διτονική ταξινόμηση εκτελεί σταθερή ακολουθία συγκρίσεων ανεξάρτητη από τα δεδομένα, ενώ η επιλογή με τοπικούς σωρούς κατά το σχήμα map-reduce προσαρμόζει την εργασία της στις τιμές της εισόδου. Και οι έξι υλοποιήσεις μετρώνται στον ίδιο παραμετρικό χώρο, με κοινή υποδομή και σημασιολογία μέτρησης, ώστε οι διαφορές που προκύπτουν να αποδίδονται στους αλγορίθμους και στο υλικό, όχι στον τρόπο μέτρησης.

Το βασικό ερώτημα είναι κάτω από ποιες συνθήκες συμφέρει η ανάθεση σε επιταχυντή, συνυπολογίζοντας τόσο την ταχύτητα της αρχιτεκτονικής, όσο και το επιπρόσθετο κόστος της διαδικασίας.

1.2 Κίνητρο

Οι μονάδες νευρωνικής επεξεργασίας πλέον ενσωματώνονται στην ίδια ψηφίδα με τον επεξεργαστή των σύγχρονων φορητών υπολογιστών, μοιράζονται την κύρια μνήμη του και είναι διαθέσιμες για κάθε εφαρμογή που δύναται να τις αξιοποιήσει. Το κατά πόσο αξίζει να ανατεθεί αυτός ο υπολογισμός στον επιταχυντή αποτελεί πλέον επίκαιρο ερώτημα για την ανάπτυξη λογισμικού σε τέτοιο υλικό.

Σχεδιάστηκαν, όμως, για συγκεκριμένο φορτίο. Το προγραμματιστικό τους μοντέλο οργανώνεται κυρίως γύρω από τελεστές νευρωνικών δικτύων και η αρχιτεκτονική τους γύρω από την παραδοχή ότι κάθε δεδομένο θα χρησιμοποιηθεί πολλές φορές αφού μεταφερθεί. Τι συμβαίνει όταν το φορτίο δεν έχει αυτή τη μορφή παραμένει εν πολλοίς ανεξερεύνητο.

Η επιλογή top-k είναι κατάλληλο πρόβλημα για να τεθεί αυτό το ερώτημα. Είναι πρόβλημα απλό στη διατύπωση και δεδομένο στην πράξη, καθώς εμφανίζεται στην

αναζήτηση πλησιέστερων γειτόνων, στα συστήματα ανάκτησης και συστάσεων, στη δειγματοληψία των γλωσσικών μοντέλων και πολλές άλλες εφαρμογές. Ταυτόχρονα, οι επιταχυντές σχεδιάστηκαν για φορτία με πολλές πράξεις ανά στοιχείο και εκτενή επαναχρησιμοποίηση δεδομένων, ενώ η επιλογή top-k έχει ελάχιστες πράξεις ανά στοιχείο και καθόλου επαναχρησιμοποίηση.

1.3 Συνεισφορά

Υλοποιήσεις. Κατασκευάστηκαν έξι υλοποιήσεις, δύο αλγόριθμοι σε τρεις αρχιτεκτονικές, με υποστήριξη πέντε τύπων δεδομένων. Στον επεξεργαστή αξιοποιούνται οι διανυσματικές επεκτάσεις AVX-512 και πολλαπλά νήματα. Στη μονάδα γραφικών η δουλειά οργανώνεται σε πλακίδια με κοινή μνήμη. Η υλοποίηση της μονάδας νευρωνικής επεξεργασίας γράφτηκε εξ ολοκλήρου στο επίπεδο του πλαισίου MLIR-AIE. Οι υλοποιήσεις επαληθεύτηκαν σε 2700 περιπτώσεις συνολικά.

Υποδομή μέτρησης. Κάθε περίπτωση μετριέται σε δύο ρητά διακριτά πεδία, το αλγοριθμικό και το end-to-end. Η κατανάλωση ενέργειας καταγράφεται με ενιαία μεθοδολογία και στις τρεις αρχιτεκτονικές, ο όγκος των διακινούμενων δεδομένων μετριέται απευθείας, ενώ η απόδοση κάθε εκτέλεσης αναλύεται βάσει του μοντέλου οροφής (Roofline) που έχει εξαχθεί για το ίδιο μηχάνημα. Η συγκεκριμένη μεθοδολογία είναι ανεξάρτητη από το πρόβλημα του top-k και μπορεί να εφαρμοστεί σε οποιοδήποτε υπολογιστικό φορτίο εξετάζεται για ανάθεση σε επιταχυντή.

Ευρήματα. Η επιλογή top-k είναι πρόβλημα κίνησης δεδομένων σε κάθε αρχιτεκτονική που εξετάστηκε και καμία υλοποίηση δεν περιορίζεται από την υπολογιστική ικανότητα. Η κατάταξη των δύο αλγορίθμων εξαρτάται από τον λόγο n/k και παραμένει ίδια σε όλες τις αρχιτεκτονικές, με διαφορά που φτάνει τις τρεις τάξεις μεγέθους. Το πεδίο της μέτρησης ανατρέπει πλήρως την κατάταξη των αρχιτεκτονικών. Ενεργειακά, η μονάδα νευρωνικής επεξεργασίας τραβάει τη μικρότερη ισχύ απ' όλες τις υλοποιήσεις και παρ' όλα αυτά καταναλώνει τη μεγαλύτερη ενέργεια, απλώς επειδή χρειάζεται πολύ περισσότερο χρόνο.

Αναπαραγωγιμότητα. Ολόκληρη η πειραματική διαδικασία εκτελείται από δέσμες εντολών που περιλαμβάνονται στο αποθετήριο. Οι συνθήκες μέτρησης καταγράφονται μαζί με τα αποτελέσματα. Οι λεπτομέρειες δίνονται στο παράρτημα Γ'.

1.4 Δομή της εργασίας

Το κεφάλαιο 2 ορίζει το πρόβλημα, παρουσιάζει τις οικογένειες αλγορίθμων που το λύνουν, περιγράφει τα εκτελεστικά μοντέλα των τριών αρχιτεκτονικών και καταλήγει στο κενό που καλείται να καλύψει η εργασία.

Ακολουθεί η μεθοδολογία (κεφάλαιο 3). Παρουσιάζει τι μετριέται, με ποια σημασιολογία, με ποια μέτρα σύγκρισης και υπό ποιες συνθήκες. Αποτελεί προϋπόθεση για την ανάγνωση των αποτελεσμάτων του κεφαλαίου 7.

Έπειτα, ακολουθούν τα κεφάλαια υλοποίησης, ένα ανά αρχιτεκτονική (κεφάλαια 4, 5, 6), με κοινή εσωτερική δομή ώστε οι αντίστοιχες ενότητες να συγκρίνονται άμεσα:

σχεδιαστικές επιλογές, οι δύο αλγόριθμοι, διαχείριση μνήμης, υλοποίηση αναφοράς, περιοριστικοί παράγοντες.

Το κεφάλαιο 7 παρουσιάζει και ερμηνεύει τα αποτελέσματα, δηλαδή τον χρόνο εκτέλεσης, τη σύγκριση αρχιτεκτονικών, το αποτύπωμα μνήμης και την ενεργειακή κατανάλωση, για διαφορετικές παραμέτρους, τύπους δεδομένων και κατανομές εισόδου.

Το κεφάλαιο 8 κλείνει με τα συμπεράσματα, τα όρια εντός των οποίων ισχύουν και πιθανές κατευθύνσεις συνέχειας.

Ακρωνύμια, γλωσσάρι ελληνικών όρων και οδηγίες αναπαραγωγής βρίσκονται στα παραρτήματα.

Κεφάλαιο 2

Υπόβαθρο και σχετική εργασία

2.1 Το πρόβλημα της επιλογής top-k

Ορισμός (Επιλογή top-k). Δοθέντος πίνακα A με n στοιχεία από ένα ολικά διατεταγμένο σύνολο και ακεραίου $k \leq n$, ζητείται το υποσύνολο των k μεγαλύτερων στοιχείων του A .

Αν και η τελική διάταξη των στοιχείων αυτών δεν απασχολεί τον ορισμό του προβλήματος, κάθε υλοποίηση της εργασίας τα ταξινομεί ούτως ή άλλως πριν τα επιστρέψει. Η τελική ταξινόμηση κοστίζει $O(k \log k)$, αμελητέο έναντι του $O(n)$ της επιλογής και δίνει μία μοναδική, ελέγξιμη απάντηση αντί για μία από τις πολλές έγκυρες διατάξεις του ίδιου συνόλου.

Εκ φύσεως, το πρόβλημα απαιτεί λιγότερη πληροφορία από την ταξινόμηση. Σε αυτήν προσδιορίζεται η σχετική σειρά και των n στοιχείων, ενώ η επιλογή top-k αρκείται στον διαχωρισμό του συνόλου σε δύο διακριτά τμήματα: στα k μεγαλύτερα στοιχεία και στα υπόλοιπα $n - k$, χωρίς να ενδιαφέρει η εσωτερική τους διάταξη. Τα κάτω φράγματα το επιβεβαιώνουν: κάθε αλγόριθμος βασισμένος σε συγκρίσεις απαιτεί $\Omega(n \log n)$ συγκρίσεις για ταξινόμηση, αλλά μόνο $\Theta(n)$ για επιλογή με σταθερό k [1]. Ταξινομώντας ολόκληρη την είσοδο για να κρατήσει k στοιχεία, μια υλοποίηση κάνει περισσότερη δουλειά απ' όση χρειάζεται (σχήμα 2.1).



Σχήμα 2.1: Η ίδια είσοδος υπό τις δύο απαιτήσεις. Η ταξινόμηση (α) προσδιορίζει και τις n θέσεις, ενώ η επιλογή (β) διαχωρίζει τα στοιχεία σε δύο ομάδες μόνο: τα k μεγαλύτερα, σε οποιαδήποτε σειρά, και τα υπόλοιπα, για τα οποία δεν κρατείται καμία πληροφορία σειράς.

Η απόσταση ανάμεσα στα δύο προβλήματα εξαρτάται από τον λόγο n/k . Για k πολύ μικρότερο του n η επιλογή είναι σχεδόν γραμμική και η ταξινόμηση σπαταλά σχεδόν όλη της την εργασία. Καθώς το k πλησιάζει το n τα δύο προβλήματα ταυτίζονται. Ολόκληρη η εργασία κινείται ανάμεσα σε αυτά τα δύο άκρα. Το κεφάλαιο 7 δείχνει πως η κατάταξη των αλγορίθμων ανατρέπεται καθώς διανύεται η απόσταση.

Το πρόβλημα εμφανίζεται παντού. Η αναζήτηση k πλησιέστερων γειτόνων απαιτεί την επιλογή των μικρότερων αποστάσεων από έναν πίνακα αποστάσεων [2]. Συστήματα ανάλυσης και συστάσεων επιστρέφουν τα k καλύτερα αποτελέσματα ενός ερωτήματος. Στα γλωσσικά μοντέλα η δειγματοληψία top- k και η αναζήτηση δέσμης εκτελούν επιλογή σε κάθε βήμα παραγωγής. Αν και αποτελεί μικρό μέρος του συνολικού φορτίου κάθε φορά, είναι ένας τόσο συχνά επαναλαμβανόμενος υπολογισμός που το κόστος του γίνεται αισθητό.

2.2 Αλγόριθμοι επιλογής

Ένα κριτήριο που χωρίζει τους αλγορίθμους επιλογής σε οικογένειες είναι το κατά πόσο η ακολουθία των πράξεων εξαρτάται από τις τιμές των δεδομένων. Ένας αλγόριθμος σταθερού χρονοδιαγράμματος εκτελεί τις ίδιες πράξεις με την ίδια σειρά ανεξάρτητα από την είσοδο. Ένας αλγόριθμος εξαρτώμενος από τα δεδομένα προσαρμόζει την πορεία του, με λιγότερη εργασία σε ευνοϊκές εισόδους και περισσότερη σε δυσμενείς.

Η διάκριση αυτή αντιστοιχεί στην καθιερωμένη ομαδοποίηση των παράλληλων αλγορίθμων επιλογής σε ταξινόμηση, μερική ταξινόμηση και μεθόδους διαμέρισης [3]. Πέρα από τη θεωρητική πολυπλοκότητα, καθορίζει το ποιο υλικό εκτελεί αποδοτικά τον αλγόριθμο, αφού οι παράλληλες αρχιτεκτονικές προϋποθέτουν πολλά στοιχεία επεξεργασίας να εκτελούν την ίδια εντολή. Καθορίζει επίσης το αν ο χρόνος εκτέλεσης είναι προβλέψιμος.

2.2.1 Δίκτυα διτονικής ταξινόμησης

Ένα δίκτυο διτονικής ταξινόμησης είναι μία σταθερή ακολουθία συγκριτών. Κάθε συγκριτής δέχεται δύο θέσεις του πίνακα, συγκρίνει τα περιεχόμενά τους και τα ανταλλάσσει αν βρίσκονται σε λάθος σειρά. Το ποιες θέσεις συγκρίνει καθορίζεται εκ των προτέρων, ανεξάρτητα από τις τιμές. Είναι τυπικό παράδειγμα αλγορίθμου ανεξάρτητου από τα δεδομένα. Η ίδια ακολουθία συγκρίσεων εκτελείται πάντα, αλλάζει μόνο το αποτέλεσμα κάθε σύγκρισης.

Οι συγκριτές οργανώνονται σε επίπεδα. Εντός ενός επιπέδου καμία θέση δεν εμφανίζεται σε δύο συγκριτές, άρα μπορούν να εκτελεστούν ταυτόχρονα όλοι. Τα επίπεδα, πάντως, διαδέχονται το ένα το άλλο και το πλήθος τους αποτελεί το βάθος του δικτύου, που ορίζει το κάτω φράγμα του χρόνου σε απειριόριστα παράλληλο υλικό.

Η κατασκευή του Batcher στηρίζεται στη διτονική ακολουθία [4] που πρώτα αυξάνει, μετά μειώνεται, ή προκύπτει από κυκλική ολίσθηση μιας τέτοιας ακολουθίας. Μια διτονική ακολουθία μήκους $2m$ ταξινομείται με $\log_2(2m)$ επίπεδα. Συγκρίνοντας κάθε στοιχείο με εκείνο που απέχει m θέσεις προκύπτουν δύο διτονικές ακολουθίες μήκους m , όπου κάθε στοιχείο της πρώτης είναι μικρότερο ή ίσο από κάθε στοιχείο της δεύτερης. Οι δύο μισές ταξινομούνται στη συνέχεια ανεξάρτητα, με την ίδια διαδικασία, αναδρομικά.

Η πλήρης ταξινόμηση n στοιχείων προκύπτει με διαδοχική εφαρμογή της παραπάνω ιδέας. Ζεύγη ταξινομημένων μπλοκ αντίθετης φοράς σχηματίζουν διτονική ακολουθία διπλάσιου μήκους, η οποία ταξινομείται και δίνει μπλοκ διπλάσιου μεγέθους. Το δίκτυο έχει $\log_2 n (\log_2 n + 1)/2$ επίπεδα και $n/2$ συγκριτές ανά επίπεδο ($O(n \log^2 n)$ συνολικά), παράγοντα $\log n$ περισσότερο από το $O(n \log n)$ των σειριακών αλγορίθμων ταξινόμησης. Ως αντιστάθμισμα προσφέρει πλήρως προκαθορισμένες, ομοιόμορφες συγκρίσεις, χωρίς καμία απόκλιση στη ροή ελέγχου. Αυτό εκτελείται εξίσου καλά σε διανυσματικές μονάδες, σε ΣΙΜΤ, ή σε διατάξεις ροής δεδομένων, γι' αυτό παραμένει ανταγωνιστικό σε παράλληλο υλικό παρά την ασυμπτωτική του μειονεξία.

2.2.2 Περικομμένη διτονική ταξινόμηση

Η περικομμένη διτονική ταξινόμηση απορρίπτει στοιχεία μόλις αποδειχθεί ότι δεν μπορούν να ανήκουν στο αποτέλεσμα [2]. Η ίδια ιδέα, της μείωσης του ενεργού πίνακα στο μισό σε κάθε βήμα, αποτελεί τη βάση και των υλοποιήσεων μερικής ταξινόμησης για μονάδες γραφικών [5].

Έστω k' η μικρότερη δύναμη του δύο που είναι μεγαλύτερη ή ίση του k . Η διαδικασία έχει δύο φάσεις. Στην πρώτη εκτελείται κανονική διτονική ταξινόμηση, αλλά σταματά μόλις σχηματιστούν ταξινομημένα μπλοκ μεγέθους k' (όχι στο μοναδικό μπλοκ μεγέθους n). Στη δεύτερη τα μπλοκ συγχωνεύονται ανά ζεύγη και από τα $2k'$ στοιχεία κάθε ζεύγους κρατούνται μόνο τα k' μεγαλύτερα, ενώ τα υπόλοιπα απορρίπτονται οριστικά. Το βήμα αυτό υποδιπλασιάζει το μέγεθος του ενεργού πίνακα και επαναλαμβάνεται έως ότου απομείνει ένα μπλοκ, το οποίο περιέχει το ζητούμενο αποτέλεσμα. Το σχήμα 2.2 αντιπαραθέτει τις δύο δομές. Η ορθότητα της απόρριψης είναι άμεση. Αν ένα στοιχείο δεν ανήκει στα k' μεγαλύτερα δύο ταξινομημένων μπλοκ, υπάρχουν ήδη k' στοιχεία μεγαλύτερά του στον πίνακα, άρα δεν ανήκει ούτε στα k μεγαλύτερα του συνόλου.

Η πρώτη φάση χρειάζεται $\log_2 k' (\log_2 k' + 1)/2$ επίπεδα, πλήθος που εξαρτάται πλέον από το k , αντί για $\log_2 n (\log_2 n + 1)/2$. Η δεύτερη εφαρμόζεται πάνω σε πίνακα που συρρικνώνεται γεωμετρικά, άρα το κόστος της αθροίζεται σε γραμμικό όρο ως προς n . Η συνολική πολυπλοκότητα πέφτει από $O(n \log^2 n)$ σε $O(n \log^2 k)$.

Για παράδειγμα, με $n = 2^{24}$ και k ίσο με 8, το πλήρες δίκτυο έχει 300 επίπεδα και 2 516 582 400 συγκριτές, ενώ το περικομμένο 90 επίπεδα και 92 274 668 συγκριτές, δηλαδή είκοσι επτά φορές λιγότερους. Αντίθετα, για $k = 131072$ απαιτούνται 279 επίπεδα και 1 441 595 392 συγκριτές, μόλις 1,7 φορές λιγότεροι από το πλήρες δίκτυο. Το όφελος της περικοπής συρρικνώνεται καθώς το k μεγαλώνει.

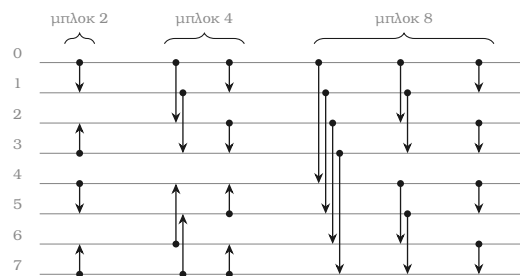
Ο συνολικός αριθμός συγκριτών του περικομμένου δικτύου δίνεται από

$$\frac{n}{2} \cdot \frac{\log_2 k' (\log_2 k' + 1)}{2} + \frac{(\log_2 k' + 2)(n - k')}{2},$$

όπου ο πρώτος όρος αντιστοιχεί στην πρώτη φάση και ο δεύτερος στη συγχώνευση-περικοπή.

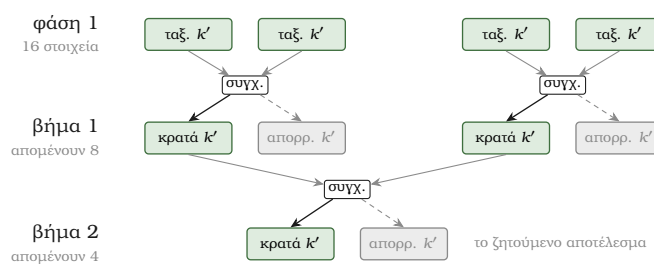
Η περικοπή διατηρεί την ανεξαρτησία από τα δεδομένα. Το χρονοδιάγραμμα των επιπέδων υπολογίζεται εκ των προτέρων αποκλειστικά από τα n και k , άρα το πόσα στοιχεία απορρίπτονται σε κάθε βήμα είναι γνωστό εκ των προτέρων, ακόμη κι αν το ποια εξαρτάται από τις τιμές.

(α) πλήρες διτονικό δίκτυο, $n = 8$



Το βέλος δείχνει πού τοποθετείται το μεγαλύτερο από τα δύο στοιχεία.

(β) περικομμένο δίκτυο, $n = 16$, $k' = 4$



Κάθε συγχώνευση δέχεται $2k'$ στοιχεία και κρατά τα k' μεγαλύτερα· τα υπόλοιπα (διακεκομμένη) απορρίπτονται οριστικά.

Σχήμα 2.2: Το πλήρες διτονικό δίκτυο και η περικοπή του. Στο (α) κάθε οριζόντια γραμμή είναι μια θέση του πίνακα και κάθε κατακόρυφο βέλος μια σύγκριση-ανταλλαγή. Το δίκτυο είναι ανεξάρτητο από τα δεδομένα, καθώς εκτελεί πάντοτε την ίδια ακολουθία συγκρίσεων με την ίδια σειρά, όποιες και αν είναι οι τιμές. Στο (β), όταν ζητούνται μόνο τα k' καλύτερα στοιχεία, η πρώτη φάση σταματά μόλις σχηματιστούν ταξινομημένα μπλοκ μεγέθους k' , και στη δεύτερη κάθε συγχώνευση ζεύγους κρατά μόνο τα k' καλύτερα από τα $2k'$ που συγκρίθηκαν. Το μέγεθος του πίνακα υποδιπλασιάζεται σε κάθε βήμα, οπότε τα επόμενα βήματα κοστίζουν ολοένα λιγότερο.

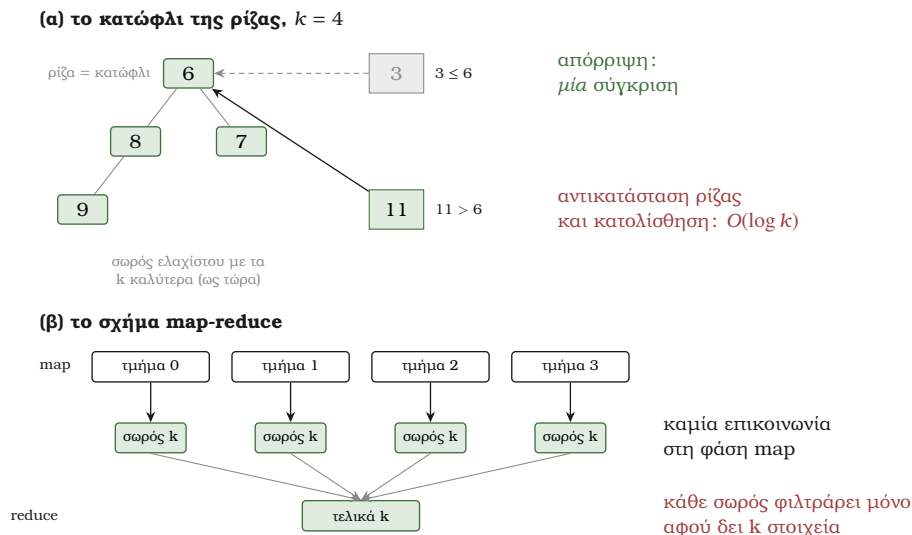
2.2.3 Επιλογή με σωρούς και map-reduce

Η επιλογή με σωρούς ακολουθεί διαφορετική λογική από τα δίκτυα ταξινόμησης. Δεν επιβάλλει σταθερό χρονοδιάγραμμα και διατηρεί μια δομή με τα k μεγαλύτερα στοιχεία που έχουν εξεταστεί μέχρι στιγμής και την ενημερώνει μόνο όταν χρειάζεται.

Η δομή είναι σωρός ελαχίστου χωρητικότητας k . Η ρίζα του συγκρατεί το μικρότερο από τα k τρέχοντα καλύτερα στοιχεία και λειτουργεί ως κατώφλι. Κάθε νέο στοιχείο συγκρίνεται με τη ρίζα με μία μόνο σύγκριση και αν δεν την υπερβαίνει απορρίπτεται χωρίς άλλη εργασία. Διαφορετικά αντικαθιστά τη ρίζα, και ο σωρός αναδιατάσσεται σε $O(\log k)$ βήματα.

Σε τυχαία είσοδο το κατώφλι ανεβαίνει γρήγορα και η συντριπτική πλειονότητα των στοιχείων απορρίπτεται με μία σύγκριση, οπότε ο χρόνος πλησιάζει το γραμμικό. Σε αύξουσα είσοδο, όμως, κάθε νέο στοιχείο υπερβαίνει τη ρίζα και επιβάλλει αναδιάταξη, δίνοντας $O(n \log k)$ στη χειρότερη περίπτωση.

Η παραλληλοποίηση ακολουθεί το σχήμα map-reduce (σχήμα 2.3). Στη φάση map η είσοδος διαμερίζεται και κάθε εργάτης διατηρεί δικό του τοπικό σωρό, χωρίς συγχρονισμό με τους υπόλοιπους, και στη φάση reduce τα μερικά αποτελέσματα συγχωνεύονται σε ένα. Το σχήμα είναι ελκυστικό επειδή η φάση map δεν απαιτεί καμία επικοινωνία. Κάθε τοπικός σωρός πρέπει πρώτα να γεμίσει με k στοιχεία για να αρχίσει να φιλτράρει, οπότε το πλήθος των εργατών που συνεισφέρουν πραγματικά δεν ξεπερνά τον λόγο n/k . Σε επεξεργαστή με οκτώ νήματα το όριο αυτό περνάει απαρατήρητο, μα σε μονάδα γραφικών με χιλιάδες νήματα γίνεται ο κύριος περιοριστικός παράγοντας, όπως αναλύεται στην ενότητα 5.3.1.



Σχήμα 2.3: Η επιλογή με σωρούς και η παραλληλοποίησή της. Στο (α) η ρίζα του σωρού ελαχίστου λειτουργεί ως κατώφλι: το στοιχείο που δεν την υπερβαίνει απορρίπτεται με μία σύγκριση, ενώ εκείνο που την υπερβαίνει την αντικαθιστά και επιβάλλει κατολίσθηση. Στο (β) η είσοδος διαμερίζεται σε τμήματα, ένας σωρός ανά εργάτη και τα μερικά αποτελέσματα συγχωνεύονται σε ένα.

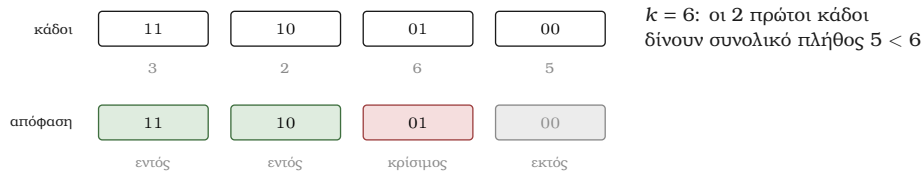
2.2.4 Επιλογή με βάση την αναπαράσταση

Η επιλογή με βάση την αναπαράσταση εξετάζει τη δυαδική μορφή των τιμών αντί να τις συγκρίνει απευθείας. Οι τιμές κατανέμονται σε κάδους ανάλογα με μια ομάδα ψηφίων. Τα πλήθη των κάδων αθροίζονται και από αυτά προκύπτει σε ποιον κάδο βρίσκεται το k -οστό μεγαλύτερο στοιχείο. Οι υπερκείμενοι κάδοι ανήκουν εξ ολοκλήρου στο αποτέλεσμα, οι υποκείμενοι απορρίπτονται εξ ολοκλήρου και η αναζήτηση συνεχίζεται μόνο εντός του κρίσιμου κάδου με την επόμενη ομάδα ψηφίων (σχήμα 2.4).

Το πλήθος των περασμάτων καθορίζεται αποκλειστικά από το πλάτος του τύπου δεδομένων. Γι' αυτό η μέθοδος δεν έχει σημείο κατάρρευσης καθώς το k μεγαλώνει. Κάθε πέραςμα είναι επιπλέον ακολουθιακή ανάγνωση με μέτρηση συχνοτήτων, μοτίβο προσιτό για αρχιτεκτονικές υψηλού εύρους ζώνης.

Η μέθοδος εξαρτάται κι αυτή από τα δεδομένα, αλλά με τρόπο διαφορετικό από τους σωρούς: εδώ ο χρόνος εξαρτάται από την κατανομή των ψηφίων στις τιμές. Όταν πολλά στοιχεία μοιράζονται τα ίδια σημαντικά ψηφία, ο κρίσιμος κάδος στενεύει αργά και χρειάζονται περισσότερα περάσματα· είναι η γνωστή στη βιβλιογραφία *radix-adversarial* κατανομή, που αντιμετωπίζεται με προσαρμοστικές στρατηγικές αποθήκευσης υποψηφίων [3]. Η οικογένεια αυτή δεν υλοποιήθηκε στην παρούσα εργασία, χρησιμοποιείται όμως ως βάση της βιβλιοθήκης αναφοράς της μονάδας γραφικών (ενότητα 2.4).

πέρασμα 1: τα 2 σημαντικότερα ψηφία



πέρασμα 2



Σχήμα 2.4: Η επιλογή με βάση την αναπαράσταση. Σε κάθε πέραςμα τα στοιχεία κατανέμονται σε κάδους ανάλογα με μια ομάδα ψηφίων και μετράται το πλήθος καθενός· από τα αθροιστικά πλήθη προκύπτει ποιοι κάδοι ανήκουν εξ ολοκλήρου στο αποτέλεσμα, ποιοι απορρίπτονται και ποιος είναι ο *κρίσιμος*. Καμία σύγκριση μεταξύ τιμών δεν εκτελείται ποτέ.

2.3 Αρχιτεκτονικές επεξεργασίας

Οι τρεις αρχιτεκτονικές εκθέτουν παραλληλία με διαφορετικούς τρόπους.

2.3.1 Διανυσματικές επεκτάσεις CPU

Ο σύγχρονος επεξεργαστής γενικού σκοπού συνδυάζει δύο μορφές παραλληλίας: πολλαπλά νήματα, καθένα με δική του ανεξάρτητη ροή εντολών, και διανυσματικές επεκτάσεις, που εφαρμόζουν την ίδια πράξη σε πολλά δεδομένα ταυτόχρονα εντός ενός νήματος.

Ένας καταχωρητής AVX-512 έχει πλάτος 512 bit, δηλαδή δεκαέξι ακέραιους των 32 bit. Πέρα από τις αριθμητικές πράξεις, το ρεπερτόριο περιλαμβάνει εντολές αναδιάταξης, που μεταθέτουν στοιχεία εντός του καταχωρητή, και εντολές ελαχίστου/μεγίστου, που επιλέγουν κατά στοιχείο χωρίς διακλάδωση. Ο συνδυασμός αναδιάταξης, ελαχίστου και μεγίστου ισοδυναμεί με έναν συγκριτή δικτύου ταξινόμησης. Για αυτό η διτονική ταξινόμηση είναι φυσικός υποψήφιος για διανυσματοποίηση.

Η τροφοδοσία δεδομένων γίνεται περιοριστική συνήθως πρώτη, πριν προλάβει να γίνει πρόβλημα ο ρυθμός των πράξεων. Η ιεραρχία κρυφών μνημών αντισταθμίζει τη διαφορά ταχύτητας από την κύρια μνήμη, αλλά μόνο για αλγόριθμους που επαναχρησιμοποιούν τα δεδομένα όσο παραμένουν σε κοντινό επίπεδο.

2.3.2 Μοντέλο SIMT και ιεραρχία μνήμης GPU

Η μονάδα γραφικών οργανώνεται γύρω από το μοντέλο SIMT. Τα νήματα ομαδοποιούνται σε δέσμες που εκτελούν την ίδια εντολή σε διαφορετικά δεδομένα. Όταν τα νήματα μιας δέσμης ακολουθήσουν διαφορετικούς κλάδους, οι κλάδοι εκτελούνται σειριακά με τα ανενεργά νήματα απενεργοποιημένα. Η απόκλιση ροής ελέγχου κοστίζει επομένως άμεσα και πλεονεκτούν οι αλγόριθμοι χωρίς διακλαδώσεις.

Οι δέσμες ομαδοποιούνται σε ομάδες νημάτων και οι ομάδες σε πλέγμα. Κάθε ομάδα διαθέτει κοινή μνήμη, μια μικρή περιοχή διαχειριζόμενη ρητά από το λογισμικό, με καθυστέρηση πολύ μικρότερη της καθολικής μνήμης. Με αυτήν, ένας αλγόριθμος πολλών περασμάτων μετατρέπεται σε αλγόριθμο ενός περάσματος· αν το τμήμα δεδομένων που επεξεργάζεται μια ομάδα χωρά εκεί, όλα τα ενδιαμέσα βήματα εκτελούνται χωρίς επιστροφή στην καθολική μνήμη.

Η πρόσβαση στην καθολική μνήμη είναι αποδοτική μόνο όταν τα νήματα μιας δέσμης ζητούν διαδοχικές διευθύνσεις, ώστε οι επιμέρους αιτήσεις να συγχωνεύονται σε λίγες συναλλαγές. Τέλος, η μονάδα γραφικών είναι διακριτή συσκευή. Τα δεδομένα πρέπει να μεταφερθούν από και προς τη μνήμη της μέσω διαύλου με εύρος ζώνης τάξη μεγέθους μικρότερο από εκείνο της τοπικής μνήμης.

2.3.3 Αρχιτεκτονική ροής δεδομένων ΑΙΕ

Ο όρος μονάδα νευρωνικής επεξεργασίας σήμαινε διαφορετικά πράγματα ανά εποχή. Οι πρώτες προτάσεις στόχευαν σε προσεγγιστικό υπολογισμό για εξοικονόμηση ενέργειας [6, 7]. Ακολούθησαν αφιερωμένοι επιταχυντές νευρωνικών δικτύων [8], των οποίων το πλεονέκτημα σε επιδόσεις ανά Watt τεκμηριώθηκε στην ανάπτυξή τους σε κλίμακα κέντρου δεδομένων [9].

Κατά την αρχή σχεδίασης Eyeriss, η μετακίνηση ενός δεδομένου κοστίζει ενεργειακά τάξεις μεγέθους περισσότερο από την πράξη που θα εκτελεστεί πάνω του, γι' αυτό η σχεδίαση οργανώνεται γύρω από τη μεγιστοποίηση της τοπικής επαναχρησιμοποίησης των δεδομένων [10]. Στη συνέλιξη και στον πολλαπλασιασμό πινάκων η παραδοχή αυτή δικαιώνεται πλήρως, αφού κάθε στοιχείο που φορτώνεται συμμετέχει σε πολλές πράξεις. Στην επιλογή top-k κάθε στοιχείο εξετάζεται ελάχιστες φορές, οπότε η ίδια παραδοχή δεν κρατά.

Η αρχιτεκτονική που εξετάζεται ανήκει στην οικογένεια των AI Engines της AMD που εισήχθη με τα προσαρμοστικά συστήματα Versal [11] και ενσωματώθηκε στη συνέχεια,

ως XDNA, σε επεξεργαστές γενικής χρήσης [12].

Το εκτελεστικό μοντέλο είναι χωρικό, εν γένει διαφορετικό από τα δύο προηγούμενα. Η μονάδα αποτελείται από διάταξη πλακιδίων, καθένα με δικό του διανυσματικό επεξεργαστή και δική του τοπική μνήμη. Δεν υπάρχει κοινή μνήμη ούτε κρυφή μνήμη, οπότε η επικοινωνία γίνεται αποκλειστικά με ρητή μεταφορά δεδομένων μέσω δικτύου διασύνδεσης.

Το πρόγραμμα εκφράζεται επομένως ως γράφος υπολογιστικών πυρήνων συνδεδεμένων με κανάλια ροής αντί για ακολουθία πράξεων πάνω σε κοινό χώρο διευθύνσεων. Ο προγραμματιστής περιγράφει ποια πλακίδια εκτελούν ποιον πυρήνα και πώς ρέουν τα δεδομένα μεταξύ τους. Η αφαίρεση που χρησιμοποιεί η εργασία για τον σκοπό αυτό είναι το `object_fifo` του πλαισίου MLIR-AIE [13], το οποίο παρουσιάζεται στην ενότητα 6.1.2.

Το μέγεθος της τοπικής μνήμης κάθε πλακιδίου είναι μικρό και σταθερό, άρα ο υπολογισμός πρέπει να τεμαχιστεί σε τμήματα που χωρούν εκεί, με τα μερικά αποτελέσματα να συγχωνεύονται αλλού. Επιπλέον, η τροφοδοσία της διάταξης παραμένει ευθύνη του `host`. Εκείνος πρέπει να προετοιμάσει και να προωθήσει κάθε τμήμα. Όταν ο υπολογισμός ανά στοιχείο είναι μικρός, όπως συμβαίνει στην επιλογή, το κόστος της τροφοδοσίας κυριαρχεί έναντι του κόστους του υπολογισμού.

2.4 Υπάρχουσες υλοποιήσεις και βιβλιοθήκες

Η διαθεσιμότητα έτοιμων υλοποιήσεων δεν είναι ίδια στις τρεις αρχιτεκτονικές και η ανισότητα αυτή διαμορφώνει τόσο το ερώτημα της εργασίας όσο και την επιλογή των μέτρων σύγκρισης της ενότητας 3.5.

Για τον επεξεργαστή, η πρότυπη βιβλιοθήκη της C++ προσφέρει τη `std::nth_element`, η οποία διαμερίζει τον πίνακα γύρω από το k -οστό στοιχείο σε αναμενόμενο γραμμικό χρόνο, και τη `std::partial_sort`, η οποία επιπλέον ταξινομεί το αποτέλεσμα διατηρώντας σωρό μεγέθους k . Και οι δύο είναι διαθέσιμες παντού και δοκιμασμένες, αλλά αυστηρά μονονηματικές, χωρίς αξιοποίηση πολλαπλών νημάτων ή διανυσματικών επεκτάσεων.

Στη μονάδα γραφικών, η CUB του συνόλου βιβλιοθηκών CCCL [14] παρέχει τη `DeviceTopK`, υλοποίηση επιλογής με βάση την αναπαράσταση κατά την ενότητα 2.2.4. Είναι ώριμη και ιδιαίτερα βελτιστοποιημένη λύση, την οποία η εργασία χρησιμοποιεί ως μέτρο σύγκρισης, χωρίς να έχει ως αυτοσκοπό την υπέρβασή της σε ολόκληρο το εύρος των παραμέτρων.

Η ένταση με την οποία εξαρτάται η επιλογή αλγορίθμου από το k αναγνωρίζεται και στη βιομηχανία. Πρόσφατη εργασία για μονάδες γραφικών AMD καταγράφει ακριβώς την ίδια ανατροπή που περιγράφει η ενότητα 7.2.2 και την αντιμετωπίζει με προσαρμωστική στρατηγική: διτονική ταξινόμηση σε επίπεδο καταχωρητών για μικρό k και επιλογή με βάση την αναπαράσταση για μεγάλο, με εμπειρικά προσδιορισμένο σημείο μετάβασης [15].

Παράλληλα με τις ακριβείς μεθόδους υπάρχει και γραμμή προσεγγιστικών αλγορίθμων, που θυσιάζουν ανάκληση για ταχύτητα και στοχεύουν κυρίως φορτία μηχανικής μάθησης, όπου η επιλογή `top-k` αποδεικνύεται συστηματικά στενωπός σε επιταχυντές

σχεδιασμένους για πυκνό πολλαπλασιασμό πινάκων [16]. Η παρούσα εργασία περιορίζεται στην ακριβή επιλογή.

Για τη μονάδα νευρωνικής επεξεργασίας δεν υπάρχει αντίστοιχη βιβλιοθήκη: το οικοσύστημά της είναι προσανατολισμένο σε τελεστές νευρωνικών δικτύων και δεν περιλαμβάνει πρωτογενή επιλογή top-k. Το πλησιέστερο διαθέσιμο είναι ένα επίσημο παράδειγμα διτονικής ταξινόμησης σε πλακίδιο AI Engine [17], το οποίο όμως εκτελεί πλήρη ταξινόμηση, μόνο για αριθμούς απλής ακρίβειας, και μόνο για μεγέθη έως 1024 στοιχεία που χωρούν στην τοπική μνήμη ενός πλακιδίου. Το κενό που καλείται να καλύψει η παρούσα εργασία είναι ακριβώς αυτό: επιλογή αντί για ταξινόμηση, πέντε τύποι δεδομένων αντί για έναν και είσοδοι έως 2^{24} στοιχείων που απαιτούν διοχέτευση μέσω πολλαπλών πλακιδίων.

Κάθε υλοποίηση πρέπει να κατασκευαστεί από το μηδέν στο επίπεδο του MLIR-AIE [13], γεγονός που καθιστά τη μονάδα το πιο απαιτητικό μέρος της εργασίας. Επιπλέον, δεν υπάρχει φυσικό μέτρο σύγκρισης επί της συσκευής, οπότε η εργασία χρησιμοποιεί ως αναφορά την υλοποίηση του host, επιλογή που τεκμηριώνεται στην ενότητα 6.5.

Η βιβλιογραφία για την επιλογή top-k σε μονάδες γραφικών είναι συνολικά εκτενής και καλύπτει τόσο δίκτυα ταξινόμησης, όσο και μεθόδους βασισμένες στην αναπαράσταση [2, 15]. Αντίθετα, η αντίστοιχη βιβλιογραφία για επιταχυντές ροής δεδομένων τύπου AIE είναι περιορισμένη, καθώς οι μονάδες αυτές είναι πρόσφατες και η χρήση τους εστιάζει σε φορτία συνέλιξης και πολλαπλασιασμού πινάκων.

Κεφάλαιο 3

Μεθοδολογία και πειραματική διάταξη

3.1 Σύστημα αναφοράς

Όλες οι μετρήσεις προέρχονται από ένα και μόνο σύστημα, που φέρει και τις τρεις αρχιτεκτονικές. Έτσι απομονώνονται οι διαφορές των υλοποιήσεων από τις διαφορές των πλατφορμών. Το κόστος είναι πως το σύστημα είναι φορητό, επομένως οι τρεις μονάδες μοιράζονται κοινό θερμικό και ενεργειακό όριο.

Σύστημα	Lenovo Legion 5 15AHP10
Λειτουργικό σύστημα	Arch Linux x86_64, πυρήνας 7.1.5-arch1-2
Κύρια μνήμη	32 GiB DDR5-5600, δύο κανάλια
CPU	AMD Ryzen 7 260
Πυρήνες / νήματα	8 / 16, έως 5.10 GHz
Κρυφή μνήμη	8 MiB L2, 16 MiB L3
Διανυσματικές επεκτάσεις	AVX-512
GPU	NVIDIA GeForce RTX 5060 Laptop
Μνήμη	8 GiB
Λογισμικό	driver 610.43.03, CUDA 13.3
Όρια ισχύος	50 W προεπιλογή, 115 W μέγιστο
NPU	AMD XDNA (RyzenAI-npu1)
Λογισμικό	XRT 2.21.75, firmware 1.5.5.391

Πίνακας 3.1: Το σύστημα αναφοράς στο οποίο εκτελέστηκαν όλες οι μετρήσεις.

Ο επεξεργαστής διαθέτει οκτώ φυσικούς πυρήνες με ταυτόχρονη πολυνηματικότητα. Σε όλες τις μετρήσεις χρησιμοποιείται ένα νήμα ανά φυσικό πυρήνα, ώστε δύο νήματα να μη μοιράζονται την ίδια διανυσματική μονάδα. Η ιεραρχία κρυφής μνήμης καθορίζει τα σημεία καμπής της μετρημένης κλίμακας εύρους ζώνης (ενότητα 3.4). Ανάλογα με το μέγεθος εισόδου και τον τύπο δεδομένων, το σύνολο εργασίας μπορεί να χωρά στην L2, στην L3, ή μόνο στην κύρια μνήμη, και κάθε υλοποίηση συγκρίνεται με το ταβάνι του δικού της συνόλου εργασίας.

Η κύρια μνήμη αποτελείται από δύο πανομοιότυπες μονάδες των 16 GiB σε διπλό

κανάλι, στα 5600 MT/s. Ένα μόνο κανάλι θα υποδιπλασίαζε το εύρος ζώνης, και μια υλοποίηση περιορισμένη από τη μνήμη θα έδινε χειρότερα αποτελέσματα σε ένα κατά τα άλλα ίδιο σύστημα.

Η μονάδα νευρωνικής επεξεργασίας δεν έχει δική της μνήμη. Τροφοδοτείται από την ίδια DDR5, μέσω των ίδιων ελεγκτών μνήμης με τον επεξεργαστή, στην ίδια ψηφίδα πυριτίου, με δύο συνέπειες που θα επανέρχονται σε όλη την εργασία. Το κόστος προετοιμασίας δεδομένων στον host δεν διαχωρίζεται από το κόστος υπολογισμού και ο μετρητής ενέργειας καλύπτει ολόκληρο το πακέτο, όχι τα πλακίδια ΑΙΕ ξεχωριστά (ενότητα 3.3.3).

Τα στοιχεία του πίνακα 3.1 μετρήθηκαν απευθείας στο σύστημα.

3.2 Δομή της υποδομής μετρήσεων

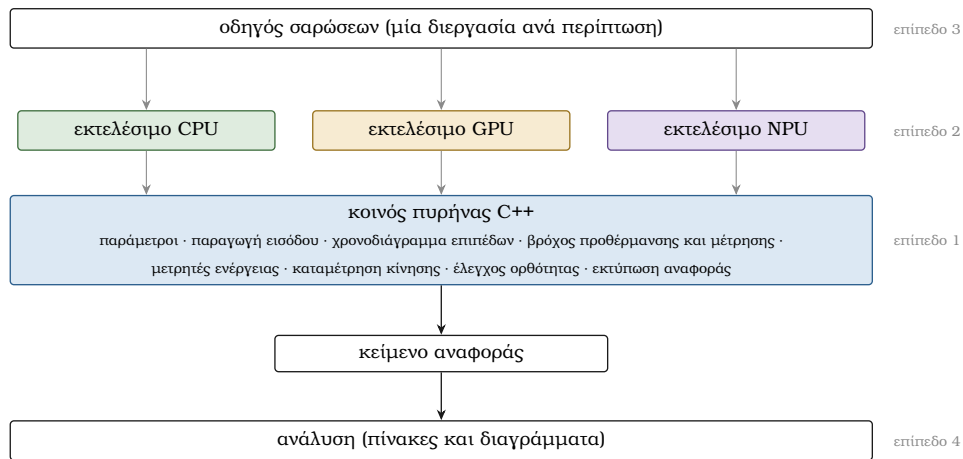
Η υποδομή οργανώνεται σε τέσσερα επίπεδα γύρω από μία αρχή: η ίδια η διαδικασία μέτρησης είναι κοινή σε όλες τις αρχιτεκτονικές, μόνο ο πυρήνας διαφέρει. Η ίδια η υλοποίηση δεν επηρεάζει τον τρόπο μέτρησής της.

Στο πρώτο επίπεδο βρίσκεται ο κοινός πυρήνας σε C++, μοιρασμένος και στις τρεις υλοποιήσεις. Διαβάζει παραμέτρους και παράγει την είσοδο, χρονοδιαγραμματίζει τα επίπεδα του διτονικού δικτύου και διαβάζει τους μετρητές ενέργειας. Στη συνέχεια τρέχει τον βρόχο προθέρμανσης και μέτρησης, ελέγχει την ορθότητα και τυπώνει την τελική αναφορά. Όλα αυτά συμβαίνουν σε ένα σημείο, ώστε χρονομετρήσεις και μετρήσεις ενέργειας κάθε αρχιτεκτονικής να ορίζονται από τον ίδιο κώδικα.

Στο δεύτερο βρίσκονται τα εκτελέσιμα ανά αρχιτεκτονική, καθένα με την ίδια διεπαφή για τους τρεις συγκρινόμενους αλγόριθμους (διτονική ταξινόμηση, map-reduce με τοπικούς σωρούς, υλοποίηση αναφοράς), παρέχοντας μόνο τη συνάρτηση υπολογισμού που καλεί ο κοινός βρόχος μέτρησης.

Το τρίτο επίπεδο είναι ο οδηγός των σαρώσεων. Απαριθμεί τον παραμετρικό χώρο της ενότητας 3.7, εκτελεί κάθε περίπτωση σε ξεχωριστή διεργασία, καταγράφει αυτούσια την έξοδο μαζί με την έκβαση. Κάθε περίπτωση ξεκινά από καθαρή μνήμη, και αν κάποια αποτύχει η σάρωση συνεχίζει κανονικά στις υπόλοιπες.

Το τέταρτο επίπεδο μετατρέπει τα ακατέργαστα αποτελέσματα σε πίνακες και διαγράμματα, ενσωματώνοντάς τα απευθείας στην τελική αναφορά. Δεν υπάρχει χωριστή διαδρομή εξόδου, οι τιμές στα διαγράμματα είναι εξ ορισμού όσες τύπωσε το εκτελέσιμο. Το περιβάλλον εκτέλεσης αποθηκεύεται μαζί με τα αποτελέσματα (ενότητα 3.8) και οι εντολές κάθε σταδίου δίνονται στο παράρτημα Γ'. Η δομή συνοψίζεται στο σχήμα 3.1.



Σχήμα 3.1: Τα τέσσερα επίπεδα της υποδομής μετρήσεων. Η διαδικασία της μέτρησης ορίζεται μία μόνο φορά, στον κοινό πυρήνα του πρώτου επιπέδου, και οι τρεις υλοποιήσεις παρέχουν αποκλειστικά τη συνάρτηση που εκτελεί τον υπολογισμό.

3.3 Μετρούμενα μεγέθη

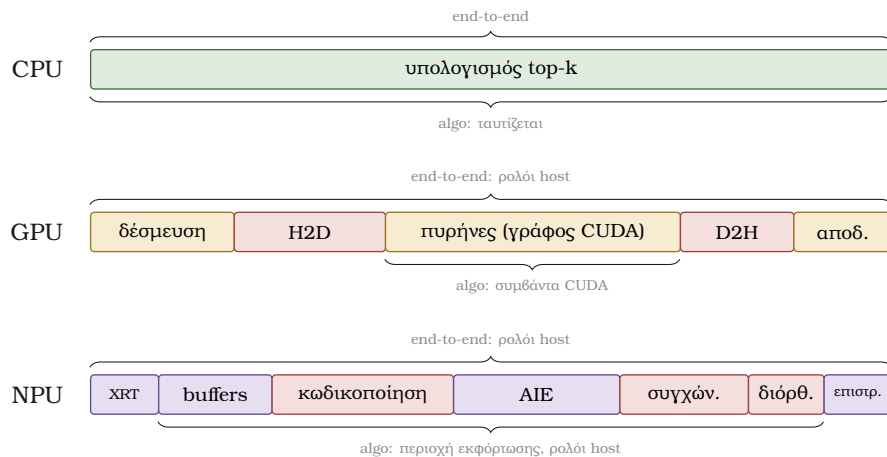
3.3.1 Πεδία μέτρησης: algo και end-to-end

Ένας επιταχυντής μπορεί να τελειώνει τον υπολογισμό ταχύτατα και παρ' όλα αυτά να χάνει από έναν επεξεργαστή που δουλεύει επιτόπου, αν χρειάζεται να μεταφέρει ολόκληρη την είσοδο από και προς την κύρια μνήμη. Γι' αυτό κάθε περίπτωση μετριέται σε δύο πεδία.

Το end-to-end περιλαμβάνει ολόκληρη την κλήση από την πλευρά του host· όλα όσα χρειάζονται ώστε, δεδομένης μιας εισόδου στην κύρια μνήμη, το αποτέλεσμα να βρεθεί κι αυτό εκεί. Το algo περιλαμβάνει μόνο την περιοχή όπου εκτελείται ο υπολογισμός. Η διαφορά τους είναι το κόστος της ίδιας της ετερογένειας, όπως συνοψίζεται στον πίνακα 3.2 και στο σχήμα 3.2.

Αρχιτεκτονική	Περιεχόμενο του algo	Διαφορά από το end-to-end
CPU	ταυτίζεται με το end-to-end	-
GPU	περιοχή μεταξύ συμβάντων CUDA	μεταφορές H2D/D2H, δε- σμεύσεις μνήμης
NPU	περιοχή της εκφόρτωσης	εντοπισμός συσκευής, φόρ- τωση ρύθμισης εκφόρτωσης

Πίνακας 3.2: Το περιεχόμενο των δύο πεδίων μέτρησης ανά αρχιτεκτονική.



Σχήμα 3.2: Τα δύο πεδία μέτρησης ανά αρχιτεκτονική. Το χρώμα κάθε σταδίου δηλώνει σε ποια μονάδα ανήκει η εργασία του, με κόκκινο για το επιπλέον κόστος που επιβάλλει η κάθε αρχιτεκτονική: μεταφορές διαύλου στη μονάδα γραφικών, κωδικοποίηση, συγχώνευση και τελική διόρθωση στον host για τη μονάδα νευρωνικής επεξεργασίας.

Στον επεξεργαστή τα δύο ταυτίζονται εξ ορισμού, αφού πρόκειται για τον ίδιο τον επιταχυντή και δεν υπάρχει κόστος host προς αφαίρεση.

Ο μετρητής ενέργειας ανοίγει και κλείνει στα ίδια ακριβώς σημεία του κώδικα με το χρονόμετρο. Χρόνος και ενέργεια αναφέρονται έτσι στην ίδια περιοχή και είναι άμεσα συγκρίσιμα. Καταγράφεται και τρίτο, ευρύτερο παράθυρο γύρω από ολόκληρο τον βρόχο μέτρησης, ως άνω φράγμα ελέγχου (ενότητα 3.3.3).

Το end-to-end είναι το δίκαιο πεδίο για σύγκριση μεταξύ αρχιτεκτονικών, αφού περιλαμβάνει το κόστος μεταφοράς για όλες. Το algo παραλείπει τις μεταφορές, οπότε χρησιμεύει μόνο για συγκρίσεις εντός της ίδιας αρχιτεκτονικής.

3.3.2 Μέτρηση χρόνου

Τα δύο πεδία δεν μετρώνται απαραίτητα με τον ίδιο μηχανισμό. Το end-to-end μετριέται πάντα με ρολόι του host, εκατέρωθεν της κλήσης, εξ ορισμού χρόνος που αντιλαμβάνεται ο host. Το algo μετριέται με ό,τι διαθέτει η κάθε αρχιτεκτονική για την περιοχή του δικού της υπολογισμού.

Στη μονάδα γραφικών η μέτρηση δεν μπορεί να γίνει με ρολόι του host. Οι πυρήνες εκκινούν ασύγχρονα και η κλήση επιστρέφει μόλις τοποθετηθεί ο πυρήνας στη ροή εντολών, πολύ πριν ολοκληρωθεί. Ένα τέτοιο ρολόι θα μετρούσε την εκκίνηση, όχι την εκτέλεση, και ο εξαναγκασμένος συγχρονισμός θα πρόσθετε δικό του κόστος. Χρησιμοποιούνται αντ' αυτού συμβάντα (events) μέσα στη ροή εντολών, χρονομετρημένα από τη συσκευή.

Στη μονάδα νευρωνικής επεξεργασίας δεν υπάρχει τέτοιος μετρητής της συσκευής. Το algo ορίζεται εδώ με ρολόι του host γύρω από την περιοχή της εκφόρτωσης. Μετρά δέσμευση και προετοιμασία buffers, αποστολή εργασιών και τελική συγχώνευση των μερικών αποτελεσμάτων. Το μέγεθος αυτό πρέπει να διαβάζεται ως κόστος ολόκληρης της εκφόρτωσης· ο υπολογισμός στα πλακίδια ΑΙΕ είναι μόνο ένα κομμάτι του.

Κάθε περίπτωση μετριέται σε πενήντα επαναλήψεις, με τρία μεγέθη αναφερόμενα ανά πεδίο: τη μέση τιμή (βάση των συγκρίσεων), την τυπική απόκλιση (ένδειξη σταθερότητας), και την ελάχιστη τιμή (κάτω φράγμα). Οι μετρητές έχουν διακριτική ικανότητα τάξης νανοδευτερολέπτου, έναντι μετρούμενων περιοχών από λίγες δεκάδες μικροδευτερόλεπτα έως εκατοντάδες χιλιοστά, οπότε το κόστος της ίδιας της χρονομέτρησης είναι αμελητέο και δεν αφαιρείται.

3.3.3 Μέτρηση ενέργειας

Το ίδιο το εκτελέσιμο μετρά την ενέργειά του. Ένα εξωτερικό εργαλείο θα μετρούσε ολόκληρη τη διεργασία, ενώ ο βρόχος μέτρησης καταλαμβάνει μόνο μέρος της, μέρος που διαφέρει συστηματικά ανά αρχιτεκτονική, εισάγοντας μεροληψία υπέρ όσων έχουν μικρότερο κόστος αρχικοποίησης.

Οι πηγές διαφέρουν ανά αρχιτεκτονική, μα το πεδίο μένει συμμετρικό. Για όλες μετριέται η ενέργεια πακέτου μέσω RAPL και για τη μονάδα γραφικών προστίθεται η ισχύς της κάρτας. Το end-to-end αντιστοιχεί έτσι στο συνολικό κόστος επίλυσης και για τις τρεις.

Αναφέρεται το άθροισμα, όχι η μέση τιμή, των 50 επαναλήψεων. Οι μετρητές RAPL κβαντίζουν κάθε ανάγνωση, αλλά μετρούν αθροιστικά χωρίς επαναφορά. Γι' αυτό το άθροισμα παραμένει ακριβές ακόμη κι όταν κάθε επιμέρους διαφορά είναι τάξης του βήματος κβαντισμού.

Χρησιμοποιείται μόνο το υποπεδίο package του RAPL. Το υποπεδίο core αποδείχθηκε αναξιόπιστο σε αυτόν τον επεξεργαστή. Κατέγραφε μόλις 6 W υπό πλήρες φορτίο οκτώ νημάτων, ενώ η ισχύς πακέτου αυξανόταν κατά περίπου 53 W.

Ο μετρητής συνολικής ενέργειας της NVML αποδείχθηκε επίσης ακατάλληλος για τη μονάδα γραφικών. Προχωρά σε βήματα περίπου 20 J κάθε λίγες δεκάδες χιλιοστά. Η οριοθέτηση ενός πυρήνα διάρκειας χιλιοστών επιστρέφει έτσι τυχαίο πολλαπλάσιο του βήματος. Τρεις πανομοιότυπες εκτελέσεις έδωσαν τιμές που διέφεραν πάνω από τέσσερις φορές, ενώ ο χρόνος διέφερε κατά 0.2 %. Αντ' αυτού ένα βοηθητικό νήμα δειγματοληπτεί τη στιγμιαία ισχύ της κάρτας κάθε 2 ms και την ολοκληρώνει ως προς τον χρόνο. Η ίδια η ένδειξη ανανεώνεται αραιότερα από την περίοδο δειγματοληψίας, οπότε ένα παράθυρο algo λίγων χιλιοστών στηρίζεται σε ένδειξη που δεν πρόλαβε να ανανεωθεί. Η άθροιση στις 50 επαναλήψεις εξομαλύνει αυτή την ανακρίβεια, χωρίς να την εξαλείφει. Οι μετρητές RAPL, που ανανεώνονται σε κλίμακα χιλιοστού, δεν έχουν αυτό το πρόβλημα.

Πέρα από τη μεικτή τιμή αναφέρεται και η καθαρή ενέργεια, μεικτή μείον όσο θα κατανάλωνε το σύστημα σε αδράνεια για το ίδιο διάστημα, με την ισχύ αδράνειας μετρημένη πριν από κάθε σάρωση. Το καθαρό μέγεθος είναι προτιμότερο ιδίως για τη μονάδα νευρωνικής επεξεργασίας, όπου κυριαρχεί η στατική κατανάλωση του υπολοίπου συστήματος, και για τη μονάδα γραφικών, όπου αναιρεί το κόστος του ίδιου του μηχανισμού δειγματοληψίας.

Ο μετρητής πακέτου καλύπτει ολόκληρη την ψηφίδα (πυρήνες host, πλακίδια NPU, uncore, ελεγκτή μνήμης, όλα κάτω από έναν κοινό μετρητή). Μια εκτέλεση στη μονάδα νευρωνικής επεξεργασίας μετριέται έτσι ως η συνολική ενέργεια του συστήματος όσο εργάζεται, συμπεριλαμβανομένης της προετοιμασίας δεδομένων στον host. Η σύγκριση

κόστους επίλυσης είναι δίκαιη και συμμετρική, δεν είναι όμως γνωστό πόση ενέργεια καταναλώνει η ίδια η μονάδα για τον υπολογισμό, αφού η δική της κατανάλωση δεν διαχωρίζεται ποτέ από εκείνη του host. Η απομόνωσή της θα απαιτούσε εξωτερική μετρητική διάταξη.

3.3.4 Μέτρηση κίνησης δεδομένων

Κάθε εκτέλεση καταγράφει επίσης τα bytes που διακίνησε και τις πράξεις που εκτέλεσε, οι οποίες αποτελούν τις συντεταγμένες της ενότητας 3.4. Ένα σφάλμα εδώ δεν δίνει απλώς ανακριβή νούμερα, αλλά οδηγεί σε εντελώς λάθος συμπέρασμα για το τι περιορίζει την υλοποίηση.

Η κίνηση δεν υπολογίζεται θεωρητικά. Καταμετράται απευθείας από τον κώδικα εκεί όπου συντελείται: ανά ομάδα συγχωνευμένων περασμάτων στον επεξεργαστή, ανά εκκίνηση πυρήνα στη μονάδα γραφικών, ανά παρτίδα δεδομένων στη μονάδα νευρωνικής επεξεργασίας. Η προφανής εκτίμηση, πλήθος στοιχείων επί μέγεθος τύπου, υποθέτει ανάγνωση κάθε στοιχείου μία φορά, υπόθεση που δεν ισχύει εδώ. Η περικυκλωμένη διτονική ταξινόμηση διατρέχει το σύνολο εργασίας επανειλημμένα, με συντελεστή περίπου επταπλάσιο της αφελούς εκτίμησης για k πολύ μικρότερο του n . Η αφελής εκτίμηση θα έκανε κάθε υλοποίηση να φαίνεται ότι αξιοποιεί μικρό μόνο κλάσμα του διαθέσιμου εύρους ζώνης, οδηγώντας εσφαλμένα στο συμπέρασμα ότι καμία δεν πλησιάζει το ταβάνι της μνήμης.

Όπου είναι εφικτό, η καταμέτρηση ελέγχεται με ανεξάρτητη επαλήθευση. Για τη διτονική ταξινόμηση με k έως 4096, ο επεξεργαστής και η μονάδα γραφικών εκτελούν το ίδιο χρονοδιάγραμμα επιπέδων και υπολογίζουν την κίνησή τους σε εντελώς ξεχωριστό κώδικα, καταλήγοντας όμως στην ίδια τιμή μέχρι το τελευταίο byte. Αυτό αποτελεί ανεξάρτητη ένδειξη ότι το μοντέλο καταμέτρησης είναι σωστό. Για το map-reduce οι δύο τιμές διαφέρουν, όπως αναμένεται. Αν και αμφότερες διατηρούν τοπικό σωρό ανά νήμα, ο επεξεργαστής εκτελεί λίγα νήματα ενώ η μονάδα γραφικών χιλιάδες, με αντίστοιχα μεγαλύτερο χώρο εργασίας και επιπλέον συγχώνευση σε δέντρο εντός κάθε ομάδας νημάτων, οπότε οι δύο υλοποιήσεις πράγματι μετακινούν διαφορετικό όγκο δεδομένων.

Η ακρίβεια της καταμέτρησης διαφέρει ανά αλγόριθμο, και κάθε εκτέλεση δηλώνει η ίδια την ποιότητα των αριθμών της. Για τη διτονική ταξινόμηση τα bytes και οι πράξεις είναι ακριβή, καθώς το χρονοδιάγραμμα είναι πλήρως απαριθμήσιμο. Για το map-reduce τόσο τα bytes όσο και το πλήθος συγκρίσεων είναι κάτω φράγμα, καθώς η κίνηση της συγχώνευσης και η αναδιάταξη των τοπικών σωρών εξαρτώνται από τα δεδομένα. Για την υλοποίηση αναφοράς και τα δύο μεγέθη είναι κάτω φράγματα, διότι τα εσωτερικά περάσματα των βιβλιοθηκών δεν είναι εξωτερικά παρατηρήσιμα.

Οι πράξεις μετρώνται σε συγκρίσεις-ανταλλαγές. Η επιλογή top- k δεν εκτελεί αριθμητικές πράξεις, άρα τα FLOP δεν έχουν εδώ νόημα.

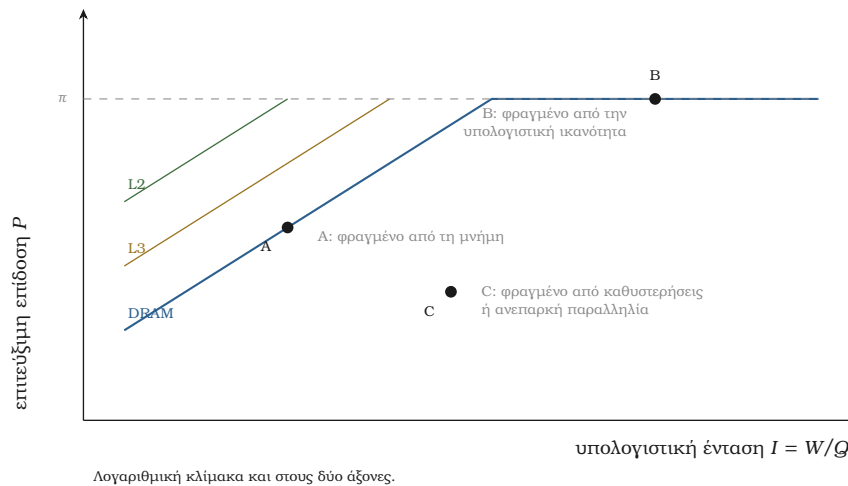
3.4 Μοντέλο roofline

Ο χρόνος εκτέλεσης από μόνος του δεν εξηγεί γιατί μια υλοποίηση δεν είναι ταχύτερη. Το μοντέλο roofline [18] απαντά σε αυτό, αντιπαραβάλλοντας κάθε μέτρηση με τα δύο ανώτατα όρια του μηχανήματος: εύρος ζώνης β και υπολογιστική ικανότητα π . Ποιο

από τα δύο δεσμεύει είναι ιδιότητα της σχέσης μηχανήματος και αλγορίθμου, όχι του ενός χωριστά, και εκφράζεται από την υπολογιστική ένταση, το πηλίκο των πράξεων W προς τα bytes Q που μετακινούνται:

$$I = \frac{W}{Q}, \quad P = \min(\pi, \beta I)$$

Το σχήμα 3.3 δείχνει πώς διαβάζεται.



Σχήμα 3.3: Η ανάγνωση του μοντέλου roofline. Το κεκλιμένο τμήμα αντιστοιχεί στο όριο βI που επιβάλλει το εύρος ζώνης και το οριζόντιο στο όριο π που επιβάλλει η υπολογιστική ικανότητα. Η επιτεύξιμη επίδοση είναι το ελάχιστο των δύο. Επειδή το εύρος ζώνης εξαρτάται από το μέγεθος του συνόλου εργασίας, το κεκλιμένο τμήμα δεν είναι μία ευθεία, αλλά μια κλίμακα από παράλληλες ευθείες, μία ανά βαθμίδα της ιεραρχίας μνήμης, και κάθε μέτρηση συγκρίνεται με εκείνη που αντιστοιχεί στο δικό της σύνολο εργασίας. Το σημείο A περιορίζεται από τη μνήμη και θα ωφεληθεί μόνο από μείωση της κίνησης δεδομένων, το B από την υπολογιστική ικανότητα, ενώ το C απέχει και από τα δύο ταβάνια. Εκεί ο περιορισμός δεν είναι το υλικό αλλά η οργάνωση της εκτέλεσης.

Η εφαρμογή του μοντέλου εδώ αποκλίνει σκόπιμα από τη συνήθη πρακτική, ώστε οι συγκρίσεις να μείνουν έντιμες.

Και τα δύο ταβάνια μετρώνται στο ίδιο μηχανήμα με χωριστά εκτελέσιμα, ώστε να καταγράφουν τι πράγματι επιτυγχάνεται στην πράξη.

Το ταβάνι της μνήμης μεταβάλλεται με το μέγεθος του συνόλου εργασίας, αφού τα δεδομένα φτάνουν πολύ γρηγορότερα όσο χωρούν σε κάποιο επίπεδο κρυφής μνήμης παρά από την κύρια. Το εύρος ζώνης μετριέται επομένως σε διαδοχικά μεγέθη, από 32 KiB και διπλασιάζοντας κάθε φορά και κάθε μέτρηση συγκρίνεται με το ταβάνι του δικού της μεγέθους. Η προσέγγιση αυτή μετρά εύρος ζώνης υπό επαναχρησιμοποίηση, κάτι που θα φαινόταν ακατάλληλο για αλγόριθμο που διατρέχει την είσοδο μία φορά. Δεν ισχύει όμως εδώ, η είσοδος ανακτάται από αντίγραφο ασφαλείας ακριβώς πριν τη χρονομετρούμενη περιοχή (ενότητα 3.8), οπότε για σύνολα εργασίας έως το μέγεθος της τελευταίας βαθμίδας τα δεδομένα είναι ήδη εκεί και η μέτρηση αντιστοιχεί σε δεύτερο πέρασμα. Πέρα από αυτό το μέγεθος, η κλίμακα συγκλίνει μόνη της στον ρυθμό της κύριας μνήμης.

Το υπολογιστικό ταβάνι μετριέται σε συγκρίσεις-ανταλλαγές, για τον λόγο της ενότητας 3.3.4. Αν κάθε σύγκριση-ανταλλαγή χρησιμοποιούσε το αποτέλεσμα της προηγούμενης θα σχηματιζόταν αλυσίδα εξαρτήσεων, μετρώντας καθυστέρηση αντί για ρυθμό, με ταβάνι πολλαπλάσια χαμηλότερο και κάθε υλοποίηση να φαίνεται εσφαλμένα να το αγγίζει. Ο πυρήνας μέτρησης εκτελεί επομένως 64 ανεξάρτητες αλυσίδες παράλληλα, με εναλλαγή τριών καταχωρητών ώστε ο μεταγλωττιστής να μην παραλείψει καμία πράξη.

Για τη μονάδα νευρωνικής επεξεργασίας το μοντέλο σημαίνει κάτι διαφορετικό, αφού δεν υπάρχει πυρήνας ΑΙΕ με ρυθμιζόμενη υπολογιστική ένταση. Τα μετρημένα ταβάνια αφορούν εδώ τη διαδρομή τροφοδοσίας, όχι τον υπολογισμό στα πλακίδια, και διαβάζονται ως «πόσο γρήγορα μπορεί ο host να ετοιμάσει τα δεδομένα που καταναλώνει η μονάδα», που είναι άλλωστε ο πραγματικός περιορισμός εδώ.

3.5 Βιβλιοθήκες σύγκρισης

Το μέτρο σύγκρισης για την επίδοση σε κάθε αρχιτεκτονική είναι μια καθιερωμένη υλοποίηση βιβλιοθήκης, ώστε η σύγκριση να αντικατοπτρίζει το πραγματικό όφελος έναντι όσων ήδη έχει στη διάθεσή του ένας προγραμματιστής.

Για τον επεξεργαστή χρησιμοποιείται η `std::partial_sort`. Η επιλογή της έναντι της `std::nth_element` δεν είναι αυτονόητη, αλλά για k πολύ μικρότερο του n είναι σαφώς ταχύτερη. Η `std::partial_sort` υπερτερεί διότι διατηρεί σωρό μεγέθους k που παραμένει στην κρυφή μνήμη και απορρίπτει τα περισσότερα στοιχεία με μία μόνο σύγκριση, ενώ η `std::nth_element` διαπερνά επανειλημμένα μεγάλα τμήματα του πίνακα κατά τη διαμέριση.

Για τη μονάδα γραφικών χρησιμοποιείται η `cub::DeviceTopK`, η οποία υλοποιεί επιλογή με βάση την αναπαράσταση. Η παραπάνω είναι πραγματική συνάρτηση `top-k`, δηλαδή δεν ταξινομεί πλήρως και μετά περικόβει. Συνεπώς, η σύγκριση γίνεται με αντίπαλο που λύνει το ίδιο πρόβλημα με βελτιστοποιημένο τρόπο.

Η αλυσίδα εργαλείων ΑΙΕ δίνει μεταγλωττιστή και παραδείγματα πυρήνων, όχι βιβλιοθήκη αλγορίθμων, οπότε δεν υφίσταται καθιερωμένη υλοποίηση `top-k` για το υλικό αυτό, ούτε από τον κατασκευαστή ούτε από τρίτους. Χρησιμοποιείται αντ' αυτού η ίδια `std::partial_sort` του επεξεργαστή, εκτελούμενη εξ ολοκλήρου στον host χωρίς καμία αξιοποίηση της μονάδας. Το ερώτημα που απαντά η σύγκριση αλλάζει από «είναι η υλοποίησή μας καλύτερη από έναν βελτιστοποιημένο πυρήνα ΑΙΕ» (τέτοιος πυρήνας δεν υπάρχει), σε «αξίζει η μεταφορά του υπολογισμού στη μονάδα, ή είναι προτιμότερο ό,τι ήδη διαθέτει ο host».

Το κοινό μέτρο σύγκρισης δίνει κι έναν ανεξάρτητο έλεγχο της υποδομής. Αφού τρέχει ο ίδιος κώδικας στο ίδιο υλικό, οι δύο μετρήσεις οφείλουν να συμπίπτουν. Η σύμπτωσή τους στα αποτελέσματα του κεφαλαίου 7 επιβεβαιώνει πως η υποδομή δεν εισάγει συστηματική μεροληψία ανάλογα με την αρχιτεκτονική.

3.6 Έλεγχος ορθότητας

Μια μέτρηση χρόνου έχει νόημα μόνο αν το αποτέλεσμα είναι σωστό. Κάθε περίπτωση κάθε σάρωσης επαληθεύεται και όσες αποτύχουν καταγράφονται ως αποτυχίες αντί να

μπουν στα αποτελέσματα.

Ο έλεγχος γίνεται ως προς ξεχωριστή υλοποίηση επαλήθευσης, η οποία δεν πρέπει να συγχέεται με τις βιβλιοθήκες της ενότητας 3.5. Εκείνες μετρούν επίδοση, αυτή κρίνει ορθότητα. Είναι η απλούστερη δυνατή υλοποίηση: αντίγραφο της εισόδου ταξινομείται πλήρως στον host και κρατούνται τα πρώτα k . Είναι αργή, χωρίς πρόθεση ανταγωνισμού, και χρειάζεται μόνο να είναι αναμφισβήτητα σωστή.

Η σύγκριση δεν γίνεται ίδια για όλους τους τύπους δεδομένων. Οι ακέραιοι απαιτούν απόλυτη ισότητα, ενώ οι τύποι κινητής υποδιαστολής ανοχή, σχετική ή απόλυτη ανάλογα με το μέγεθος των τιμών, αφού διαφορετικές αρχιτεκτονικές εκτελούν τις πράξεις με διαφορετική σειρά και ενδιάμεση ακρίβεια. Κάθε υλοποίηση επιστρέφει το αποτέλεσμα της ταξινομημένο, με την ίδια σειρά της υλοποίησης επαλήθευσης (ενότητα 2.1), άρα η σύγκριση γίνεται θέση προς θέση, όχι ως σύνολο. Όταν εκτελούνται και το πλήρες και το περικομμένο δίκτυο, τα δύο αποτελέσματα συγκρίνονται κι αυτά μεταξύ τους, κι αυτό ελέγχει ανεξάρτητα ότι η περικοπή δεν αλλοιώνει το αποτέλεσμα.

Στην κύρια σάρωση της εργασίας επαληθεύτηκαν και οι 2700 περιπτώσεις χωρίς καμία αποτυχία.

3.7 Παραμετρικός χώρος

Κάθε εκτέλεση καθορίζεται πλήρως από παραμέτρους γραμμής εντολών, καμία από αρχείο ρυθμίσεων ή μεταβλητή περιβάλλοντος. Η περιγραφή μιας μέτρησης είναι έτσι πάντα πλήρης και αναπαραγωγίμη. Πλήρης κατάλογος βρίσκεται στο παράρτημα Γ'.

Το μέγεθος εισόδου n είναι δύναμη του δύο, ίσο με 2^q , όπου το q παίρνει τιμές από 3 έως 24. Το άνω όριο, περίπου 16,8 εκατομμύρια στοιχεία, ξεπερνά κατά πολύ την κρυφή μνήμη, οπότε η μέτρηση αφορά την κύρια μνήμη.

Η παράμετρος k λαμβάνει τιμές 8, 256, 4096, 131072, τάξεις μεγέθους απόσταση μεταξύ τους, από k τόσο μικρό που ο σωρός χωρά σε ελάχιστους καταχωρητές έως k που είναι σημαντικό ποσοστό της ίδιας της εισόδου. Όπου το k υπερβαίνει το n , περιορίζεται σε αυτό.

3.7.1 Τύποι δεδομένων

Τύπος	Αντιστοίχιση σε C++	Μέγεθος	Περιγραφή
int	std::int32_t	32 bit	ακέραιος με πρόσημο
uint	std::uint32_t	32 bit	ακέραιος χωρίς πρόσημο
float	float	32 bit	κινητής υποδιαστολής απλής ακρίβειας
double	double	64 bit	κινητής υποδιαστολής διπλής ακρίβειας
half	_Float16	16 bit	κινητής υποδιαστολής μισής ακρίβειας

Πίνακας 3.3: Οι τύποι δεδομένων που σαρώνονται.

Η σάρωση καλύπτει πέντε τύπους δεδομένων (πίνακας 3.3). Ο τύπος επηρεάζει τη μέτρηση με δύο τρόπους: πόσα bytes μετακινούνται ανά στοιχείο, κάτι που μετατο-

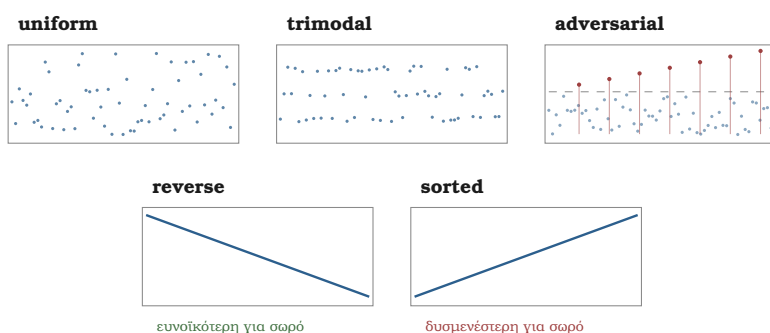
πίζει τη θέση της ίδιας υλοποίησης απέναντι στο ταβάνι μνήμης, και πόσο κοστίζει μια σύγκριση, που δεν είναι το ίδιο για όλους.

3.7.2 Κατανομές εισόδου

Η κατανομή των τιμών εισόδου επηρεάζει τον χρόνο εκτέλεσης. Λόγου χάρη, οι αλγόριθμοι με σωρούς δουλεύουν μόνο όταν ένα νέο στοιχείο ξεπερνά τη ρίζα, άρα η σειρά εμφάνισης των τιμών μεταβάλλει δραστικά τον χρόνο τους. Σαρώνονται πέντε κατανομές (πίνακας 3.4, σχήμα 3.4). Το εκτελέσιμο υποστηρίζει και κανονική κατανομή, που δεν χρησιμοποιείται εδώ καθώς η trimodal καλύπτει εντονότερα την περίπτωση συσσωρευμένων τιμών.

Κατανομή	Περιγραφή
uniform	ομοιόμορφη στο εύρος τιμών
trimodal	τρεις στενές συστάδες εντός του εύρους
sorted	αύξουσα σειρά
reverse	φθίνουσα σειρά
adversarial	τιμές στο κάτω ήμισυ με αύξουσες αιχμές

Πίνακας 3.4: Οι κατανομές εισόδου που σαρώνονται.



Σχήμα 3.4: Οι πέντε κατανομές εισόδου, με την τιμή κάθε στοιχείου συναρτήσει της θέσης του στον πίνακα. Η uniform και η trimodal δεν έχουν διάταξη και διαφέρουν μόνο ως προς το ποιες τιμές εμφανίζονται, οι δύο διατεταγμένες είναι τα άκρα για την επιλογή με σωρό. Στην adversarial το σώμα των τιμών παραμένει κάτω από τη μέση τιμή του εύρους (διακεκομμένη), με μεμονωμένες αιχμές σε τακτά διαστήματα.

Για την επιλογή με σωρό (ενότητα 2.2.3) οι δύο διατεταγμένες κατανομές είναι τα δύο άκρα. Η φθίνουσα είναι ευνοϊκότερη, η αύξουσα δυσμενέστερη. Η adversarial διατηρεί τη δυσμενή συμπεριφορά χωρίς να είναι πλήρως ταξινομημένη. Το μεγαλύτερο μέρος των τιμών βρίσκεται στο κάτω ήμισυ του εύρους, με αύξουσες αιχμές σε τακτά διαστήματα.

Η διτονική ταξινόμηση δεν εξαρτάται από τα δεδομένα, εκτελεί το ίδιο δίκτυο συγκρίσεων ανεξάρτητα από τις τιμές, και λειτουργεί έτσι ως μάρτυρας. Τυχόν μεταβολή του χρόνου της μεταξύ κατανομών θα πρόδιδε σφάλμα στη μέτρηση, αφού ο ίδιος ο αλγόριθμος δεν έχει τρόπο να το προκαλέσει.

3.8 Συνθήκες μέτρησης και επαναληψιμότητα

Στην προεπιλογή ο διαχειριστής συχνότητας είναι powersave και η ενίσχυση συχνότητας ενεργή. Η συχνότητα μεταβάλλεται με το φορτίο και τη θερμοκρασία, με συστηματική μετατόπιση όσο διαρκεί η σάρωση, καθώς θερμαίνεται το σύστημα, όχι τυχαίο θόρυβο. Η τυπική απόκλιση δειγμάτων υπό τέτοιες μετατοπιζόμενες συνθήκες δεν θα αντανακλούσε την πραγματική αβεβαιότητα.

Πριν από κάθε σάρωση λοιπόν το σύστημα καθηλώνεται σε σταθερή, τεκμηριωμένη κατάσταση (πίνακας 3.5), καταγράφεται πριν από τη σάρωση και αποκαθίσταται μετά.

Παράμετρος	Κατάσταση κατά τη μέτρηση
Διαχειριστής συχνότητας CPU	performance
Ενίσχυση συχνότητας CPU	απενεργοποιημένη
Νήματα CPU	8, ένα ανά φυσικό πυρήνα
GPU persistence mode	ενεργοποιημένο
Όριο ισχύος GPU	σταθερό σε όλη τη σάρωση
Περίοδος ψύξης μεταξύ μονάδων	15 s
Επαναλήψεις προθέρμανσης	5 ανά περίπτωση
Μετρούμενες επαναλήψεις	50 ανά περίπτωση

Πίνακας 3.5: Οι συνθήκες στις οποίες καθηλώνεται το σύστημα πριν από κάθε σάρωση.

Το όριο ισχύος της μονάδας γραφικών επιλέγεται μία φορά, εντός των ορίων του πίνακα 3.1 και μένει σταθερό. Η τιμή του είναι αυθαίρετη, αλλά κάθε αλλαγή κατά τη σάρωση θα μετέβαλλε ταυτόχρονα χρόνο και ενέργεια.

Οι τρεις μονάδες μοιράζονται κοινό θερμικό όριο (ενότητα 3.1). Περίοδος ψύξης 15 s μεταξύ διαδοχικών μονάδων εμποδίζει καθεμία να ξεκινά από τη θερμική κατάσταση της προηγούμενης. Πρόκειται πάντως για μερική μόνο αντιμετώπιση του προβλήματος. Σε φορητό σύστημα η θερμική σύζευξη μεταξύ των μονάδων δεν αίρεται πλήρως και παραμένει αναγνωρισμένος περιορισμός των μετρήσεων.

Κάθε περίπτωση εκτελείται 5 φορές ως προθέρμανση και 50 υπό μέτρηση, με αναφερόμενη τη μέση τιμή, την τυπική απόκλιση και το ελάχιστο. Πριν από κάθε επανάληψη η είσοδος ανακτάται από αντίγραφο ασφαλείας, εκτός της χρονομετρούμενης περιοχής, ώστε το κόστος της αντιγραφής να μην προσμετράται και κάθε επανάληψη να ξεκινά από πανομοιότυπη είσοδο.

Κάθε συνδυασμός κατανομής και σπόρου γεννήτριας τυχαίων αριθμών είναι ανεξάρτητο δείγμα, πέρα από τις επαναλήψεις εντός μιας περίπτωσης. Τα διαγράμματα συναθροίζουν αυτά τα δείγματα, απεικονίζουν τη διασπορά τους ως ζώνες σφάλματος, αβεβαιότητα δεδομένων ξεχωριστή από αβεβαιότητα μέτρησης.

Τέλος κάθε εκτέλεση καταγράφει το ίδιο το περιβάλλον όπου έτρεξε (αναθεώρηση αποθετηρίου, πυρήνα, μεταγλωττιστή, εκδόσεις οδηγών και βιβλιοθηκών κάθε μονάδας). Κάθε σύνολο μετρήσεων τεκμηριώνει έτσι τις συνθήκες παραγωγής του χωρίς να εξαρτάται από εξωτερική καταγραφή.

Κεφάλαιο 4

Υλοποίηση σε CPU

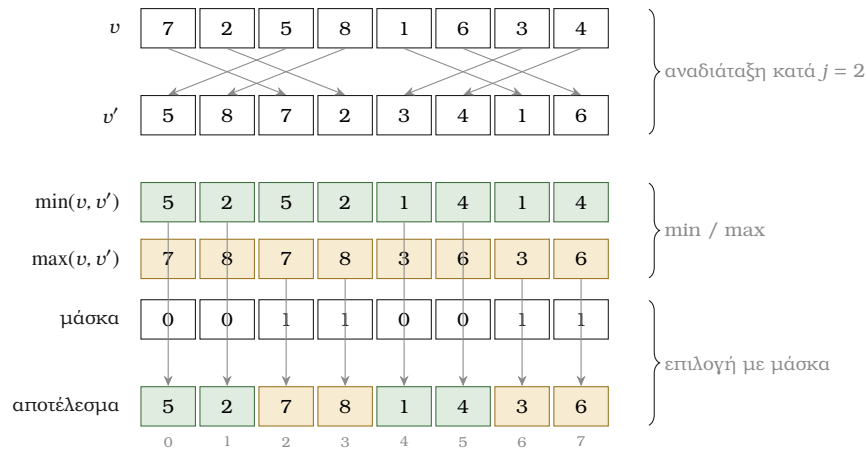
4.1 Σχεδιαστικές επιλογές

Οι δύο υλοποιήσεις του κεφαλαίου έχουν κοινή σχεδιαστική βάση: διανυσματοποιούν τη σύγκριση-ανταλλαγή αντί να μείνουν σε βαθμωτό κώδικα και περνούν από κοινή αφαιρετική διεπαφή αντί να γράφονται σε πανομοιότυπα αντίγραφα. Επιπλέον, μετατρέπουν τη μισή ακρίβεια σε απλή για τον υπολογισμό, ενώ η κατανομή νημάτων στα δύο διαφέρει.

4.1.1 Διανυσματοποίηση

Και οι δύο αλγόριθμοι στηρίζονται σε μια πράξη που διανυσματοποιείται εξαιρετικά καλά. Η σύγκριση-ανταλλαγή δύο στοιχείων ισοδυναμεί με τον υπολογισμό ελαχίστου και μεγίστου, και οι δύο υπάρχουν ως εντολές που δρουν ταυτόχρονα σε ολόκληρο διανυσματικό καταχωρητή, χωρίς διακλαδώσεις. Ένας καταχωρητής 512 bit εκτελεί δεκαέξι συγκρίσεις ακεραίων απλής ακρίβειας στον χρόνο μίας. Οι εντολές αναφέρονται εδώ με τα ονόματα των ενδογενών συναρτήσεων του προτύπου [19], που υλοποιεί και ο επεξεργαστής της ενότητας 3.1.

Η σύγκριση αυτή καθαυτή είναι απλή. Η δυσκολία είναι η τοποθέτηση. Για να συγκριθούν δύο στοιχεία που απέχουν j θέσεις, πρέπει να καταλήξουν στην ίδια θέση δύο διαφορετικών καταχωρητών. Όταν το j είναι μικρότερο από το πλάτος του καταχωρητή, αυτό γίνεται με αναδιάταξη εντός του καταχωρητή: το περιεχόμενο μετατίθεται κατά j θέσεις, υπολογίζονται ελάχιστο και μέγιστο, το αποτέλεσμα ανασυντίθεται επιλέγοντας εναλλάξ από τα δύο με μάσκα. Η ακολουθία «αναδιάταξη, ελάχιστο, μέγιστο, επιλογή» είναι το βασικό μοτίβο πάνω στο οποίο χτίζεται η συγχώνευση επιπέδων της ενότητας 4.2 (σχήμα 4.1).



Σχήμα 4.1: Το θεμελιώδες μοτίβο «αναδιάταξη, ελάχιστο, μέγιστο, επιλογή» για σύγκριση-ανταλλαγή σε απόσταση $j = 2$, σε καταχωρητή οκτώ θέσεων. Το περιεχόμενο μετατίθεται ώστε κάθε στοιχείο να βρεθεί απέναντι στο ταίρι του, υπολογίζονται ταυτόχρονα ελάχιστο και μέγιστο για όλες τις θέσεις, και το αποτέλεσμα ανασυντίθεται επιλέγοντας το ελάχιστο όπου το αντίστοιχο bit της μάσκας είναι μηδέν και το μέγιστο όπου είναι ένα. Καμία διακλάδωση δεν μεσολαβεί και καμία τιμή δεν εξετάζεται ξεχωριστά.

Το πλάτος των καταχωρητών δεν είναι γνωστό κατά τη μεταγλώττιση. Ο κώδικας μεταγλωττίζεται με όλες τις διαδρομές παρούσες. Η επιλογή γίνεται κατά την εκκίνηση, με ερώτημα στον επεξεργαστή για τις εντολές που υποστηρίζει. Αν υποστηρίζει AVX-512 επιλέγει καταχωρητές 512 bit, αλλιώς 256 bit, διαφορετικά, ως εφεδρεία, βαθμωτή διαδρομή. Το ίδιο εκτελέσιμο τρέχει επομένως σε οποιοδήποτε μηχάνημα, με τη διαφορά ότι σε διαφορετικό υλικό ακολουθεί διαφορετική διαδρομή κώδικα και τα αποτελέσματά του δεν είναι άμεσα συγκρίσιμα.

4.1.2 Αφαίρεση διανυσματικών τύπων

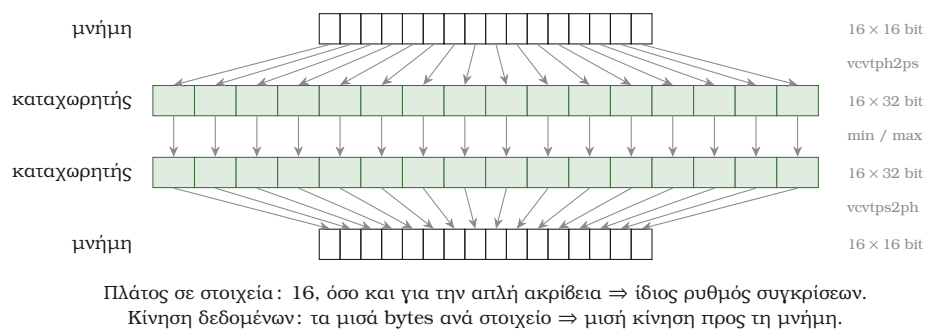
Οι διανυσματικές εντολές είναι εξειδικευμένες ανά τύπο δεδομένων. Άλλη εντολή χρησιμοποιείται για ελάχιστο ακεραίων με πρόσημο, άλλη για χωρίς, άλλη για κινητή υποδιαστολή. Πέντε τύποι επί δύο πλάτη καταχωρητών επί δύο αλγόριθμους: η άμεση συγγραφή θα έδινε είκοσι σχεδόν πανομοιότυπες παραλλαγές, με το αντίστοιχο κόστος συντήρησης.

Για να αποφευχθεί αυτό, ορίζεται ενιαία διεπαφή. Για κάθε συνδυασμό τύπου και πλάτους δηλώνεται μια δομή που εκθέτει πάντα τα ίδια ονόματα (διανυσματικό τύπο, πλάτος σε στοιχεία, φόρτωση, αποθήκευση, ελάχιστο, μέγιστο, επιλογή με μάσκα, αναδιάταξη, παραγωγή μάσκας υποψηφίων για το map-reduce). Οι αλγόριθμοι γράφονται μία φορά ως προς αυτήν, χωρίς ποτέ να αναφέρονται σε συγκεκριμένη εντολή.

Πιθανή προσθήκη νέου τύπου χρειάζεται μόνο μια εξειδίκευση της δομής, καμία αλλαγή στους αλγόριθμους, και η απουσία μιας εξειδίκευσης φαίνεται αμέσως στη μεταγλώττιση. Χωρίς δηλωμένη διεπαφή για κάποιον συνδυασμό, ο κώδικας μεταβαίνει αυτόματα στη βαθμωτή διαδρομή. Η ισοδυναμία των διαδρομών είναι ελεγμένη.

4.1.3 Υποστήριξη αριθμών κινητής υποδιαστολής μισής ακρίβειας

Ο επεξεργαστής αποθηκεύει τιμές των 16 bit χωρίς να έχει αριθμητικές εντολές που δρουν απευθείας πάνω τους στο πλάτος που χρειάζεται ο αλγόριθμος. Η βαθμωτή επεξεργασία θα ακύρωνε κάθε όφελος. Η λύση εκμεταλλεύεται το γεγονός ότι η μετατροπή μεταξύ των δύο αναπαραστάσεων γίνεται με μία μόνο εντολή. Η εξειδίκευση της διεπαφής για μισή ακρίβεια ορίζει ως διανυσματικό τύπο εκείνον της απλής. Η φόρτωση διαβάζει δεκαέξι τιμές των 16 bit και τις μετατρέπει σε δεκαέξι των 32, όλες οι πράξεις εκτελούνται σε απλή ακρίβεια, και κατά την αποθήκευση μετατρέπονται πίσω. Τα δεδομένα μένουν σε μισή ακρίβεια στη μνήμη, γίνονται απλή μόνο όσο διαρκεί ο υπολογισμός (σχήμα 4.2).



Σχήμα 4.2: Η διαδρομή της μισής ακρίβειας. Τα δεδομένα παραμένουν σε 16 bit στη μνήμη, όπου καταλαμβάνουν τον μισό χώρο, και μετατρέπονται σε απλή ακρίβεια μόνο εντός των καταχωρητών, όσο διαρκεί ο υπολογισμός.

Κάθε τιμή μισής ακρίβειας αναπαρίσταται επακριβώς σε απλή, και οι πράξεις ελαχίστου/μεγίστου απλώς επιλέγουν μία από τις δύο εισόδους, χωρίς να μεσολαβεί καμία προσέγγιση ή στρογγυλοποίηση. Το αποτέλεσμα ταυτίζεται με εκείνο υλοποίησης που θα δούλευε εγγενώς σε μισή ακρίβεια.

Το πλάτος του καταχωρητή σε στοιχεία παραμένει δεκαέξι, όσο και για την απλή ακρίβεια, άρα ο ρυθμός συγκρίσεων δεν αλλάζει. Η κίνηση δεδομένων πάντως υποδιπλασιάζεται και κάθε στοιχείο καταλαμβάνει τα μισά bytes. Για αλγόριθμους περιορισμένους από μνήμη αυτό μεταφράζεται άμεσα σε επιτάχυνση.

4.1.4 Παραλληλοποίηση σε πολλαπλά νήματα

Οι δύο αλγόριθμοι παραλληλοποιούνται με θεμελιωδώς διαφορετικό τρόπο, και η διαφορά καθορίζει πόσο επιθετικά κλιμακώνεται ο καθένας.

Η διτονική ταξινόμηση έχει καθολικές εξαρτήσεις. Κάθε ομάδα επιπέδων χρειάζεται ολοκληρωμένο το αποτέλεσμα της προηγούμενης. Η παραλληλία είναι εφικτή μόνο εντός των ομάδων, με φράγμα συγχρονισμού ανάμεσά τους.

Το map-reduce έχει διαφορετική δομή. Τα νήματα είναι εντελώς ανεξάρτητα σε ξένα μεταξύ τους τμήματα και συναντιούνται μία φορά στη συγχώνευση. Δεν απαιτείται κανένας συγχρονισμός κατά τη σάρωση, αλλά κάθε νήμα κρατά δικό του σωρό μεγέθους k , και η τελική συγχώνευση είναι σειριακή, ανάλογη του πλήθους νημάτων.

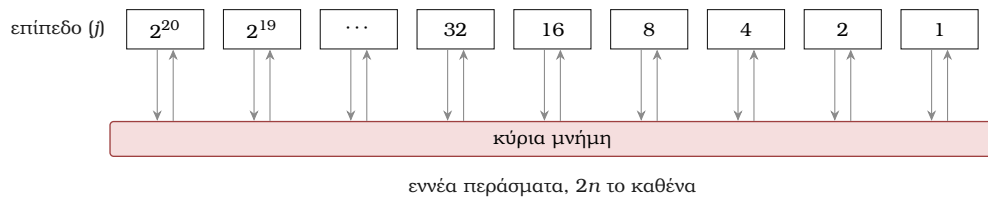
Το βέλτιστο πλήθος νημάτων διαφέρει λοιπόν. Η διτονική ταξινόμηση αντέχει περισσότερο σε μικρότερες εισόδους, το map-reduce χρειάζεται πολύ μεγαλύτερο μερίδιο ανά νήμα για να αποσβέσει το κόστος του επιπλέον σωρού και της σειριακής φάσης. Και στις δύο περιπτώσεις το πλήθος περιορίζεται δυναμικά ανάλογα με το μέγεθος της εισόδου.

4.2 Διτονική ταξινόμηση

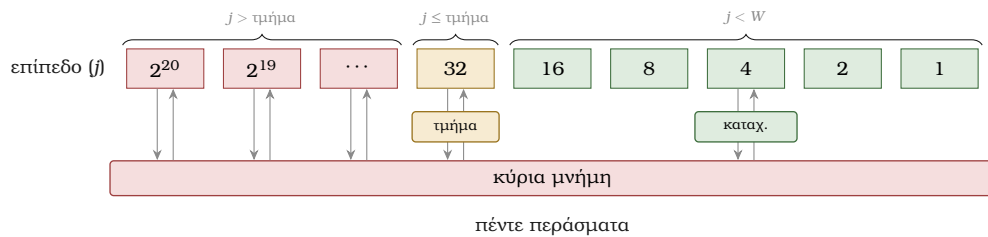
Το χρονοδιάγραμμα επιπέδων της ενότητας 2.2.2 χτίζεται μία φορά και είναι κοινό και για τις τρεις αρχιτεκτονικές. Όμως, η άμεση μεταφορά του σε κώδικα δίνει υλοποίηση απελπιστικά περιορισμένη από τη μνήμη. Για k ίσο με 256, μόνο η πρώτη φάση έχει 36 επίπεδα, 36 ξεχωριστά περάσματα πάνω σε ολόκληρο τον πίνακα. Σε είσοδο με τέσσερα εκατομμύρια στοιχεία αυτό σημαίνει δεκάδες διαδοχικές αναγνώσεις και εγγραφές δεκαέξι MB, με ελάχιστο υπολογισμό ανάμεσα. Η υλοποίηση είναι οργανωμένη γύρω από την αποφυγή του παραπάνω σχήματος.

Η βασική τεχνική είναι συγχώνευση διαδοχικών επιπέδων σε ένα πέρασμα. Ένα επίπεδο χαρακτηρίζεται από την απόσταση j ανάμεσα στα στοιχεία που συγκρίνει, και αυτή καθορίζει τι είναι εφικτό. Για j μικρότερο του πλάτους του καταχωρητή, τα δύο στοιχεία κάθε σύγκρισης είναι ήδη στον ίδιο καταχωρητή. Μια ολόκληρη ακολουθία τέτοιων επιπέδων εκτελείται τότε ως μία φόρτωση, σειρά αναδιατάξεων και συγκρίσεων και αποθήκευση. Για μεγαλύτερο j , αλλά σύνολο εργασίας που χωρά στην κρυφή μνήμη, η ακολουθία τρέχει τμήμα προς τμήμα, παραμένοντας εκεί για όλα τα επίπεδα. Μόνο τα επίπεδα με πολύ μεγάλο j θέλουν ξεχωριστό πέρασμα. Ένα επίπεδο περικοπής με απόσταση ίση με το πλάτος του καταχωρητή απορροφά και τα επίπεδα επαναταξινόμησης που το ακολουθούν (σχήμα 4.3). Όταν οι παράμετροι ενός επιπέδου βγαίνουν εκτός εύρους, η ίδια ακολουθία τρέχει βαθμωτά αντί για διανυσματικά. Το εύρος εξόδου κάθε ομάδας μοιράζεται στα νήματα, με όρια ευθυγραμμισμένα στο μέγεθος τμήματος ή γραμμής κρυφής μνήμης, ώστε δύο νήματα να μη γράφουν ποτέ στην ίδια γραμμή.

(α) άμεση μεταφορά



(β) συγχώνευση επιπέδων



Σχήμα 4.3: Η συγχώνευση διαδοχικών επιπέδων σε ένα πέρασμα. Κάθε ζεύγος βελών είναι μία ανάγνωση και μία εγγραφή ολόκληρου του πίνακα. Στο (α) κάθε επίπεδο είναι ξεχωριστό πέρασμα προς τη μνήμη· στο (β) τα επίπεδα ομαδοποιούνται κατά απόσταση j και μόνο η τελευταία ομάδα διατηρεί ξεχωριστό πέρασμα. Οι αριθμοί του σχήματος είναι ενδεικτικοί.

Η κίνηση που προκύπτει είναι προβλέψιμη από τη δομή του χρονοδιαγράμματος. Η πρώτη φάση συμπύσσεται σε ένα πέρασμα επί τόπου, δηλαδή μία ανάγνωση και μία εγγραφή ανά στοιχείο, συνολικά $2n$. Κάθε βήμα της δεύτερης διαβάζει n_i στοιχεία και γράφει $n_i/2$, η επαναταξινόμηση που ακολουθεί προσθέτει άλλα $2 \cdot n_i/2$, σύνολο $2,5 n_i$. Και επειδή το n_i υποδιπλασιάζεται σε κάθε βήμα, το άθροισμα είναι γεωμετρική σειρά:

$$\sum_i 2,5 n_i = 2,5 \left(n + \frac{n}{2} + \frac{n}{4} + \dots \right) = 2,5 \times 2n = 5n.$$

Περίπου $7n$ στοιχεία συνολικά. Από εδώ προκύπτει ο συντελεστής επανάληψης της ενότητας 3.3.4. Για $n = 2^{22}$, κ ίσο με 256, αυτό δίνει 117 435 392 bytes, ακριβώς όσο η μετρούμενη τιμή.

Τα επίπεδα περικοπής δεν τρέχουν επί τόπου, αφού κάθε στοιχείο της εξόδου προκύπτει από δύο στοιχεία της εισόδου σε διαφορετικές θέσεις. Γι' αυτό διατηρείται δεύτερος πίνακας ίδιου μεγέθους, με τους δύο να εναλλάσσονται μετά από κάθε τέτοια ομάδα. Η επιπλέον μνήμη είναι το τμήμα για αποφυγή ενδιάμεσων αντιγραφών.

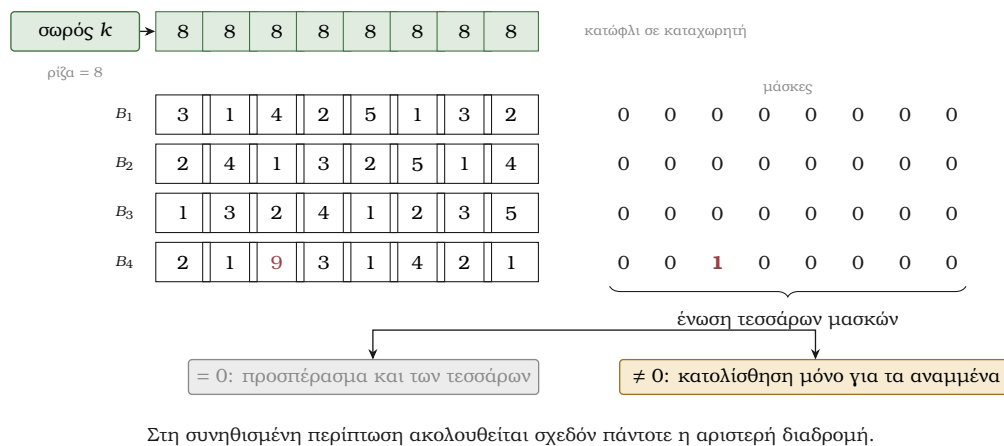
4.3 Map-reduce με τοπικούς σωρούς

Εδώ ακολουθείται εντελώς διαφορετική λογική. Αντί για σταθερή ακολουθία συγκρίσεων, διατηρούνται τα k καλύτερα στοιχεία που έχουν εξεταστεί μέχρι στιγμής και κάθε νέο εξετάζεται ως προς αυτά. Η είσοδος χωρίζεται σε τμήματα, ένα ανά νήμα, και κάθε νήμα παράγει τα δικά του k καλύτερα. Τα επιμέρους αποτελέσματα συγχωνεύονται στο τέλος.

Κάθε νήμα διατηρεί σωρό μεγέθους k , με τη ρίζα το χειρότερο από τα επιλεγμένα

στοιχεία. Ο κάθε σωρός αρχικοποιείται μαζικά από τα πρώτα k στοιχεία του τμήματος, που κοστίζει γραμμικά και είναι φθηνότερο από k διαδοχικές εισαγωγές. Από εκεί και πέρα, η ρίζα λειτουργεί ως κατώφλι: κάθε στοιχείο που δεν το ξεπερνά μπορεί να απορριφθεί χωρίς καμία περαιτέρω εργασία.

Η παραπάνω απόρριψη γίνεται μαζικά. Το κατώφλι αντιγράφεται σε διανυσματικό καταχωρητή και συγκρίνεται ταυτόχρονα με ένα ολόκληρο μπλοκ στοιχείων, παράγοντας μια δυαδική μάσκα υποψηφίων. Η υλοποίηση εξετάζει τέσσερα τέτοια μπλοκ μαζί και, αν η ένωση των τεσσάρων масκών είναι μηδενική, προσπερνά όλα με έναν έλεγχο. Όταν ο σωρός έχει ήδη γεμίσει με μεγάλες τιμές (η συνηθισμένη περίπτωση), το μεγαλύτερο μέρος της εισόδου περνάει ακριβώς έτσι: διαβάζεται, συγκρίνεται και απορρίπτεται, χωρίς κανένα στοιχείο να εξεταστεί ξεχωριστά (σχήμα 4.4).



Σχήμα 4.4: Η μαζική απόρριψη του map-reduce. Η ρίζα του σωρού αντιγράφεται σε ολόκληρο έναν διανυσματικό καταχωρητή και συγκρίνεται ταυτόχρονα με τέσσερα μπλοκ της εισόδου, παράγοντας μία δυαδική μάσκα ανά μπλοκ. Η ένωση των масκών επιλέγει τη μία από τις δύο διαδρομές στο κάτω μέρος του σχήματος.

Μόνο για τα πραγματικά αναμμένα bits τρέχει βαθμωτή εργασία. Η ρίζα αντικαθίσταται και ο σωρός αποκαθίσταται με κατολίσθηση προς τα κάτω. Η αντικατάσταση της ρίζας προτιμάται από εξαγωγή/επανεισαγωγή, καθώς η δεύτερη θα διέσχισε τον σωρό δύο φορές αντί για μία.

Εδώ αποδίδεται η εξάρτηση από τα δεδομένα της ενότητας 3.7.2. Σε φθίνουσα είσοδο ο σωρός γεμίζει αμέσως με τις μεγαλύτερες τιμές, το κατώφλι ανεβαίνει στο μέγιστο από την αρχή και όλη η υπόλοιπη είσοδος απορρίπτεται μαζικά. Σε αύξουσα είσοδο συμβαίνει το αντίθετο, κάθε νέο στοιχείο ξεπερνά τη ρίζα, καμία μάσκα δεν είναι μηδενική και κάθε στοιχείο επιβάλλει πλήρη κατολίσθηση. Ο ίδιος κώδικας, με είσοδο ίδιου μεγέθους, εκτελεί έτσι στη μία περίπτωση σχεδόν καμία εργασία πέρα από την ανάγνωση και στην άλλη τη μέγιστη δυνατή, με μόνη διαφορά τη σειρά με την οποία εμφανίζονται οι τιμές.

Η συγχώνευση είναι απλή, σειριακή. Τα στοιχεία κάθε τοπικού σωρού προσφέρονται διαδοχικά στον πρώτο, με το ίδιο κριτήριο κατωφλίου και το τελικό αποτέλεσμα ταξινομείται. Το κόστος είναι ανάλογο του γινομένου των νημάτων επί k και ανεξάρτητο του μεγέθους εισόδου, συνεπώς αμελητέο για μεγάλα n .

Η κίνηση δεδομένων προκύπτει άμεσα. Κάθε στοιχείο της εισόδου διαβάζεται ακριβώς μία φορά, η συγχώνευση διακινεί δύο φορές το γινόμενο νημάτων επί k . Το πλήθος συγκρίσεων, όμως, αναφέρεται ως κάτω φράγμα, αφού το βάθος κάθε κατολίσθησης εξαρτάται από τις τιμές. Η καταμέτρηση προσμετρά μία σύγκριση ανά στοιχείο και αγνοεί την αναδιάταξη του σωρού.

Το πλήθος νημάτων περιορίζεται πολύ πιο συντηρητικά απ' ό,τι στη διτονική ταξινόμηση. Χρειάζεται περίπου ένα εκατομμύριο στοιχεία ανά νήμα (όριο που βρέθηκε εμπειρικά), γιατί κάθε επιπλέον νήμα προσθέτει σωρό μεγέθους k και επιβαρύνει τη σειριακή συγχώνευση. Για τέσσερα εκατομμύρια στοιχεία εισόδου χρησιμοποιούνται τέσσερα νήματα και όχι οκτώ, ακόμη κι όταν ζητούνται οκτώ.

4.4 Διαχείριση μνήμης και κίνηση δεδομένων

Οι δύο αλγόριθμοι επιλύουν το ίδιο πρόβλημα στο ίδιο μηχάνημα, με εντελώς διαφορετική όμως συμπεριφορά ως προς τη μνήμη. Από εκεί προέρχεται το μεγαλύτερο μέρος της διαφοράς επίδοσης.

Η θεμελιώδης διαφορά είναι πρώτα ο όγκος της κίνησης. Από τις εκφράσεις των ενότητων 4.2 και 4.3, η διτονική ταξινόμηση διακινεί περίπου $7n$ στοιχεία, ενώ το map-reduce n συν όρο ανάλογο νημάτων επί k , αμελητέο για μικρό k . Για είσοδο τεσσάρων εκατομμυρίων ακεραίων, η πρώτη διακινεί 117 MB και η δεύτερη 17 MB.

Η φύση της πρόσβασης μετράει εξίσου. Το map-reduce διαβάζει την είσοδο και δεν την ξαναγγίζει, καθαρά ακολουθιακή ανάγνωση, το ευνοϊκότερο μοτίβο για τον προσκομιστή δεδομένων. Η διτονική ταξινόμηση αντιθέτως γράφει σε κάθε πέρασμα, με συγκρίσεις ανάμεσα σε στοιχεία που απέχουν μεταβλητή απόσταση, σε ορισμένα επίπεδα μεγαλύτερη από ό,τι χωρά στην κρυφή μνήμη.

Το αποτύπωμα διαφέρει επίσης. Η διτονική ταξινόμηση χρειάζεται δεύτερο πίνακα ισομεγέθη με τον πρώτο, $2n$ στοιχεία συνολικά. Το map-reduce δεσμεύει μόνο τους τοπικούς σωρούς, δηλαδή γινόμενο νημάτων επί k στοιχεία, ανεξάρτητα του μεγέθους εισόδου. Για μικρό k είναι τόσο μικρό ώστε να μένει μόνιμα στα ταχύτερα επίπεδα της ιεραρχίας, με τη μόνη κίνηση προς την κύρια μνήμη να είναι η ίδια η ανάγνωση της εισόδου.

Απέναντι στα ταβάνια της ενότητας 3.4 οι δύο αλγόριθμοι καταλήγουν έτσι σε διαφορετικά σημεία. Και οι δύο εκτελούν λίγες πράξεις ανά byte, οπότε κανένας δεν πλησιάζει το υπολογιστικό ταβάνι. Το map-reduce πετυχαίνει σημαντικά υψηλότερο ρυθμό διακίνησης, όπως αναμένεται από το ευνοϊκότερο μοτίβο πρόσβασης. Η διτονική ταξινόμηση πληρώνει διπλά, γιατί διακινεί επταπλάσιο όγκο και με χαμηλότερο ρυθμό.

Το πλεονέκτημα αυτό όμως δεν κρατά σε όλο το εύρος του k . Ο όρος των τοπικών σωρών μεγαλώνει με το k και για τις μεγαλύτερες τιμές παύει να είναι αμελητέος. Ο σωρός δεν χωρά πια στις ταχύτερες βαθμίδες της κρυφής μνήμης και η σειριακή συγχώνευση αποκτά βάρος συγκρίσιμο με τη σάρωση. Η διτονική ταξινόμηση, με κίνηση πολύ πιο ασθενώς εξαρτημένη από το k , δεν έχει αντίστοιχη επιβάρυνση.

4.5 Υλοποίηση αναφοράς

Το μέτρο σύγκρισης για τον επεξεργαστή είναι η `std::partial_sort` της πρότυπης βιβλιοθήκης (ενότητα 3.5). Ενσωματώνεται στην ίδια υποδομή, ίδιο βρόχο μέτρησης και ίδιο έλεγχο ορθότητας με τις δύο υλοποιήσεις της εργασίας. Ως αποτέλεσμα, οι χρόνοι είναι άμεσα συγκρίσιμοι.

Υπάρχει, ωστόσο, μια ασυμμετρία που επηρεάζει τις μετρήσεις της επιτάχυνσης. Η συνάρτηση της βιβλιοθήκης είναι αυστηρά μονονηματική. Δεν δέχεται παράμετρο πλήθους νημάτων και αξιοποιεί αποκλειστικά έναν πυρήνα. Απεναντίας, η διτονική ταξινόμηση χρησιμοποιεί έως οκτώ νήματα και το `map-reduce` τέσσερα ή οκτώ, ανάλογα με το μέγεθος της εισόδου. Επομένως, η όποια διαφορά στην απόδοση μεταξύ των υλοποιήσεων της εργασίας και της βιβλιοθήκης οφείλεται ταυτόχρονα σε δύο παράγοντες, τις αλγοριθμικές διαφορές και στον άνισο αριθμό πυρήνων.

Η πρακτική επιτάχυνση έναντι της τυπικής λύσης που ήδη έχει στη διάθεσή του ο προγραμματιστής παραμένει όμως το κεντρικό ζητούμενο, και εκείνη είναι πράγματι μονονηματική. Ο αναγνώστης πρέπει να έχει υπόψη ότι μια αυστηρότερη αξιολόγηση νήμα προς νήμα θα έδινε μικρότερες επιταχύνσεις.

4.6 Περιοριστικοί παράγοντες

Ο περιοριστικός παράγοντας δεν είναι ο ίδιος σε όλο το εύρος του k . Μάλιστα, αυτή η μεταβολή είναι τόσο καθοριστική που αλλάζει το ποια από τις δύο υλοποιήσεις υπερτερεί σε ταχύτητα.

Για μικρό k και μεγάλη είσοδο και οι δύο υλοποιήσεις περιορίζονται σαφώς από τη μνήμη. Στη μεγαλύτερη εξεταζόμενη είσοδο, η διτονική ταξινόμηση φτάνει τα τέσσερα πέμπτα του μετρημένου ταβανιού εύρους ζώνης, το `map-reduce` τα τρία τέταρτα, με αξιοποίηση υπολογιστικού ταβανιού γύρω στο πέντε τοις εκατό και για τις δύο. Ο περιορισμός εδώ είναι η μνήμη.

Η παραπάνω διάγνωση καθορίζει τα όρια μιας πιθανής βελτίωσης. Η διτονική ταξινόμηση δεν εκτελεί τις συγκρίσεις της αναποτελεσματικά. Είναι αργή επειδή διακινεί επαπλάσιο όγκο και καμία βελτιστοποίηση του πυρήνα δεν θα την επιτάχυνε ουσιαστικά. Μόνο λιγότερη κίνηση δεδομένων θα την επιτάχυνε. Το `map-reduce` ήδη διαβάσει κάθε στοιχείο ακριβώς μία φορά, κοντά στον ρυθμό που η μνήμη μπορεί να το τροφοδοτήσει, ουσιαστικά βέλτιστο, με την έννοια πως δεν γίνεται ταχύτερο χωρίς να διαβάσει λιγότερα.

Για μεγάλο k η εικόνα αλλάζει άρδην και καμία υλοποίηση δεν πλησιάζει κανένα ταβάνι. Στη μεγαλύτερη τιμή του k που εξετάζεται, η αξιοποίηση πέφτει σε ποσοστά τάξης της μονάδας για το `map-reduce` και λίγο πάνω από δέκα τοις εκατό για τη διτονική ταξινόμηση. Όταν μια μέτρηση απέχει από αμφότερα τα ταβάνια, ο πραγματικός περιορισμός μετατοπίζεται από το εύρος ζώνης ή την υπολογιστική ικανότητα στις καθυστερήσεις και στην ανεπαρκή παραλληλία.

Όσον αφορά το `map-reduce`, όσο μεγαλώνει το k , χαμηλώνει το κατώφλι του σωρού και καταρρέει το ποσοστό απόρριψης. Η γρήγορη διαδρομή που προσπερνά τέσσερα μπλοκ σταματά να ενεργοποιείται και σχεδόν κάθε στοιχείο επιβάλλει πλήρη κατολίπηση. Ταυτόχρονα ο σωρός παύει να χωρά στις ταχύτερες βαθμίδες κρυφής μνήμης, οπότε

κάθε κατολίσθηση γίνεται ακολουθία προσβάσεων τυχαίας διεύθυνσης, βάθους λογαριθμικού ως προς k . Ο αλγόριθμος μετατρέπεται έτσι από φραγμένος στο εύρος ζώνης σε φραγμένο από την καθυστέρηση και στη μεγαλύτερη τιμή k καταλήγει βραδύτερος από τη διτονική ταξινόμηση.

Δευτερεύων αλλά υπαρκτός παράγοντας είναι τα όρια κλιμάκωσης νημάτων. Επειδή το `map-reduce` χρειάζεται περίπου ένα εκατομμύριο στοιχεία ανά νήμα, σε μεσαία μεγέθη εισόδου δουλεύει μόνο μέρος των διαθέσιμων πυρήνων. Μέρος της απόστασης από το ταβάνι εκεί οφείλεται απλώς στο ότι δεν εργάζεται ολόκληρος ο επεξεργαστής.

Κεφάλαιο 5

Υλοποίηση σε GPU

5.1 Σχεδιαστικές επιλογές

Αμφότεροι οι αλγόριθμοι εδώ αντιμετωπίζουν δύο βασικές προκλήσεις: τη διαχείριση των τύπων δεδομένων και την κατανομή του φόρτου εργασίας στους πόρους εκτέλεσης της συσκευής.

Ο χειρισμός τύπων ακολουθεί τη φιλοσοφία του επεξεργαστή σε απλούστερη μορφή. Συγκεκριμένα, μια δομή ανά τύπο εκθέτει όλες τις απαραίτητες λειτουργίες για τους αλγορίθμους (συγκρίσεις, εύρεση ελαχίστου/μεγίστου), επιτρέποντας στον κύριο κώδικα να γραφτεί μία μόνο φορά, ανεξαρτήτως τύπου. Η ειδοποιός διαφορά σε σχέση με τον επεξεργαστή είναι πως, στη μονάδα γραφικών, δεν τίθεται ζήτημα δυναμικής επιλογής πλάτους κατά την εκτέλεση. Ο μεταγλωττιστής παράγει απευθείας τον βέλτιστο κώδικα για τη συγκεκριμένη αρχιτεκτονική της συσκευής, συνεπώς η αφαιρετική διεπαφή χρησιμεύει αποκλειστικά για τη διαφοροποίηση των τύπων.

Επιπλέον, απαιτείται ρητή αντιστοίχιση μεταξύ των τύπων του host και της συσκευής, δεδομένου ότι η μισή ακρίβεια εκπροσωπείται από διαφορετικό τύπο στα δύο περιβάλλοντα. Η αντιστοίχιση αυτή υλοποιείται κεντρικά σε ένα μόνο σημείο, παραμένοντας απόλυτα διαφανής προς τους αλγορίθμους, οι οποίοι είναι γραμμένοι έτσι ώστε να αγνοούν το περιβάλλον στο οποίο εκτελούνται.

5.1.1 Αντιστοίχιση σε πλέγμα και ομάδες νημάτων

Η συσκευή εκτελεί νήματα οργανωμένα σε ομάδες και τις ομάδες σε πλέγμα. Η διάκριση είναι ουσιαστική, διότι νήματα της ίδιας ομάδας μοιράζονται γρήγορη μνήμη και συγχρονίζονται μεταξύ τους, ενώ νήματα διαφορετικών ομάδων στερούνται τέτοιες δυνατότητες. Κάθε αλγόριθμος πρέπει επομένως να τοποθετεί τις εξαρτήσεις του εντός ομάδων, όπου αυτό καθίσταται δυνατόν.

Χρησιμοποιούνται 256 νήματα ανά ομάδα και στις δύο υλοποιήσεις, αριθμός επαρκής για την κάλυψη των καθυστερήσεων της μνήμης, χωρίς να περιορίζεται το πλήθος των ομάδων που συνυπάρχουν σε κάθε πολυεπεξεργαστή.

Το μέγεθος του πλέγματος προκύπτει με διαφορετικό τρόπο στους δύο αλγορίθμους. Στη διτονική ταξινόμηση καθορίζεται αυστηρά από τα δεδομένα. Κάθε ομάδα αναλαμβάνει ένα πλακίδιο, οπότε το πλήθος των ομάδων ταυτίζεται με το πλήθος των πλακιδίων

στα οποία χωρίζεται το ενεργό τμήμα. Η αντιστοίχιση είναι απόλυτη χωρίς ελευθερία επιλογής. Στο map-reduce, αντίθετα, το πλέγμα είναι σχεδιαστική απόφαση, περιορισμένη από την κατάληψη που επιτρέπει το υλικό, τη συνθήκη πλήρωσης σωρών και τον διαθέσιμο χώρο εργασίας (ενότητα 5.3.1).

Ανεξάρτητα από το μέγεθος του πλέγματος, η σειρά προσπέλασης μνήμης έχει καθοριστική σημασία. Ταυτόχρονες προσβάσεις γειτονικών νημάτων συγχωνεύονται σε ενιαίες συναλλαγές μόνο αν αφορούν γειτονικές διευθύνσεις.

5.2 Διτονική ταξινόμηση

Το χρονοδιάγραμμα επιπέδων είναι το ίδιο με της ενότητας 4.2, άλλωστε το κατασκευάζει κοινός κώδικας. Αλλάζει μόνο ο τρόπος εκτέλεσης, μα το πρόβλημα μένει το ίδιο. Υπάρχουν πολλά επίπεδα και η αφελής αντιστοίχιση ενός επιπέδου σε μία εκκίνηση πυρήνα θα σήμαινε δεκάδες διαδοχικά περάσματα πάνω από την καθολική μνήμη.

Εδώ προστίθεται κι ένα δεύτερο κόστος που δεν υπάρχει στον επεξεργαστή. Κάθε εκκίνηση πυρήνα έχει σταθερή επιβάρυνση, που εκατοντάδες εκκινήσεις πληρώνουν εκατοντάδες φορές. Ολόκληρο το δίκτυο καταγράφεται λοιπόν μία φορά ως γράφος εκτέλεσης, αντί να εκκινείται πυρήνα προς πυρήνα, και αναπαράγεται. Η δρομολόγηση γίνεται έτσι μία και μόνη φορά για όλο το δίκτυο.

Ο γράφος συντίθεται από τρία είδη πυρήνων, με την επιλογή να καθορίζεται από την απόσταση j του επιπέδου, όπως στον επεξεργαστή. Όταν η απόσταση είναι μικρή, μια ολόκληρη ακολουθία επιπέδων τρέχει μέσα στην κοινή μνήμη μιας ομάδας νημάτων (ενότητα 5.2.1). Όταν είναι πολύ μεγάλη ώστε να μη χωρά σε πλακίδιο, τα δεδομένα μένουν στην καθολική μνήμη και ένας πυρήνας εκτελεί έως πέντε διαδοχικά επίπεδα κρατώντας τα ενδιάμεσα σε καταχωρητές. Τα επίπεδα περικοπής έχουν δικό τους πυρήνα, που διαβάσει δύο στοιχεία και γράφει ένα σε δεύτερο πίνακα, με εναλλαγή όπως στον επεξεργαστή.

5.2.1 Οργάνωση σε πλακίδια και κοινή μνήμη

Η κοινή μνήμη μιας ομάδας νημάτων είναι ρητά διαχειριζόμενη, σε αντίθεση με την αυτόματη κρυφή μνήμη του επεξεργαστή. Η υλοποίηση φορτώνει εκεί το πλακίδιο κάθε ομάδας (ενότητα 5.1.1), εκτελεί όσα επίπεδα είναι δυνατόν εξ ολοκλήρου μέσα της και το γράφει πίσω μία φορά. Καμία επιπλέον κίνηση προς την καθολική μνήμη για όσα επίπεδα συγχωνευθούν έτσι.

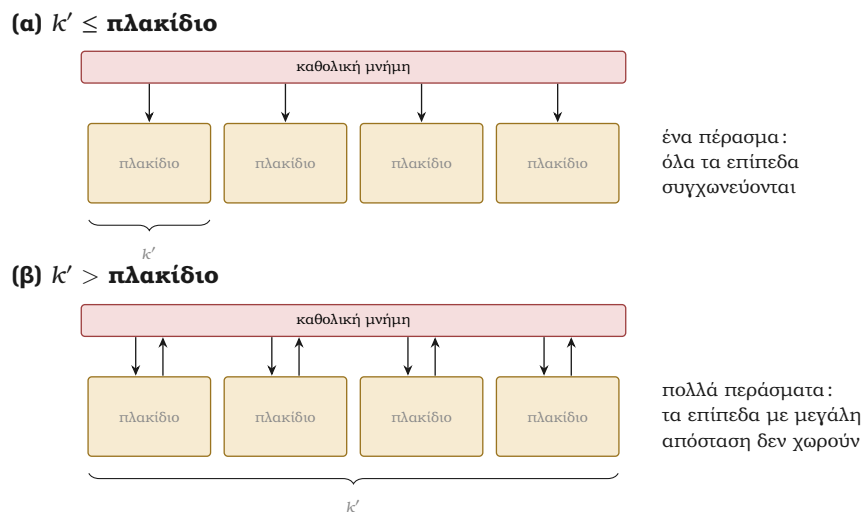
Το μέγεθος πλακιδίου προκύπτει από τους εξής περιορισμούς: δεν ξεπερνά τα 48 KB ανά ομάδα χωρίς ειδική ρύθμιση και μικραίνει όταν το σύνολο εργασίας χωρά στην κρυφή μνήμη δεύτερου επιπέδου. Εκεί η συμφόρηση μετατοπίζεται από την κίνηση προς την κύρια μνήμη στο πλήθος ομάδων ταυτόχρονα ενεργών. Για ακεραίους 32 bit το πλακίδιο καταλήγει στα 8192 στοιχεία.

Ο συγκεκριμένος περιορισμός είναι εγγενής στην αρχιτεκτονική. Οι δημοσιευμένες μέθοδοι μερικής ταξινόμησης για μονάδες γραφικών υπόκεινται στο ίδιο φράγμα, περιορίζοντας το k σε μερικές εκατοντάδες, αφού οι δομές τους πρέπει να χωρούν στην κοινή μνήμη και στους καταχωρητές [3, 5].

Ένα επίπεδο συγχωνεύεται μόνο αν η απόσταση των στοιχείων που συγκρίνει χωρά στο πλακίδιο, μικρότερη από το ήμισυ του μεγέθους του. Οι αποστάσεις της πρώτης φάσης φτάνουν το ήμισυ του μεγέθους των ταξινομημένων μπλοκ, άρα το κρίσιμο μέγεθος είναι το k' της ενότητας 4.2. Όσο τα μπλοκ χωρούν στο πλακίδιο, σχεδόν όλα τα επίπεδα συγχωνεύονται (σχήμα 5.1).

Για μικρές και μεσαίες τιμές k αυτό ισχύει και η κίνηση δεδομένων βγαίνει ουσιαστικά ίδια με τον επεξεργαστή (περίπου επταπλάσια της αφελούς εκτίμησης) με τις δύο αρχιτεκτονικές να συμφωνούν μέχρι το τελευταίο byte για k έως 4096 (ενότητα 3.3.4). Έχουν ίδιο χρονοδιάγραμμα, ισοδύναμη στρατηγική συγχώνευσης, απλώς ρητά διαχειριζόμενη μνήμη έναντι αυτόματης.

Στη μεγαλύτερη τιμή k η συνθήκη καταρρέει. Τα ταξινομημένα μπλοκ φτάνουν 131072 στοιχεία, δεκαέξι φορές το πλακίδιο. Το μεγαλύτερο μέρος των επιπέδων της πρώτης φάσης δεν συγχωνεύεται πια και εκτελείται με περάσματα πάνω από την καθολική μνήμη. Η μετρούμενη κίνηση ανεβαίνει αντίστοιχα, από επταπλάσια σε εικοσιδιπλάσια έως εικοσιεννιαπλάσια της αφελούς εκτίμησης ανάλογα με το μέγεθος εισόδου.



Η μετρούμενη κίνηση περνά από $7n$ σε $25n$ έως $56n$ όταν το k' ξεπεράσει το πλακίδιο.

Σχήμα 5.1: Η σχέση του ταξινομημένου μπλοκ με το πλακίδιο της κοινής μνήμης. Ένα επίπεδο μπορεί να εκτελεστεί χωρίς επιστροφή στην καθολική μνήμη μόνο αν η απόσταση των στοιχείων που συγκρίνει χωρά στο πλακίδιο. Όσο τα ταξινομημένα μπλοκ μεγέθους k' χωρούν σε αυτό (α), σχεδόν όλα τα επίπεδα συγχωνεύονται σε ένα πέρασμα και η κίνηση δεδομένων ταυτίζεται με εκείνη του επεξεργαστή. Όταν το k' ξεπεράσει το μέγεθος του πλακιδίου (β), τα επίπεδα με μεγάλη απόσταση εκτελούνται με ξεχωριστά περάσματα πάνω από την καθολική μνήμη και η κίνηση πολλαπλασιάζεται. Για ακεραίους των 32 bit το πλακίδιο είναι 8192 στοιχεία, οπότε η μετάβαση συμβαίνει μεταξύ των δύο μεγαλύτερων τιμών του k που εξετάστηκαν.

5.3 Map-reduce με τοπικούς σωρούς

Ο αλγόριθμος είναι ίδιος με την ενότητα 4.3. Κάθε νήμα κρατά σωρό μεγέθους k , όπου η ρίζα λειτουργεί ως κατώφλι και τα επιμέρους αποτελέσματα συγχωνεύονται στο τέλος.

Η διαφορά είναι η κλίμακα της αποσύνθεσης.

Στον επεξεργαστή τα νήματα είναι λίγα, καθένα με συνεχόμενο τμήμα εισόδου. Στη μονάδα γραφικών χιλιάδες νήματα διατρέχουν την είσοδο με βήμα ίσο με το συνολικό τους πλήθος, ώστε γειτονικές προσβάσεις να συγχωνεύονται σε ενιαίες συναλλαγές μνήμης.

Κάθε νήμα κρατά δικό του σωρό. Οι σωροί δεν χωρούν στην κοινή μνήμη, διότι το γινόμενο νημάτων επί k ξεπερνά κατά πολύ τα διαθέσιμα 48 KB, και φυλάσσονται σε χώρο εργασίας στην καθολική μνήμη. Εδώ η υλοποίηση αποκλίνει από τον επεξεργαστή ως προς την κίνηση δεδομένων. Με χιλιάδες νήματα, ο όρος που εκεί ήταν αμελητέος πια παύει να είναι. Επειδή κάθε νήμα απαιτεί σωρό μεγέθους k , το συνολικό πλήθος νημάτων υπόκειται σε ανώτατο όριο μνήμης, δεσμευτικό για τα μεγάλα k .

Η συγχώνευση γίνεται σε δύο επίπεδα. Εντός κάθε ομάδας οι σωροί συνενώνονται ανά ζεύγη σε δέντρο, με λογαριθμικό πλήθος βημάτων ως προς το μέγεθος ομάδας. Το αποτέλεσμα κάθε ομάδας συγχωνεύεται στη συνέχεια με τα υπόλοιπα. Η δομή αυτή αποφεύγει τη σειριακή φάση του επεξεργαστή, αλλά εισάγει δικό της κόστος, καθώς κάθε βήμα του δέντρου διαβάζει και γράφει σωρούς από την καθολική μνήμη.

5.3.1 Όριο παραλληλίας για μεγάλα k

Όλη η αποτελεσματικότητα εξαρτάται από την απόρριψη. Κάθε στοιχείο που δεν ξεπερνά τη ρίζα κοστίζει μία σύγκριση. Η ρίζα όμως αποκτά νόημα κατωφλίου μόνο αφού γεμίσει ο σωρός. Με λιγότερα από k στοιχεία μέσα, κάθε νέο εισάγεται υποχρεωτικά, δίχως καμία απόρριψη.

Από αυτό προκύπτει ποσοτικός περιορισμός. Αν είσοδος μεγέθους n μοιραστεί σε P νήματα κάθε νήμα βλέπει n/P στοιχεία και ο σωρός του γεμίζει μόνο αν $n/P \geq k$, δηλαδή

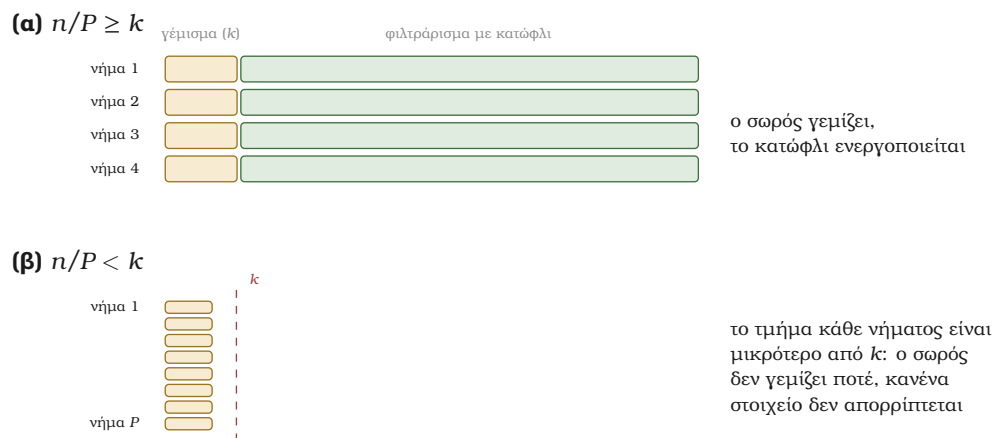
$$P \leq \frac{n}{k}$$

Αυτό θέτει ανώτατο όριο στο πλήθος νημάτων που συνεισφέρουν ουσιαστικά. Πέρα από αυτό, τα επιπλέον νήματα δεν επιταχύνουν τον υπολογισμό, τον επιβαρύνουν. Γεμίζουν χώρο εργασίας με σωρούς που δεν φιλτράρουν ποτέ και προσθέτουν επίπεδα στο δέντρο συγχώνευσης.

Ο περιορισμός είναι δεσμευτικός εκεί όπου η μονάδα γραφικών θα ήταν πιο χρήσιμη. Στη μεγαλύτερη τιμή k και είσοδο δεκαέξι εκατομμυρίων στοιχείων, ο λόγος n/k είναι μόλις 128. Μια συσκευή με χιλιάδες πυρήνες μπορεί να απασχολήσει ουσιαστικά εκατόν είκοσι οκτώ νήματα, κλάσμα της τάξης του ένα τοις εκατό των διαθέσιμων. Κανένας κώδικας δεν αποφεύγει αυτή την υποαπασχόληση, αφού προέρχεται από τον ίδιο τον τρόπο δουλειάς του αλγορίθμου.

Η υλοποίηση αντιδρά στην παραπάνω συνθήκη. Πριν την εκκίνηση υπολογίζεται αν οι σωροί προλαβαίνουν να γεμίσουν με πλήρη κατάληψη της συσκευής. Αν ναι, το πλήθος ομάδων περιορίζεται σκόπιμα, αντιστρόφως ανάλογα του k , ώστε κάθε νήμα να αναλάβει μεγαλύτερο μερίδιο και ο σωρός του να φιλτράρει αποδοτικά. Ονομαστική παραλληλία θυσιάζεται για πραγματική απόρριψη. Όταν οι σωροί δεν πρόκειται να γεμίσουν σε καμία περίπτωση, η μείωση δεν θα πρόσφερε τίποτα και η συσκευή δουλεύει με πλήρη κατάληψη (σχήμα 5.2).

Σε κάθε εκτέλεση επικρατεί ο αυστηρότερος από τους περιορισμούς. Το όριο μνήμης της ενότητας 5.3 δρα συμπληρωματικά μ' αυτόν, εξηγώντας από κοινού γιατί η υλοποίηση αδυνατεί να αξιοποιήσει τη συσκευή στα μεγάλα k .



Η ωφέλιμη παραλληλία φράσσεται επομένως στο $P \leq n/k$, ανεξάρτητα από το μέγεθος του πλέγματος.

Σχήμα 5.2: Το όριο παραλληλίας του map-reduce για μεγάλα k . Κάθε νήμα διατηρεί δικό του σωρό μεγέθους k και μπορεί να απορρίψει στοιχεία μόνο αφού ο σωρός γεμίσει, δηλαδή μόνο αφού δει τουλάχιστον k στοιχεία. Όταν το τμήμα που αναλογεί σε κάθε νήμα είναι μεγαλύτερο από k (α), το κατώφλι ενεργοποιείται νωρίς και το υπόλοιπο του τμήματος φιλτράρεται. Όταν όμως το πλήθος των νημάτων αυξηθεί τόσο ώστε το τμήμα να γίνει μικρότερο από k (β), κανένας σωρός δεν γεμίζει και το φιλτράρισμα δεν ενεργοποιείται ποτέ. Πρόκειται για ιδιότητα του αλγορίθμου και όχι της υλοποίησης: ούτε το μέγεθος του πλέγματος ούτε η διάταξη του χώρου εργασίας μπορούν να την άρουν.

5.4 Διαχείριση μνήμης και κίνηση δεδομένων

Η μονάδα γραφικών χαρακτηρίζεται από δύο ξεχωριστά συστήματα μνήμης συνδεδεμένα με δίαυλο ασύγκριτα βραδύτερο κι από τα δύο. Η τοπική μνήμη της συσκευής αποδίδει περίπου 320 GB/s, ο δίαυλος προς τον host μόλις 14 GB/s, διαφορά πάνω από εικοσαπλάσια.

Εντός της συσκευής η εικόνα μοιάζει με του επεξεργαστή. Για μικρό k η διτονική ταξινόμηση διακινεί το ίδιο επταπλάσιο όγκο. Η κίνηση του map-reduce όμως εξαρτάται αισθητά από το μέγεθος εισόδου, λόγω του χώρου εργασίας ανά νήμα (ενότητα 5.3).

Η μεταφορά προς και από τη συσκευή είναι όμως εκείνη που κυριαρχεί. Σε τέσσερα εκατομμύρια ακεραίους, τα δεκαεπτά περίπου MB θέλουν πάνω από ένα χιλιοστό δευτερολέπτου μόνο για να φτάσουν, ενώ ο ίδιος ο πυρήνας της διτονικής ταξινόμησης τελειώνει σε μισό. Η μεταφορά κοστίζει πάνω από διπλάσια από τον υπολογισμό που εξυπηρετεί.

Αυτό εξηγεί γιατί η εργασία μετρά δύο πεδία (ενότητα 3.3.1): για την ίδια εκτέλεση σύγκριση μόνο στο αλγοριθμικό πεδίο θα έδινε στη μονάδα γραφικών πλεονέκτημα που δεν έχει όταν το πρόβλημα τίθεται ως μεταφορά από την κύρια μνήμη και πίσω.

Η καθιερωμένη τεχνική βελτίωσης μεταφορών, κλειδωμένη μνήμη στον host για να αποφεύγεται ενδιάμεση αντιγραφή, δεν προσφέρει εδώ ουσιαστικό όφελος. Οι δύο διαδρομές διαφέρουν κατά περίπου δύο τοις εκατό, ένδειξη ότι τον περιορισμό τον θέτει ο ίδιος ο δίαυλος.

Ως προς το αποτύπωμα της συσκευής, οι δύο υλοποιήσεις ακολουθούν τη λογική του επεξεργαστή. Η διτονική ταξινόμηση δεσμεύει δύο πίνακες μεγέθους n (ενότητα 5.2) και το `map-reduce` τους σωρούς ανά νήμα (ενότητα 5.3). Για τα μεγάλα k , ο χώρος εργασίας των σωρών μπορεί να πλησιάσει σε μέγεθος την ίδια την είσοδο.

5.5 Υλοποίηση αναφοράς

Το μέτρο σύγκρισης είναι η συνάρτηση επιλογής `top-k` της CUB, που συνοδεύει την εργαλειοθήκη CUDA. Στον επεξεργαστή η υλοποίηση αναφοράς ήταν μονονηματική και η σύγκριση συγκέντρωνε δύο αιτίες. Εδώ η βιβλιοθήκη τρέχει στην ίδια συσκευή, αξιοποιεί την ίδια παραλληλία και μετριέται με τον ίδιο τρόπο. Η σύγκριση είναι επομένως αυστηρή.

Η ενσωμάτωση ακολουθεί το πρωτόκολλο δύο κλήσεων της βιβλιοθήκης. Η πρώτη δηλώνει τον απαιτούμενο βοηθητικό χώρο χωρίς υπολογισμό, ο χώρος δεσμεύεται, και η δεύτερη εκτελεί την επιλογή. Μόνο η δεύτερη περικλείεται από τα συμβάντα που ορίζουν το αλγοριθμικό πεδίο. Δέσμευση και μεταφορές μένουν εκτός, όπως και στις δύο υλοποιήσεις της εργασίας.

Η καταμετρούμενη κίνηση και το πλήθος συγκρίσεων αναφέρονται κι εδώ ως κάτω φράγμα, για τον λόγο της ενότητας 4.5. Μόνο είσοδος και έξοδος, τα εσωτερικά περάσματα δεν είναι παρατηρήσιμα.

Στη μεγαλύτερη εξεταζόμενη είσοδο η βιβλιοθήκη ολοκληρώνει σε χρόνο σχεδόν σταθερό καθώς το k μεταβάλλεται κατά τέσσερις τάξεις μεγέθους, ενώ οι δύο υλοποιήσεις της εργασίας επιβαρύνονται σημαντικά. Μόνο για το μικρότερο k το `map-reduce` την προσπερνά, και μόλις οριακά.

5.5.1 Επιλογή με βάση την αναπαράσταση στη CUB

Η βιβλιοθήκη [14] χρησιμοποιεί επιλογή με βάση την αναπαράσταση (ενότητα 2.2.4). Δεν ταξινομεί ποτέ ολόκληρη την είσοδο, εκτελεί λίγα περάσματα, καθένα ακολουθιακή ανάγνωση με μέτρηση συχνοτήτων, ακριβώς το μοτίβο που η αρχιτεκτονική εκτελεί καλύτερα. Επίσης, το πλήθος περασμάτων καθορίζεται από το πλάτος του τύπου δεδομένων. Η επιλογή οκτώ στοιχείων κοστίζει ουσιαστικά όσο η επιλογή εκατόν τριάντα χιλιάδων.

Η ανεξαρτησία από το k είναι η θεμελιώδης διαφορά από αμφοτέρους τους αλγορίθμους της εργασίας. Η διτονική ταξινόμηση επιβαρύνεται καθώς μεγαλώνει το k : μεγαλώνουν τα ταξινομημένα μπλοκ, παύουν να χωρούν στο πλακίδιο (ενότητα 5.2.1). Το `map-reduce` επιβαρύνεται ακόμη περισσότερο (ενότητα 5.3.1). Η επιλογή με βάση την αναπαράσταση δεν έχει κανένα αντίστοιχο σημείο κατάρρευσης. Δεν είναι όμως αδιάφορη ως προς τα δεδομένα, με τον τρόπο της ενότητας 2.2.4. Εξαρτάται από την κατανομή των ψηφίων στις τιμές.

5.6 Περιοριστικοί παράγοντες

Τα δύο πεδία μέτρησης εδώ σημαίνουν διαφορετικούς περιοριστικούς παράγοντες. Στο end-to-end κυριαρχεί ο δίαυλος προς τον host (ενότητα 5.4). Ό,τι ακολουθεί αφορά το αλγοριθμικό πεδίο, όπου φαίνονται τα όρια της ίδιας της συσκευής.

Για μικρό k η διτονική ταξινόμηση περιορίζεται σαφώς από τη μνήμη, φτάνοντας τα τρία πέμπτα περίπου του μετρημένου ταβανιού εύρους ζώνης, με αξιοποίηση του υπολογιστικού μόλις δυόμισι τοις εκατό. Αξιοποιεί τη συσκευή αποδοτικά, απλώς πρέπει να κινήσει περισσότερο όγκο.

Η βιβλιοθήκη αναφοράς κάνει τη διάκριση ποσοτική, και είναι το πιο διδακτικό σημείο του κεφαλαίου. Αξιοποιεί μόλις το ένα πέμπτο περίπου του ταβανιού εύρους ζώνης, σαφώς λιγότερο από τη διτονική ταξινόμηση, και παρ' όλα αυτά ολοκληρώνει ταχύτερα. Η εγγύτητα στο ταβάνι μετρά την ποιότητα της υλοποίησης, ενώ ο συνολικός όγκος κίνησης μετρά την ποιότητα του αλγορίθμου.

Το map-reduce για μικρό k είναι το μόνο σημείο όπου μια υλοποίηση της εργασίας υπερτερεί της βιβλιοθήκης, έστω οριακά. Διαβάζει την είσοδο μία φορά, απορρίπτει σχεδόν τα πάντα, πλησιάζοντας το ελάχιστο δυνατό έργο. Η αξιοποίηση ταβανιού μένει μέτρια, δείχνοντας περιθώριο βελτίωσης στην οργάνωση προσβάσεων.

Για μεγάλο k το map-reduce καταρρέει σε βαθμό που η αξιοποίηση αμφοτέρων των ταβανιών γίνεται κλάσμα του ένα τοις εκατό, ελλείπει εργασίας προς εκτέλεση. Όπως εξηγήθηκε στην ενότητα 5.3.1, στα μεγάλα μεγέθη του k η συντριπτική πλειονότητα των νημάτων δεν προλαβαίνει να γεμίσει τον τοπικό της σωρό, αφήνοντας το μεγαλύτερο μέρος της συσκευής αδρανές.

Η διτονική ταξινόμηση επιβαρύνεται πολύ ηπιότερα. Η αξιοποίηση εύρους ζώνης υποχωρεί περίπου στο ένα τρίτο, καθώς η απώλεια της συγχώνευσης στην κοινή μνήμη αυξάνει την κίνηση χωρίς αντίστοιχη αύξηση χρήσιμης εργασίας. Παραμένει λειτουργική, και στο μεγαλύτερο k καταλήγει ταχύτερη από το map-reduce κατά δύο τάξεις μεγέθους.

Καμία από τις δύο δεν περιορίζεται τελικά από κάτι που θα διόρθωνε η περαιτέρω βελτιστοποίηση κώδικα, αφού το εμπόδιο βρίσκεται στον ίδιο τον αλγόριθμο. Η διτονική ταξινόμηση θα ωφελούνταν μόνο από λιγότερη κίνηση, δηλαδή διαφορετικό δίκτυο, ενώ το map-reduce μόνο από διαφορετική αποσύνθεση για μεγάλα k , ώστε η παραλληλία να μην εξαρτάται από τον λόγο n/k .

Κεφάλαιο 6

Υλοποίηση σε NPU

6.1 Σχεδιαστικές επιλογές

Η μονάδα νευρωνικής επεξεργασίας είναι άρδην διαφορετική αρχιτεκτονική από τις δύο προηγούμενες. Δεν εκτελεί ένα πρόγραμμα με τη συνήθη έννοια, γιατί είναι διάταξη πλακιδίων διαμορφωμένη εκ των προτέρων, χωρίς κρυφή μνήμη, χωρίς δυναμική δέσμευση μνήμης και χωρίς δυνατότητα να αλλάξει τη ροή δεδομένων της κατά την εκτέλεση.

Στον επεξεργαστή και στη μονάδα γραφικών το ζητούμενο ήταν πώς κατανέμεται ο αλγόριθμος σε νήματα. Εδώ το ζητούμενο είναι ποιο μικρό και σταθερό κομμάτι εργασίας εκτελεί η διάταξη αποδοτικά και πώς την τροφοδοτεί αδιάκοπα ο host.

6.1.1 Ροή ανάπτυξης με MLIR-AIE

Δεν υπάρχει περιβάλλον προγραμματισμού αντίστοιχο εκείνου της μονάδας γραφικών. Η εργασία στηρίζεται στο MLIR-AIE [13] και στη διεπαφή IRON [20]. Η ανάπτυξη απαιτεί δύο ξεχωριστά τεχνουργήματα, από διαφορετικές αλυσίδες εργαλείων, που συνενώνονται στο τέλος.

Το πρώτο περιγράφει την ίδια τη διάταξη: ποια πλακίδια χρησιμοποιούνται, ποια κανάλια τα συνδέουν, με ποιο μοτίβο κινούνται τα δεδομένα από και προς την κύρια μνήμη. Γράφεται σε ειδική γλώσσα ενσωματωμένη στην Python, μεταφράζεται σε ενδιάμεση αναπαράσταση και τελικά σε δυαδικό αρχείο διαμόρφωσης υλικού. Το δεύτερο είναι ο υπολογιστικός πυρήνας κάθε πλακιδίου, γραμμένος σε C++ με τα διανυσματικά πρωτογενή της αρχιτεκτονικής και μεταγλωττισμένος χωριστά.

Η διαμόρφωση είναι σταθερή κατά τη μεταγλώττιση. Η τοπολογία, τα μεγέθη ενδιάμεσων απομονωτών, το σχήμα μεταφοράς, είναι όλα αποφασισμένα πριν από κάθε εκτέλεση, ανεξάρτητα από τις παραμέτρους του προβλήματος. Αλλάζοντας το k ή το μέγεθος εισόδου δεν αλλάζει τίποτα στη συσκευή, μόνο ο τρόπος που ο host τεμαχίζει την είσοδο και ερμηνεύει τα αποτελέσματα.

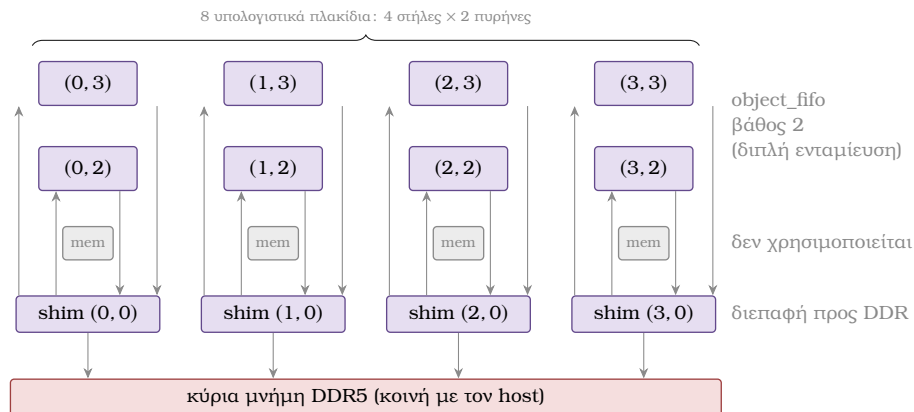
6.1.2 Τοπολογία πλακιδίων και object_fifo

Η διάταξη οργανώνεται σε στήλες. Κάθε στήλη έχει πλακίδιο διασύνδεσης που επικοινωνεί με την κύρια μνήμη, και πάνω από αυτό, υπολογιστικά πλακίδια.

Η μεταφορά δεδομένων δεν προγραμματίζεται με ρητές εντολές αντιγραφής. Δηλώνεται ως κανάλι μεταξύ δύο πλακιδίων. Κάθε κανάλι συνοδεύεται από πλήθος ενδιάμεσων απομονωτών, εδώ δύο. Αυτό επιτρέπει την επικάλυψη, όσο το πλακίδιο επεξεργάζεται τον έναν απομονωτή, ο μηχανισμός άμεσης προσπέλασης μνήμης γεμίζει τον άλλο. Χωρίς αυτό κάθε πλακίδιο θα σταματούσε σε κάθε μεταφορά. Η τεχνική αυτή είναι γνωστή ως διπλή ενταμίευση.

Κάθε υπολογιστικό πλακίδιο έχει δικό του ζεύγος καναλιών προς το πλακίδιο διασύνδεσης της στήλης του, ένα για είσοδο, ένα για έξοδο. Δεν υπάρχει επικοινωνία μεταξύ υπολογιστικών πλακιδίων, κάτι που αποτελεί συνειδητή επιλογή. Η εργασία που τους ανατίθεται είναι εξ ορισμού ανεξάρτητη ανά πλακίδιο και καμία ανταλλαγή δεν απαιτείται.

Η ποσότητα δεδομένων ανά αποστολή προκύπτει από το γινόμενο των παραπάνω (σχήμα 6.1). Κάθε πλακίδιο δέχεται τμήματα 1024 στοιχείων, 128 τέτοια τμήματα ανά αποστολή, οπότε το μέγεθος της παρτίδας κλιμακώνεται με το πλήθος των πλακιδίων που χρησιμοποιεί ο εκάστοτε αλγόριθμος. Μεγαλύτερες εισοδοί τεμαχίζονται σε διαδοχικές παρτίδες.



Ανά πυρήνα: 128 τεμάχια $\times$ 1024 στοιχεία int32. Ανά παρτίδα: $8 \times 128 \times 1024 = 1\,048\,576$ στοιχεία.

Σχήμα 6.1: Η τοπολογία πλακιδίων της υλοποίησης και οι ουρές object_fifo που τα συνδέουν. Κάθε στήλη διαθέτει ένα πλακίδιο διεπαφής (shim) που επικοινωνεί με την κύρια μνήμη και δύο υπολογιστικά πλακίδια, καθένα από τα οποία συνδέεται με το shim της στήλης του μέσω ενός ζεύγους ουρών βάθους δύο: μία για την είσοδο και μία για την έξοδο. Το βάθος δύο επιτρέπει στον πυρήνα να επεξεργάζεται ένα τεμάχιο ενώ το επόμενο μεταφέρεται. Τα πλακίδια μνήμης της ενδιάμεσης σειράς δεν χρησιμοποιούνται, καθώς κάθε πυρήνας καταναλώνει ξένο προς τους υπόλοιπους τμήμα της εισόδου και δεν απαιτείται διαμοιρασμός ή αναδιανομή δεδομένων μεταξύ τους.

6.2 Διτονική ταξινόμηση

Μια ουσιώδης διαφορά επηρεάζει το σύνολο των μετρήσεων: η διάταξη αποκλίνει από το χρονοδιάγραμμα περικομμένης διτονικής ταξινόμησης της ενότητας 4.2 για αμιγώς δομικούς λόγους. Συγκεκριμένα, το χρονοδιάγραμμα απαιτεί συγκρίσεις μεταξύ στοιχείων σε αυθαίρετα μεγάλη απόσταση, που προϋποθέτει ανταλλαγή δεδομένων μεταξύ πλακιδίων και πολλαπλά περάσματα επί της εισόδου. Η τοπολογία της ενότητας 6.1.2 στερείται αυτών των μηχανισμών, καθώς τα πλακίδια λειτουργούν ανεξάρτητα και κάθε στοιχείο διέρχεται από το σύστημα ακριβώς μία φορά.

Το πρόβλημα αναδιατυπώνεται λοιπόν. Η διάταξη αναλαμβάνει σταθερή, τοπική εργασία. Ταξινομεί ανεξάρτητα διαδοχικές ακολουθίες δεκαέξι στοιχείων και ο host αναλαμβάνει την επιλογή, συγχωνεύοντας αυτές τις ταξινομημένες ακολουθίες σε σωρό μεγέθους k .

Επειδή κάθε ακολουθία 16 στοιχείων επιστρέφει ταξινομημένη, ο host μπορεί να διακόψει την εξέτασή της αμέσως μόλις συναντήσει στοιχείο που δεν υπερβαίνει τη ρίζα του σωρού. Λόγω της ταξινόμησης, είναι εγγυημένο πως όλα τα επόμενα στοιχεία της ίδιας ακολουθίας θα είναι εξίσου ή περισσότερο ακατάλληλα. Η ταξινόμηση στη διάταξη δεν είναι αυτοσκοπός, μα μηχανισμός πρόωρης απόρριψης.

6.2.1 Πυρήνας υπολογιστικού πλακιδίου

Αν και παρέχεται επίσημη υλοποίηση αναφοράς για διτονική ταξινόμηση στο AI Engine [17], το συγκεκριμένο παράδειγμα περιορίζεται στην πλήρη ταξινόμηση εσωτερικά σε ένα μόνο πλακίδιο.

Το μέγεθος των δεκαέξι υπαγορεύεται απευθείας από το υλικό. Αντιστοιχεί ακριβώς στο πλήθος ακεραίων 32 bit που χωρούν σε διανυσματικό καταχωρητή της αρχιτεκτονικής. Δεκαέξι στοιχεία με διτονικό δίκτυο θέλουν δέκα βήματα σύγκρισης-ανταλλαγής, καθένα εξ ολοκλήρου εντός του καταχωρητή.

Κάθε βήμα ακολουθεί το ίδιο μοτίβο με την ενότητα 4.1.1. Αναδιάταξη φέρνει κάθε στοιχείο δίπλα στο ταίρι του, υπολογίζονται ελάχιστο και μέγιστο, και μία μάσκα επιλέγει ποιο κρατά κάθε θέση. Ολόκληρο το δίκτυο τρέχει χωρίς καμία πρόσβαση στη μνήμη του πλακιδίου. Τα δεδομένα φορτώνονται μία φορά, υφίστανται δέκα μετασχηματισμούς σε καταχωρητές και αποθηκεύονται μία φορά.

Κάθε κλήση επεξεργάζεται τμήμα 1024 στοιχείων, εξήντα τέσσερις ανεξάρτητες ακολουθίες. Καμία εξάρτηση δεν υπάρχει μεταξύ τους, γεγονός που επιτρέπει στον μεταγωγτιστή να τις διοχετεύει και να καλύπτει τις καθυστερήσεις των επιμέρους εντολών.

6.2.2 Κλιμάκωση σε πολλαπλά πλακίδια

Οι ακολουθίες είναι ανεξάρτητες, άρα η κλιμάκωση σε περισσότερα πλακίδια είναι τριμμένη. Κάθενα αναλαμβάνει διαφορετικό τμήμα της παρτίδας, χωρίς συγχρονισμό ή ανταλλαγή δεδομένων. Αλλάζει μόνο το μοτίβο μεταφοράς στη διαμόρφωση της συσκευής.

Η υλοποίηση επεκτάθηκε ώστε να χρησιμοποιεί δύο υπολογιστικά πλακίδια ανά στήλη αντί για ένα, οκτώ συνολικά. Η επέκταση απέδωσε αισθητή βελτίωση, επιβεβαιώνοντας

πως μέχρι εκείνο το σημείο η υλοποίηση περιοριζόταν από τον υπολογισμό στη διάταξη. Το σημαντικότερο ήταν πως μετακίνησε τον περιοριστικό παράγοντα αλλού. Με οκτώ πλακίδια η διάταξη έπαψε να είναι το στενό σημείο και ο χρόνος πλέον καταναλώνεται σχεδόν εξ ολοκλήρου στον host. Κλιμάκωση με περισσότερα πλακίδια δεν θα άλλαζε πια τίποτα.

6.3 Map-reduce με τοπικούς σωρούς

Ο δεύτερος αλγόριθμος ακολουθεί την ίδια αρχή κατανομής εργασίας, με ακόμη πιο περιορισμένο ρόλο για τη διάταξη. Χρησιμοποιούνται τέσσερα υπολογιστικά πλακίδια, ένα ανά στήλη, και ο πυρήνας δεν ταξινομεί. Για κάθε τμήμα εισόδου υπολογίζει μία σύνοψη, την καλύτερη τιμή που περιέχει.

Ο host κρατά τον σωρό μεγέθους k και χρησιμοποιεί τις συνόψεις ως φίλτρο. Αν η καλύτερη τιμή ενός ολόκληρου τμήματος δεν ξεπερνά τη ρίζα, το τμήμα απορρίπτεται στο σύνολό του, χωρίς να εξεταστεί κανένα στοιχείο του. Μόνο τα τμήματα που περνούν το φίλτρο εξετάζονται στοιχείο προς στοιχείο.

Το φίλτρο προϋποθέτει ρεαλιστικό κατώφλι πριν το πρώτο τμήμα. Στον επεξεργαστή και στη μονάδα γραφικών αυτό προκύπτει αυτόματα, καθώς ο σωρός αρχικοποιείται μαζικά από τα πρώτα k στοιχεία της εισόδου. Στην περίπτωση της διάταξης, ο host δειγματοληπτεί εκ των προτέρων ένα τμήμα, ίσο σε μέγεθος με το μεγαλύτερο από k και δύο τοις εκατό του n και το εισάγει απευθείας στον σωρό πριν σταλεί η πρώτη εργασία, ώστε το φίλτρο να λειτουργεί αποτελεσματικά από την αρχή.

Η κατανομή ρόλων είναι έτσι αντιδιαδιαθητική. Η διάταξη παράγει την πληροφορία που επιτρέπει στον host να εκτελέσει την επιλογή φθηνά. Αυτό ταιριάζει στη φύση της ως σταθερής, τοπικής και πλήρως προβλέψιμης για πράξεις πάνω σε ροή δεδομένων.

6.4 Διαχείριση μνήμης και κίνηση δεδομένων

Η μονάδα δεν έχει δική της μνήμη και τα δεδομένα δεν διασχίζουν δίαυλο προς ξεχωριστή συσκευή, όπως στη μονάδα γραφικών. Βρίσκονται ήδη στην ίδια φυσική μνήμη με του host και αυτό που ονομάζεται μεταφορά είναι στην πραγματικότητα συγχρονισμός κοινού απομονωτή.

Ως προς το κόστος μεταφοράς αυτό είναι πλεονέκτημα· δεν υπάρχει χάσμα εύρους ζώνης όπως στην ενότητα 5.4. Υπάρχει όμως και η αντίστροφη όψη. Επειδή η μνήμη είναι κοινή, κάθε byte που ετοιμάζει ο host για τη διάταξη καταναλώνει το ίδιο εύρος ζώνης που θα χρησιμοποιούσε αν εκτελούσε ο ίδιος τον υπολογισμό.

6.4.1 Διοχέτευση και ρόλος του host

Για να μη μένει η διάταξη αδρανής μεταξύ διαδοχικών παρτίδων, η εκτέλεση οργανώνεται ως διοχέτευση τριών σταδίων, με δύο σύνολα απομονωτών που εναλλάσσονται. Όσο η διάταξη επεξεργάζεται μια παρτίδα, ο host προετοιμάζει την επόμενη και συγχωνεύει τα αποτελέσματα της προηγούμενης (σχήμα 6.2).

Η μέτρηση των επιμέρους σταδίων είναι ίσως το πιο διαφωτιστικό αποτέλεσμα του κεφαλαίου. Για είσοδο δεκαέξι εκατομμυρίων ακεραίων (δεκαέξι παρτίδες), μέσος όρος

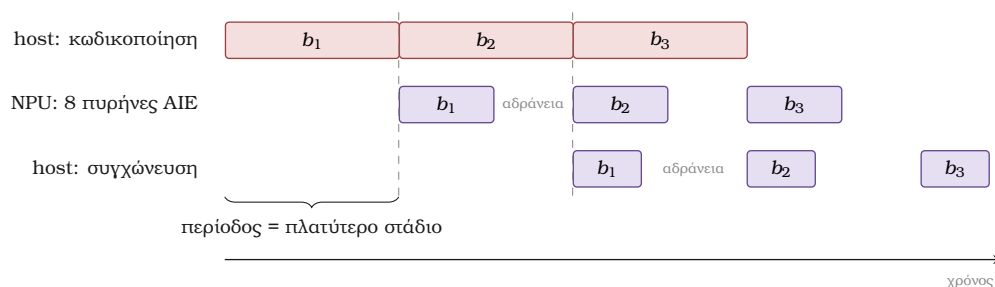
δεκαπέντε επαναλήψεων, ο χρόνος κατανέμεται όπως στον πίνακα 6.1.

Στάδιο	Χρόνος (ms)	Ποσοστό
Προετοιμασία παρτίδας	4,84	58 %
Συγχώνευση αποτελεσμάτων	2,07	25 %
Αναμονή της διάταξης	0,69	8 %
Αποστολή εργασιών	0,10	1 %
Αρχικοποίηση	0,08	1 %
Λοιπά	0,56	7 %
Σύνολο	8,34	100 %

Πίνακας 6.1: Κατανομή του χρόνου της περιοχής εκφόρτωσης ανά στάδιο. Η γραμμή «Λοιπά» καλύπτει το τμήμα της μέτρησης που δεν αποδίδεται σε κανένα από τα υπόλοιπα στάδια, σταθερά παρόν σε κάθε επανάληψη.

Λιγότερο από ένα δέκατο του χρόνου αναλώνεται σε αναμονή της διάταξης. Το υπόλοιπο ενενήντα δύο τοις εκατό είναι δουλειά που κάνει ο ίδιος ο host. Μια διοχέτευση αποδίδει όσο το βραδύτερο στάδιο της και το βραδύτερο στάδιο δεν είναι στη συσκευή.

Γι' αυτό η μέτρηση του αλγοριθμικού πεδίου εδώ δεν πρέπει να διαβάζεται ως χρόνος υπολογισμού των πλακιδίων (ενότητα 3.3.2). Περικλείει ολόκληρη τη διαδικασία εκφόρτωσης, της οποίας το μεγαλύτερο μέρος εκτελείται στον επεξεργαστή.



Σχήμα 6.2: Η διοχέτευση τριών σταδίων της εκφόρτωσης. Ενώ τα πλακίδια επεξεργάζονται την παρτίδα b_i , ο host κωδικοποιεί ήδη την b_{i+1} στον δεύτερο buffer και συγχωνεύει τα αποτελέσματα της b_{i-1} . Επειδή τα τρία στάδια επικαλύπτονται, ο συνολικός χρόνος δεν είναι το άθροισμά τους αλλά το γινόμενο του πλήθους των παρτίδων επί τη διάρκεια του πλατύτερου σταδίου. Στη συγκεκριμένη υλοποίηση το στάδιο αυτό είναι η κωδικοποίηση στον host: μόνο αυτό εκτελείται χωρίς διακοπή, ενώ τα υπόλοιπα δύο μένουν αδρανή για μέρος κάθε περιόδου. Καμία περαιτέρω επιτάχυνση του πυρήνα ΑΙΕ δεν μπορεί επομένως να μειώσει τον συνολικό χρόνο.

6.4.2 Κωδικοποίηση δεδομένων και τύποι

Ο πυρήνας της διάταξης λειτουργεί αποκλειστικά σε ακεραίους των 32 bit. Δεν χρειάζεται να ξέρει τι αναπαριστούν οι τιμές που συγκρίνει. Κάθε τύπος δεδομένων της εργασίας αντιστοιχίζεται λοιπόν σε ακέραιο κλειδί που διατηρεί τη σχετική διάταξη των τιμών, με τους κανόνες μετατροπής τύπων της πλατφόρμας [21].

Η αντιστοίχιση είναι διαφορετική ανά τύπο. Οι προσημασμένοι ακέραιοι χρησιμοποιούνται αυτούσιοι, στους απρόσημους αντιστρέφεται το πλέον σημαντικό bit. Στην απλή

ακρίβεια αντιστρέφεται είτε το πρόσημο είτε το σύνολο των υπόλοιπων bit, ανάλογα με το πρόσημο, ώστε η δυαδική σύγκριση να συμπίπτει με την αριθμητική. Η μισή ακρίβεια διευρύνεται πρώτα σε απλή (ακριβής μετατροπή) και ακολουθεί την παραπάνω διαδρομή.

Η κωδικοποίηση τρέχει στον host κι απαρτίζει μέρος του σταδίου προετοιμασίας του πίνακα 6.1. Η διαχείριση του τύπου δεδομένων επιβαρύνει υπολογιστικά αποκλειστικά τον επεξεργαστή. Μάλιστα, η αρχική, βαθμωτή υλοποίηση της κωδικοποίησης σε μισή ακρίβεια αποδείχθηκε τόσο δαπανηρή που καθυστέρουσε αισθητά το συνολικό στάδιο. Για τον λόγο αυτό, αντικαταστάθηκε από μια διανυσματική διαδρομή, η οποία μετατρέπει οκτώ τιμές ανά εντολή.

6.4.3 Τελική διόρθωση στον host

Η διπλή ακρίβεια δεν χωρά σε κλειδί 32 bit. Η κωδικοποίηση κρατά τα 32 πλέον σημαντικά bit του πλήρους κλειδιού των 64, άρα δύο διαφορετικές τιμές μπορούν να καταλήξουν στο ίδιο κλειδί. Η διάταξη επιστρέφει κάτι ασθενέστερο από το ακριβές αποτέλεσμα, ένα κατώφλι.

Ένα επιπλέον στάδιο στον host αποκαθιστά την ορθότητα. Από τα κλειδιά που επέστρεψε η διάταξη προκύπτει το χειρότερο, το οποίο ορίζει το κατώφλι. Ο host διατρέχει ξανά ολόκληρη την είσοδο, συλλέγει όσα στοιχεία έχουν κλειδί εντός του κατωφλίου και εκτελεί ακριβή επιλογή μεταξύ αυτών.

Το κόστος είναι ένα πρόσθετο πέρασμα πάνω από την είσοδο, που επιβαρύνει κατ'εξαίρεση τη διπλή ακρίβεια. Το όφελος είναι ότι το αποτέλεσμα είναι ακριβές και περνάει τον ίδιο έλεγχο ορθότητας με όλους τους άλλους τύπους δεδομένων, χωρίς χαλάρωση του κριτηρίου. Προσεγγιστικό αποτέλεσμα για έναν μόνο τύπο θα έκανε τη σύγκριση μεταξύ των τύπων άνιση.

6.5 Υλοποίηση αναφοράς

Το μέτρο σύγκρισης είναι η ίδια συνάρτηση της πρότυπης βιβλιοθήκης με τον επεξεργαστή, εκτελούμενη εξ ολοκλήρου στον host. Οι λόγοι της επιλογής αναλύθηκαν στην ενότητα 3.5. Αξίζει μόνο να υπενθυμιστεί το ουσιώδες: δεν υπάρχει υλοποίηση top-k για την ίδια τη διάταξη.

6.6 Περιοριστικοί παράγοντες

Ο περιοριστικός παράγοντας φαίνεται ήδη στον πίνακα 6.1 και είναι ο host. Γιατί όμως είναι τόσο ακριβή η εργασία του host;

Το στάδιο προετοιμασίας (τα τρία πέμπτα του χρόνου) περιλαμβάνει πλήρες πέρασμα ανάγνωσης πάνω από την είσοδο, μετατροπή κάθε στοιχείου σε ακέραιο κλειδί και εγγραφή ισόποσου όγκου στον κοινό απομονωτή. Αθροιστικά προκύπτει κίνηση δεδομένων τουλάχιστον διπλάσια του μεγέθους εισόδου, στην ίδια μνήμη και με το ίδιο εύρος ζώνης που θα χρησιμοποιούσε ο επεξεργαστής. Η συγχώνευση προσθέτει ακόμη ένα πέρασμα ανάγνωσης πάνω από τα αποτελέσματα.

Ακόμη κι αν η διάταξη ολοκλήρωνε τη δική της εργασία ακαριαία, ο συνολικός χρόνος θα μειωνόταν κατά λιγότερο από ένα δέκατο, αφού ο χρόνος δεν δαπανάται εκεί. Η επιτάχυνση επομένως δεν μπορεί να έρθει από τη συσκευή.

Η σύγκριση με τον επεξεργαστή κάνει το επιχείρημα οξύτερο. Ο host που τροφοδοτεί τη μονάδα είναι ο ίδιος επεξεργαστής που λύνει ολόκληρο το πρόβλημα μόνος του. Εδώ του ζητείται σχεδόν η ίδια ποσότητα κίνησης δεδομένων, μόνο και μόνο για να προετοιμάσει τα δεδομένα για κάποιον άλλον. Το κόστος προετοιμασίας είναι συγκρίσιμο με το κόστος της ίδιας της λύσης, γι' αυτό η εκφόρτωση δεν αποδίδει.

Το παραπάνω συμπέρασμα αφορά όμως το συγκεκριμένο πρόβλημα. Η επιλογή top-k έχει ελάχιστη υπολογιστική ένταση, με μία σύγκριση ανά στοιχείο και σχεδόν καμία επαναχρησιμοποίηση. Μια διάταξη ροής δεδομένων αποδίδει όταν κάθε στοιχείο που μπαίνει υφίσταται πολλές πράξεις, ώστε το κόστος τροφοδοσίας να αποσβένεται. Εδώ συμβαίνει το αντίθετο, και αυτή η αναντιστοιχία ανάμεσα στο πρόβλημα και στην αρχιτεκτονική εξηγεί το αποτέλεσμα καλύτερα από οποιαδήποτε λεπτομέρεια υλοποίησης.

Κεφάλαιο 7

Αποτελέσματα και αξιολόγηση

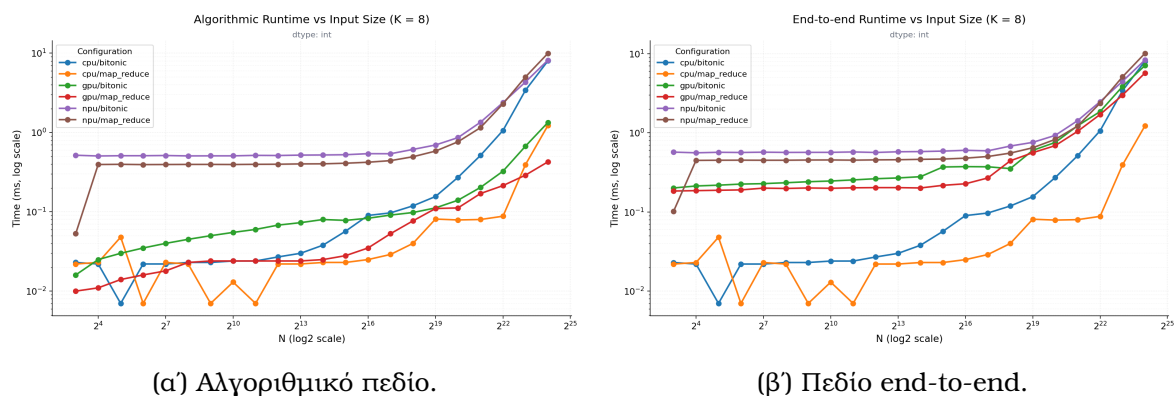
Το παρόν κεφάλαιο παρουσιάζει τα αποτελέσματα και τα ερμηνεύει με βάση τους μηχανισμούς των τριών προηγούμενων κεφαλαίων. Όπου δεν δηλώνεται διαφορετικά, τα διαγράμματα αφορούν ακραίους 32 bit και ομοιόμορφη κατανομή εισόδου. Η επίδραση τύπου δεδομένων και κατανομής εξετάζεται χωριστά στις ενότητες 7.6 και 7.7.

7.1 Ορθότητα

Κάθε σύγκριση χρόνου προϋποθέτει σωστό αποτέλεσμα. Η κύρια σάρωση καλύπτει 2700 περιπτώσεις: δύο αρχιτεκτονικές, τρεις αλγορίθμους, πέντε τύπους δεδομένων, μεγέθη εισόδου από 2^3 έως 2^{24} , τέσσερις τιμές k . Όλες επαληθεύτηκαν ως προς την υλοποίηση επαλήθευσης (ενότητα 3.6) με ποσοστό επιτυχίας 100%. Κάθε χρόνος στη συνέχεια αντιστοιχεί λοιπόν σε σωστό σύνολο των k στοιχείων.

7.2 Χρόνος εκτέλεσης

7.2.1 Εξάρτηση από το μέγεθος εισόδου



(α) Αλγοριθμικό πεδίο.

(β) Πεδίο end-to-end.

Σχήμα 7.1: Χρόνος συναρτήσεως του μεγέθους εισόδου, για k ίσο με 8, στα δύο πεδία μέτρησης. Η κατάταξη των αρχιτεκτονικών ανατρέπεται πλήρως μεταξύ των δύο.

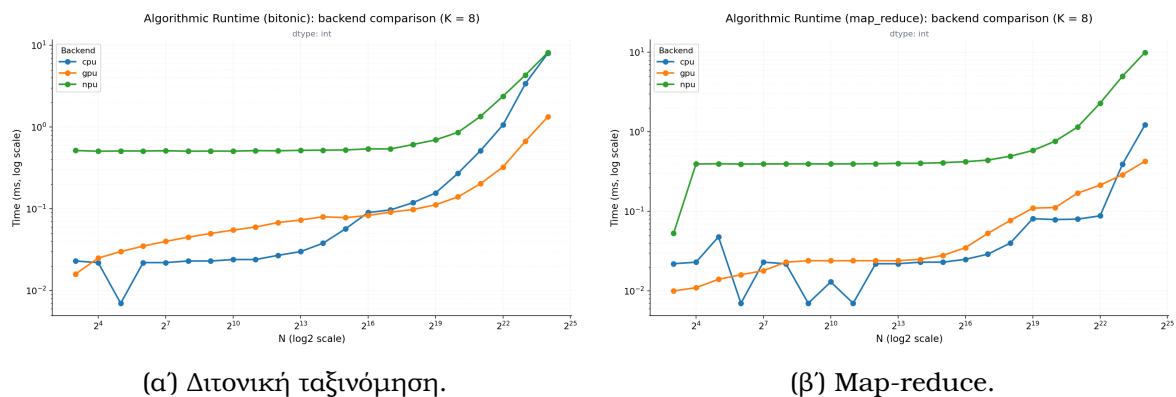
Στο αλγοριθμικό πεδίο (αριστερά), η μονάδα γραφικών είναι ταχύτερη στη μικρότερη είσοδο, 0.010 ms έναντι 0.022 ms του επεξεργαστή, και πάλι στη μεγαλύτερη, 0.43 ms έναντι 1.23 ms. Σχεδόν σε όλη την ενδιάμεση περιοχή όμως προηγείται ο επεξεργαστής. Το ακριβές σημείο εναλλαγής μετακινείται μεταξύ επαναλαμβανόμενων εκτελέσεων της ίδιας σάρωσης. Σε τόσο μικρούς απόλυτους χρόνους αμφότερες οι αρχιτεκτονικές βρίσκονται κοντά στην ανάλυση του χρονομέτρου, οπότε εκεί η κατάταξη αντικατοπτρίζει μάλλον θόρυβο μέτρησης.

Η μονάδα νευρωνικής επεξεργασίας δείχνει σαφές κατώφλι στα μικρά μεγέθη. Καμπύλες επίπεδες γύρω στο μισό χιλιοστό δευτερολέπτου μέχρι περίπου 2^{16} στοιχεία, κυριαρχούμενες από το σταθερό κόστος αποστολής και αναμονής. Ο επεξεργαστής έχει ανάλογο αλλά πολύ χαμηλότερο κατώφλι, τάξης των πέντε εκατοστών του χιλιοστού.

Ένα σημείο μόνο ξεχωρίζει. Στο μικρότερο μέγεθος, όπου n και k ταυτίζονται, το map-reduce της μονάδας νευρωνικής επεξεργασίας πέφτει αισθητά κάτω από αυτό το κατώφλι. Αυτό οφείλεται στη δειγματοληψία της ενότητας 6.3. Όταν $n=k=8$ το δείγμα που τροφοδοτεί τον σωρό καλύπτει ολόκληρη την είσοδο και καμία εργασία δεν φτάνει ποτέ στα πλακίδια.

Στο πεδίο end-to-end (δεξιά), με τη μεταφορά δεδομένων μέσα στη μέτρηση, η εικόνα ανατρέπεται πλήρως. Ο επεξεργαστής είναι ταχύτερος από κάθε άλλη αρχιτεκτονική σε *όλα* τα μεγέθη, ακόμη και στο μεγαλύτερο. Τα 0.43 ms του πυρήνα της μονάδας γραφικών γίνονται 5.69 ms συνολικά, ενώ ο επεξεργαστής μένει στα 1.23 ms, καθώς γι' αυτόν τα δύο πεδία ταυτίζονται.

Το αλγοριθμικό πεδίο δείχνει πόσο καλά υπολογίζει η κάθε συσκευή, το συνολικό απαντά στο αν αξίζει να της ανατεθεί το φορτίο. Η μονάδα νευρωνικής επεξεργασίας χρειάζεται 9.97 ms στη μεγαλύτερη είσοδο, περισσότερο από κάθε άλλη αρχιτεκτονική σε αμφότερα τα πεδία.



Σχήμα 7.2: Κάθε αλγόριθμος στις τρεις αρχιτεκτονικές, για k ίσο με 8.

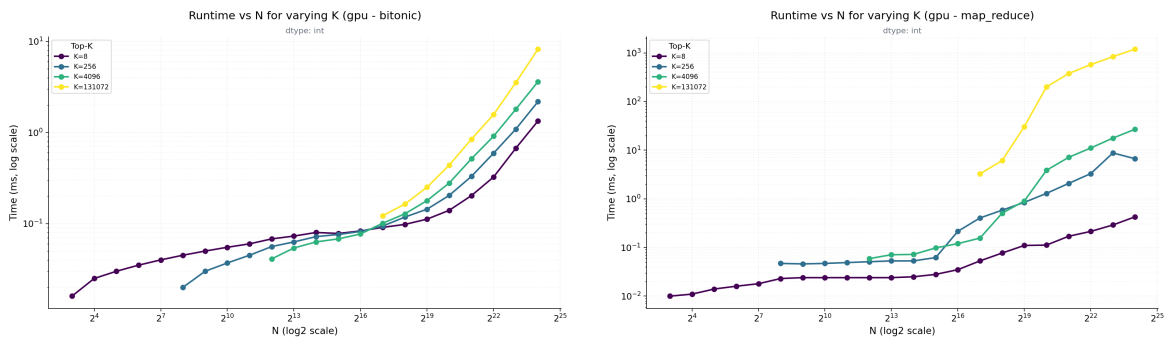
Η σύγκριση της ίδιας διτονικής ταξινόμησης στις τρεις αρχιτεκτονικές είναι διαφωτιστική, αφού εκτελεί το ίδιο χρονοδιάγραμμα επιπέδων και η διαφορά αποδίδεται αποκλειστικά στην υλοποίηση. Οι δύο αρχιτεκτονικές γενικού σκοπού εναλλάσσουν θέσεις κατά μήκος του εύρους: ο επεξεργαστής ταχύτερος στο μεγαλύτερο μέρος των μικρών έως μεσαίων μεγεθών, η μονάδα γραφικών μόνο στα μεγαλύτερα, φτάνοντας παράγοντα

έξι στο μεγαλύτερο μέγεθος, συνεπές με τη διαφορά εύρους ζώνης. Όπως και παραπάνω, το ακριβές σημείο εναλλαγής μετακινείται μεταξύ εκτελέσεων και δεν πρέπει να διαβάζεται ως σταθερό όριο. Η μονάδα νευρωνικής επεξεργασίας μένει κοντά στον επεξεργαστή σε αυτά τα μεγαλύτερα μεγέθη, συνεπές με το ότι ο χρόνος της δαπανάται σχεδόν εξ ολοκλήρου στον host.

Το map-reduce δεν επιδέχεται την ίδια σύγκριση εξαρχής, αφού κάθε αρχιτεκτονική διατηρεί και συγχωνεύει τους τοπικούς σωρούς της με ουσιωδώς διαφορετικό τρόπο (κεφάλαια 4 έως 6). Το δεξιό σχήμα δείχνει αντ' αυτού πόσο διαφορετικά εκδιπλώνεται η ίδια αλγοριθμική ιδέα σε τρεις υλοποιήσεις. Η μονάδα γραφικών είναι η ταχύτερη σχεδόν σε όλο το εύρος, με τον επεξεργαστή να ακολουθεί σε περίπου σταθερή απόσταση, εκτός από τα μικρά μεγέθη όπου ισχύει η ίδια επιφύλαξη περί θορύβου μέτρησης. Στη μεγαλύτερη είσοδο η μονάδα γραφικών μένει ταχύτερη, χωρίς όμως την ενδιάμεση εναλλαγή θέσεων της διτονικής ταξινόμησης. Η μονάδα νευρωνικής επεξεργασίας εδώ δεν πλησιάζει τον επεξεργαστή (έντονη αντίθεση με τη συμπεριφορά της στη διτονική ταξινόμηση) και παραμένει περίπου μία τάξη μεγέθους βραδύτερη από τις δύο σε κάθε μέγεθος. Ο ρόλος της διάταξης στο map-reduce είναι μόνο ένα φίλτρο ανά τμήμα, όχι κοινό χρονοδιάγραμμα δικτύου (ενότητα 6.6). Η ομοιότητα δομής που κρατά τη διτονική ταξινόμηση κοντά στον επεξεργαστή δεν έχει εδώ αντίστοιχο.

7.2.2 Εξάρτηση από την παράμετρο k

Η εξάρτηση από το k είναι το σημείο όπου η κατάταξη των αλγορίθμων ανατρέπεται. Συνιστά ίσως το σημαντικότερο εύρημα της εργασίας.



(α') Διτονική ταξινόμηση, GPU.

(β') Map-reduce, GPU.

Σχήμα 7.3: Χρόνος συναρτήσει του μεγέθους εισόδου, με χωριστή καμπύλη ανά τιμή του k. Οι καμπύλες της διτονικής ταξινόμησης παραμένουν κοντά μεταξύ τους, ενώ εκείνες του map-reduce αποκλίνουν κατά τάξεις μεγέθους.

Για τη διτονική ταξινόμηση οι τέσσερις καμπύλες k σχεδόν ταυτίζονται, αφού το k επηρεάζει μόνο το μέγεθος των ταξινομημένων μπλοκ. Για το map-reduce στη μονάδα γραφικών οι ίδιες καμπύλες απέχουν τρεις τάξεις μεγέθους.

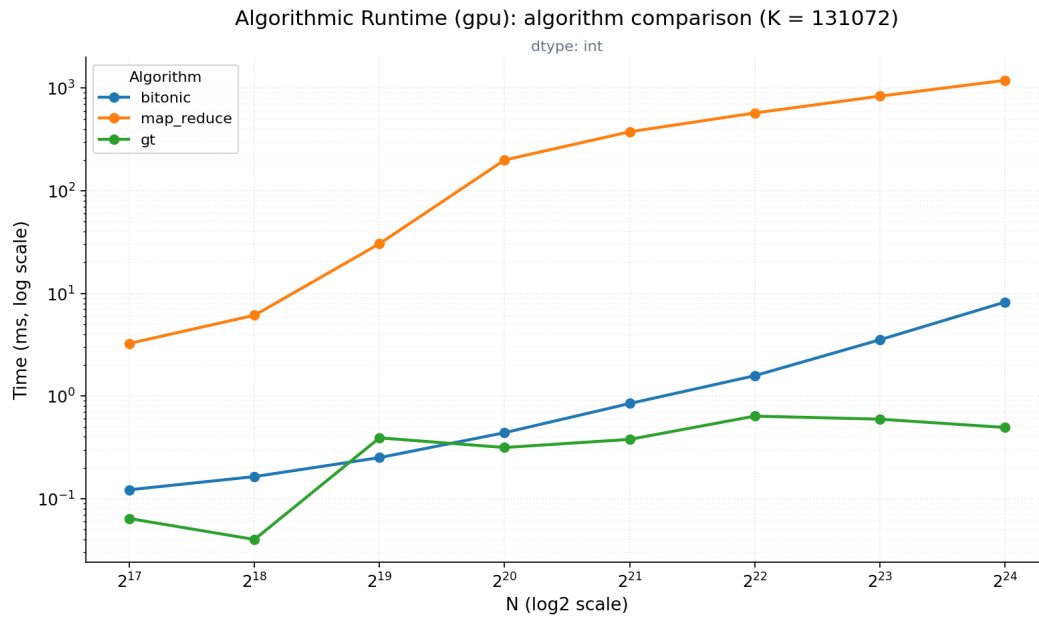
Υλοποίηση	k=8		k=131072	
	ms	θέση	ms	θέση
CPU διτονική	8,01	4	43,33	2
CPU map-reduce	1,23	2	60,37	5
GPU διτονική	1,34	3	8,22	1
GPU map-reduce	0,43	1	1191,37	6
NPU διτονική	8,17	5	56,70	3
NPU map-reduce	9,97	6	59,69	4

Πίνακας 7.1: Αλγοριθμικός χρόνος για τα δύο άκρα του k , σε είσοδο 2^{24} στοιχείων. Η κατάταξη αφορά τις έξι υλοποιήσεις της εργασίας και δεν περιλαμβάνει τις υλοποιήσεις αναφοράς.

Η ταχύτερη υλοποίηση για μικρό k γίνεται η βραδύτερη για μεγάλο, με διαφορά τριών τάξεων μεγέθους. Το map-reduce στη μονάδα γραφικών επιβραδύνεται κατά παράγοντα 2800 ανάμεσα στα δύο άκρα. Η διτονική ταξινόμηση στην ίδια αρχιτεκτονική μόλις κατά έξι. Στο μικρό k οι δύο υλοποιήσεις του επεξεργαστή κατατάσσονται υψηλότερα απ' ό,τι στο μεγάλο: το map-reduce προσπερνά τη διτονική της μονάδας γραφικών, και η διτονική προσπερνά εκείνη της μονάδας νευρωνικής επεξεργασίας. Η ανατροπή όμως μένει σταθερή: η φθηνότερη υλοποίηση για $k=8$ δεν είναι ποτέ η φθηνότερη για $k=131072$.

Ανεξάρτητες μελέτες σε μονάδες γραφικών δύο διαφορετικών κατασκευαστών καταγράφουν την ίδια εξάρτηση από το k , και την αντιμετωπίζουν κι εκείνες με προσαρμωστική εναλλαγή αλγορίθμου [3, 15]. Είναι μάλλον απίθανο λοιπόν να πρόκειται για ιδιαιτερότητα της συγκεκριμένης υλοποίησης. Εκεί η αύξηση χρόνου των μεθόδων μερικής ταξινόμησης με το k αποδίδεται ρητά στην πολυπλοκότητα $O(\log^2 k)$ του υποκείμενου διτονικού δικτύου.

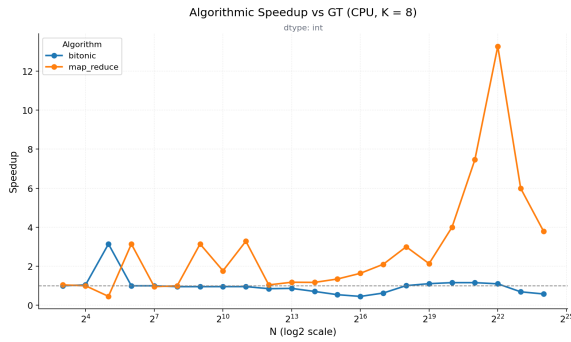
Η αιτία δόθηκε στην ενότητα 5.3.1. Ο σωρός φιλτράρει μόνο αφού γεμίσει και το πλήθος νημάτων που συνεισφέρουν περιορίζεται στον λόγο n/k . Για τα μεγέθη του πίνακα 7.1 αυτός ο λόγος είναι 128. Η συσκευή απασχολεί δηλαδή ουσιαστικά εκατόν είκοσι οκτώ νήματα από τις χιλιάδες που διαθέτει. Ο επεξεργαστής, με μόλις οκτώ νήματα, δεν προσκρούει ποτέ στο ίδιο όριο, οπότε η επιβράδυνσή του είναι σαφώς ηπιότερη.



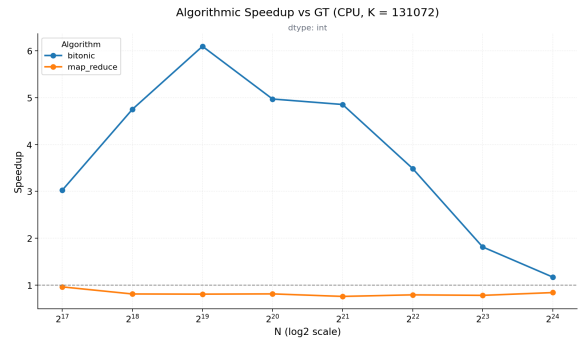
Σχήμα 7.4: Οι τρεις αλγόριθμοι στη μονάδα γραφικών για k ίσο με 131072.

Καθολικά προτιμότερος αλγόριθμος δεν υπάρχει. Η επιλογή ανάμεσα σε δίκτυο ταξινόμησης και επιλογή με σωρό εξαρτάται από τον λόγο n/k , όχι από την αρχιτεκτονική.

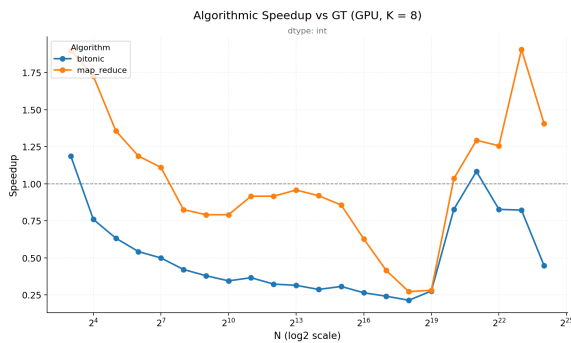
7.3 Σύγκριση αρχιτεκτονικών



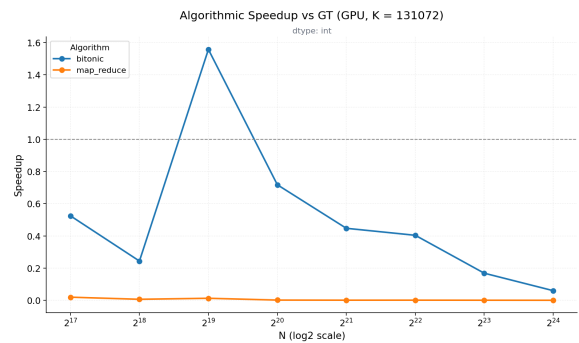
(α) CPU, k=8.



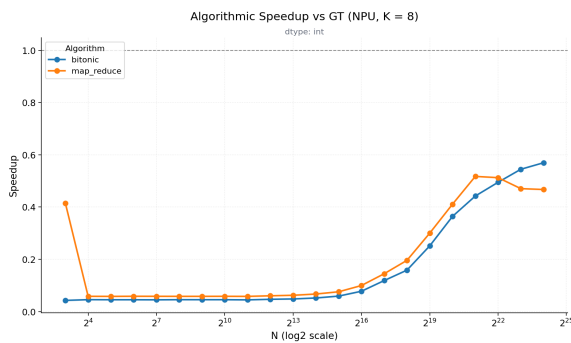
(β) CPU, k=131072.



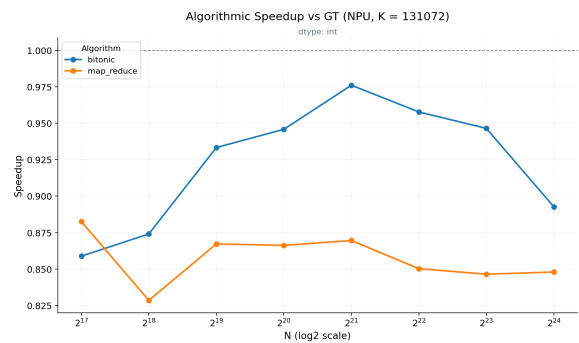
(γ) GPU, k=8.



(δ) GPU, k=131072.



(ε) NPU, k=8.



(ζ) NPU, k=131072.

Σχήμα 7.5: Επιτάχυνση έναντι της υλοποίησης αναφοράς της ίδιας αρχιτεκτονικής. Αριστερή στήλη k ίσο με 8, δεξιά στήλη k ίσο με 131072, μία γραμμή ανά αρχιτεκτονική. Τιμές πάνω από τη μονάδα σημαίνουν ότι η υλοποίηση της εργασίας είναι ταχύτερη από τη βιβλιοθήκη.

Η σύγκριση κάθε υλοποίησης με το μέτρο σύγκρισης της ίδιας αρχιτεκτονικής εξουδετερώνει τις διαφορές υλικού, άρα δίνει την πιο ειλικρινή εικόνα.

Στον επεξεργαστή το map-reduce είναι ταχύτερο της βιβλιοθήκης κατά παράγοντα τρία για μικρό k, αν και η επιτάχυνση δεν είναι σταθερή. Κορυφώνεται σε δεκατρείς φορές για n ίσο με 2^{22} , όπου το σύνολο εργασίας (16 MB) χωράει ακριβώς στην L3 κρυφή μνήμη του επεξεργαστή (σχήμα 7.7), και υποχωρεί ξανά μόλις η είσοδος ξεπεράσει αυτό το όριο. Ακόμα όμως ισχύει ο περιορισμός της ενότητας 4.5: η βιβλιοθήκη είναι μονονηματική, ενώ η υλοποίηση χρησιμοποιεί οκτώ νήματα, οπότε ο παράγοντας δεν

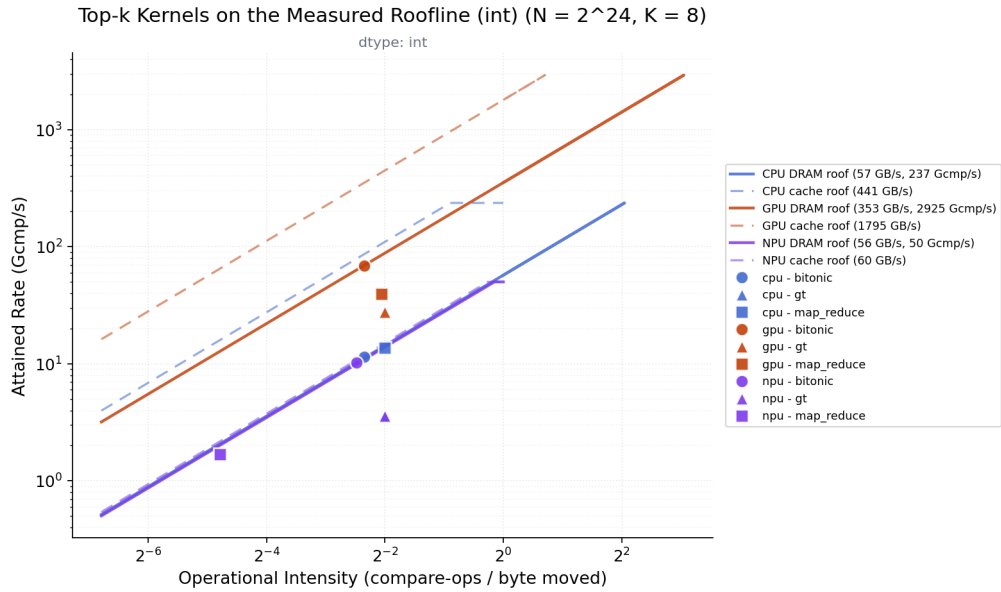
είναι αμιγώς αλγοριθμικός.

Μια δεύτερη αξιοσημείωτη επιτάχυνση εμφανίζεται αλλού στο σχήμα, συνέπεια της ανατροπής της ενότητας 7.2.2. Στο μεγάλο k , όπου η επιλογή με σωρό καταρρέει, η διτονική ταξινόμηση στον επεξεργαστή φτάνει να είναι περίπου έξι φορές ταχύτερη από τη βιβλιοθήκη, η οποία στηρίζεται κι αυτή σε σωρό μεγέθους k και υφίσταται την ίδια επιβάρυνση, ενώ το δίκτυο ταξινόμησης παραμένει εν πολλοίς ανεπηρέαστο. Το πλεονέκτημα εξασθενεί καθώς η είσοδος μεγαλώνει περαιτέρω, η κίνηση δεδομένων του δικτύου αρχίζει να κυριαρχεί, και περιορίζεται σε λίγο πάνω από τη μονάδα στη μεγαλύτερη είσοδο.

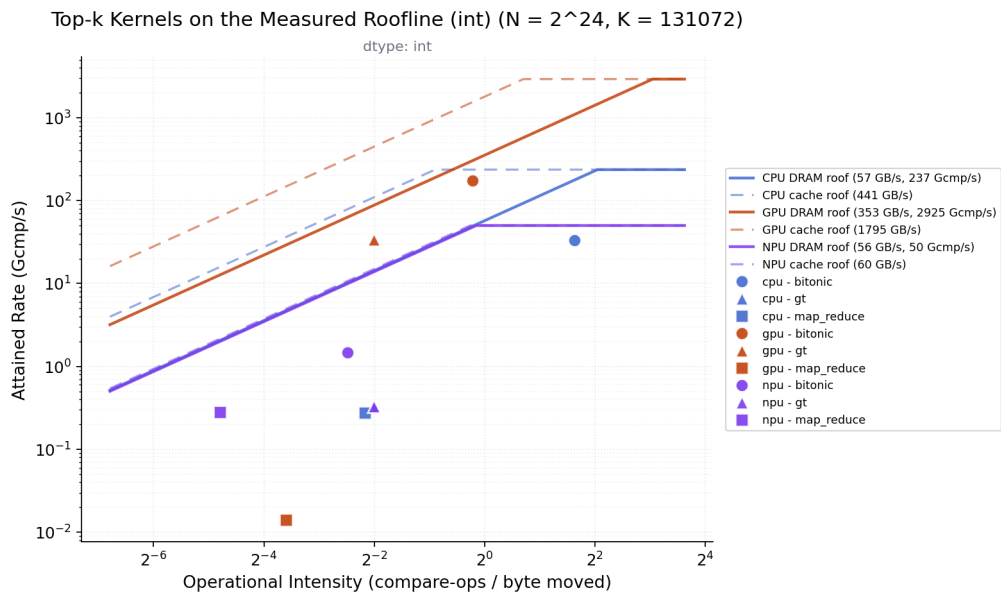
Στη μονάδα γραφικών η σύγκριση πραγματοποιείται υπό απολύτως ίσους όρους, δεδομένου ότι αξιοποιείται η ίδια συσκευή και το ίδιο επίπεδο παραλληλίας. Το map-reduce υπερτερεί οριακά για $k=8$, ενώ για μεγάλο k η βιβλιοθήκη είναι ταχύτερη κατά περισσότερο από χίλιες φορές, αφού η επιλογή με βάση την αναπαράσταση δεν έχει σημείο κατάρρευσης ως προς το k (ενότητα 5.5.1).

Στη μονάδα νευρωνικής επεξεργασίας, για τους ακέραιους τύπους στους οποίους αναφέρεται το σχήμα, καμία υλοποίηση δεν ξεπερνά τη βιβλιοθήκη πουθενά. Δεν προκαλεί έκπληξη, αφού ο ίδιος επεξεργαστής που θα εκτελούσε τη βιβλιοθήκη καταναλώνεται ούτως ή άλλως στην προετοιμασία δεδομένων για τη μονάδα (ενότητα 6.6). Η διευκρίνιση περί ακέραιων τύπων γίνεται στοχευμένα. Για κινητή υποδιαστολή, ιδίως μισή ακρίβεια, η εικόνα ανατρέπεται και η μονάδα νευρωνικής επεξεργασίας πράγματι ξεπερνά τη βιβλιοθήκη εκεί. Η εξήγηση παρατίθεται στην ενότητα 7.6.

7.4 Ανάλυση με το μοντέλο roofline



(α') k ίσο με 8.



(β) k ίσο με 131072.

Σχήμα 7.6: Τοποθέτηση των εκτελέσεων απέναντι στα μετρημένα ταβάνια, για τη μεγαλύτερη είσοδο. Περιορίζεται στο μέγιστο μέγεθος, όπου το σύνολο εργασίας ξεπερνά την κρυφή μνήμη και το ταβάνι της κύριας μνήμης είναι το σχετικό.

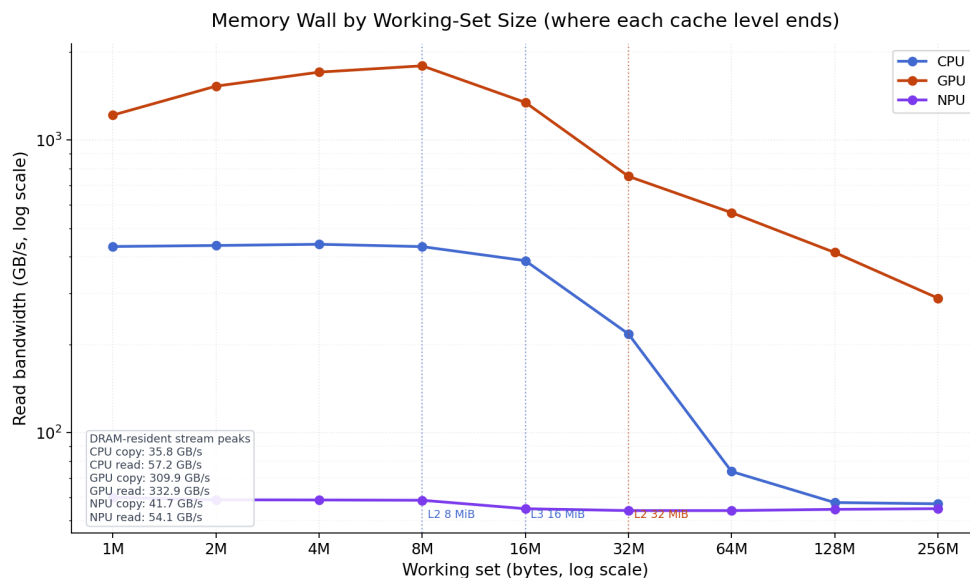
Κάθε εκτέλεση τοποθετείται εδώ σε επίπεδο υπολογιστικής έντασης και επιτευχθέντος ρυθμού. Όλες οι υλοποιήσεις συγκεντρώνονται αριστερά, σε πολύ χαμηλή υπολογιστική ένταση και καμία δεν πλησιάζει το οριζόντιο τμήμα της στέγης. Καμία υλοποίηση, σε καμία αρχιτεκτονική, δεν περιορίζεται από υπολογιστική ικανότητα. Το εύρημα συμφωνεί με ανεξάρτητη μέτρηση σε επιταχυντές κέντρου δεδομένων, όπου ο ταχύτερος γνωστός πυρήνας επιλογής αξιοποιεί πάνω από το ενενήντα τοις εκατό του εύρους ζώνης έναντι περίπου του μισού της υπολογιστικής ικανότητας, και ο χρόνος κλιμακώνεται με

το εύρος ζώνης κάθε συσκευής [3].

Η αντιπαράθεση των δύο τιμών k αναδεικνύει την έντονη διαφοροποίηση των αλγορίθμων ως προς τη χρήση του υλικού. Αφενός, η διτονική ταξινόμηση στη μονάδα γραφικών εντοπίζεται σταθερά πάνω στο κεκλιμένο τμήμα της θεωρητικής στέγης, ανεξαρτήτως του k . Η θέση αυτή επιβεβαιώνει οπτικά το εύρημα της ενότητας 5.6. Η υλοποίηση αξιοποιεί σχεδόν πλήρως το διαθέσιμο εύρος ζώνης και περιορίζεται αποκλειστικά από τον όγκο δεδομένων που οφείλει να διακινήσει.

Αφετέρου, η συμπεριφορά του map-reduce στην ίδια αρχιτεκτονική μεταβάλλεται ολότελα. Ενώ για $k=8$ βρίσκεται κοντά στα υπόλοιπα σημεία, στη μεγάλη τιμή του k καταρρέει κατά τρεις τάξεις μεγέθους, απέχοντας πλέον σημαντικά και από τα δύο ταβάνια. Η συγκεκριμένη τοποθέτηση (ούτε στο κεκλιμένο ούτε στο οριζόντιο τμήμα του διαγράμματος) αποτελεί την υπογραφή ενός περιορισμού ανεξάρτητου από τη μνήμη ή τον υπολογισμό. Αποτυπώνει την κατάρρευση παραλληλίας της ενότητας 5.3.1.

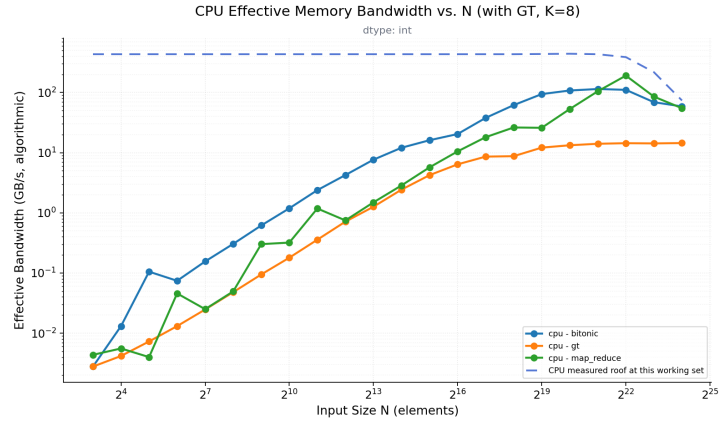
Το συγκεκριμένο εύρημα απορρέει από την ίδια τη φύση του προβλήματος. Επειδή η επιλογή top- k εκτελεί πρακτικά μόνο μία σύγκριση ανά στοιχείο χωρίς επαναχρησιμοποίηση, η υπολογιστική της ένταση είναι εξ ορισμού χαμηλή.



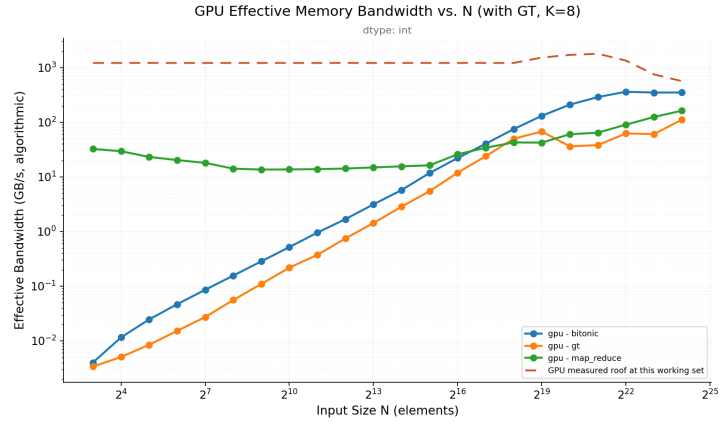
Σχήμα 7.7: Μετρημένο εύρος ζώνης ανάγνωσης συναρτήσει του μεγέθους του συνόλου εργασίας, για τις τρεις αρχιτεκτονικές.

Το μέγιστο θεωρητικό όριο (ταβάνι) με το οποίο συγκρίνεται κάθε εκτέλεση μεταβάλλεται δυναμικά, καθώς αποτελεί συνάρτηση του εκάστοτε συνόλου εργασίας (ενότητα 3.4). Το σχήμα 7.7 αποτυπώνει τη μετρούμενη κλιμάκωση. Τα διακριτά επίπεδα αντιστοιχούν στις βαθμίδες της ιεραρχίας μνήμης, ενώ η πτώση στο δεξιό άκρο υποδηλώνει το σημείο όπου το σύνολο εργασίας υπερβαίνει τη χωρητικότητα της τελευταίας βαθμίδας.

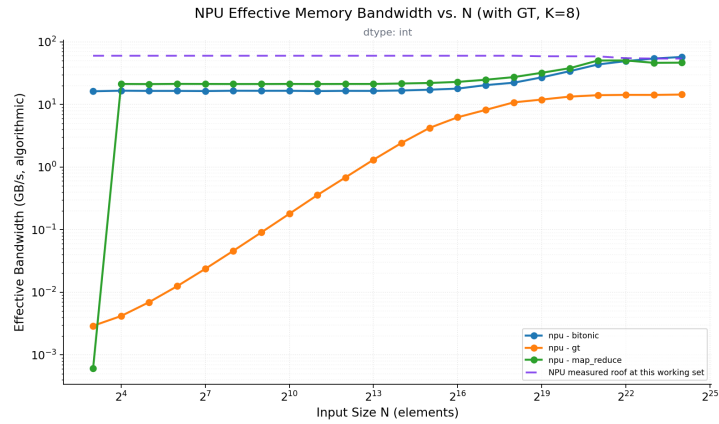
7.4.1 Αξιοποίηση εύρους ζώνης μνήμης



(α) CPU.



(β) GPU.



(γ) NPU.

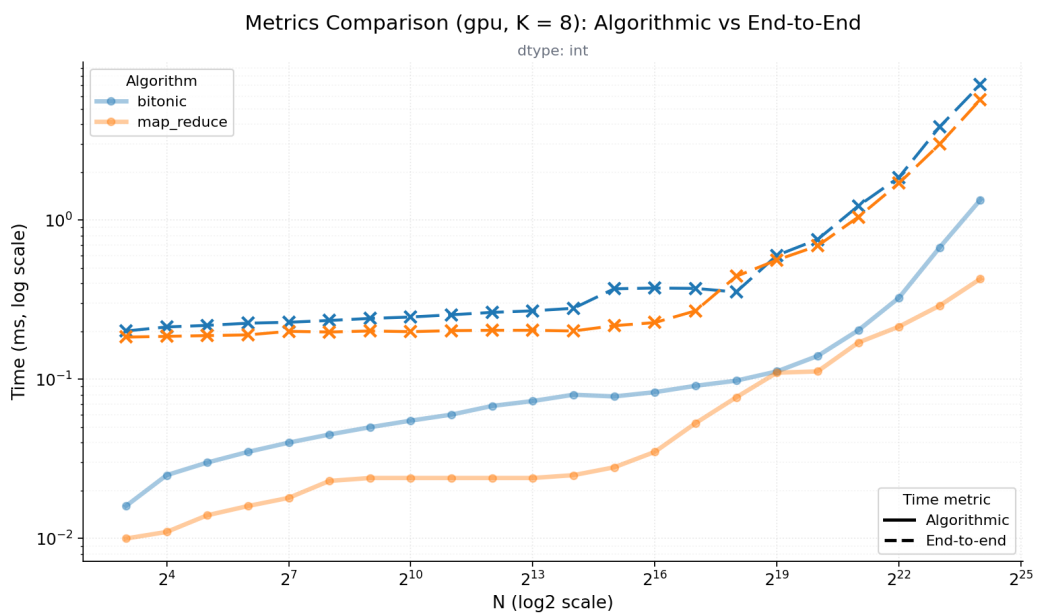
Σχήμα 7.8: Επιτευχθέν εύρος ζώνης συναρτήσει του μεγέθους εισόδου, μία αρχιτεκτονική ανά σχήμα, με το μετρημένο ταβάνι της σχεδιασμένο με διακεκομμένη γραμμή. Η απόσταση κάθε καμπύλης από τη διακεκομμένη είναι το αναξιοποίητο περιθώριο.

Η αξιοποίηση του ταβανιού μνήμης αυξάνεται με το μέγεθος εισόδου και σταθεροποιείται όταν το σύνολο εργασίας ξεπεράσει την κρυφή μνήμη. Στη μεγαλύτερη είσοδο, η διτονική ταξινόμηση στον επεξεργαστή φτάνει περίπου τα τέσσερα πέμπτα του ταβανιο-

ύ, μπροστά από τα τρία πέμπτα της μονάδας γραφικών. Η αξιοποίηση υπολογιστικού ταβανιού μένει κάτω του πέντε τοις εκατό και στις δύο.

Η αντιπαράθεση με τη βιβλιοθήκη στη μονάδα γραφικών δείχνει πως εγγύτητα στο ταβάνι δεν συνεπάγεται ταχύτητα. Η διτονική ταξινόμηση αξιοποιεί σαφώς μεγαλύτερο ποσοστό του εύρους ζώνης από τη βιβλιοθήκη και παρ' όλα αυτά είναι βραδύτερη. Το πρώτο δείχνει πόσο καλά γραμμένος είναι ο κώδικας, το δεύτερο πόσο αποδοτικός είναι ο ίδιος ο αλγόριθμος.

7.5 Ενεργειακή αποτίμηση

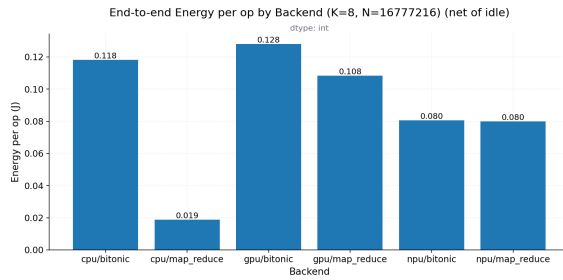


Σχήμα 7.9: Αλγοριθμικός και συνολικός χρόνος στη μονάδα γραφικών, για k ίσο με 8. Η απόσταση των δύο καμπυλών κάθε αλγορίθμου είναι το κόστος των μεταφορών και της έναρξης των πυρήνων.

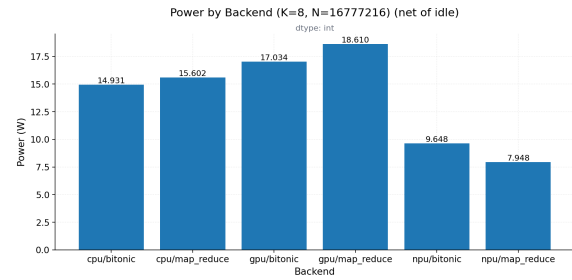
Πριν τις ενεργειακές μετρήσεις, το σχήμα 7.9 δείχνει γιατί τα δύο πεδία αποκλίνουν τόσο στη μονάδα γραφικών. Η καμπύλη του συνολικού χρόνου μένει σταθερή κοντά στα τρία δέκατα του χιλιοστού μέχρι περίπου 2^{16} στοιχεία, όπου ο πυρήνας είναι ουσιαστικά αόρατος και ο χρόνος αναλώνεται σε μεταφορές και εκκινήσεις. Ακόμη και στη μεγαλύτερη είσοδο ο συνολικός χρόνος μένει πολλαπλάσιος του αλγοριθμικού, περίπου πέντε φορές για τη διτονική ταξινόμηση και δεκατρείς για το map-reduce.

Η ενεργειακή εικόνα διαφέρει ουσιαστικά από αυτή των χρόνων, λόγω του συμμετρικού πεδίου μέτρησης (ενότητα 3.3.1). Κάθε αρχιτεκτονική χρεώνεται το συνολικό κόστος επίλυσης.

Στη μεγαλύτερη είσοδο και μικρό k , το map-reduce στον επεξεργαστή καταναλώνει περίπου 19 mJ ανά εκτέλεση, η οικονομικότερη υλοποίηση. Οι αντίστοιχες στη μονάδα γραφικών καταναλώνουν από 108 mJ έως 128 mJ, έως επτά φορές περισσότερο, παρότι ταχύτερες στο αλγοριθμικό πεδίο. Η ακριβότερη υλοποίηση όλων είναι η διτονική ταξινόμηση στη μονάδα γραφικών. Η αντίστοιχη στον επεξεργαστή κοστίζει 118 mJ.



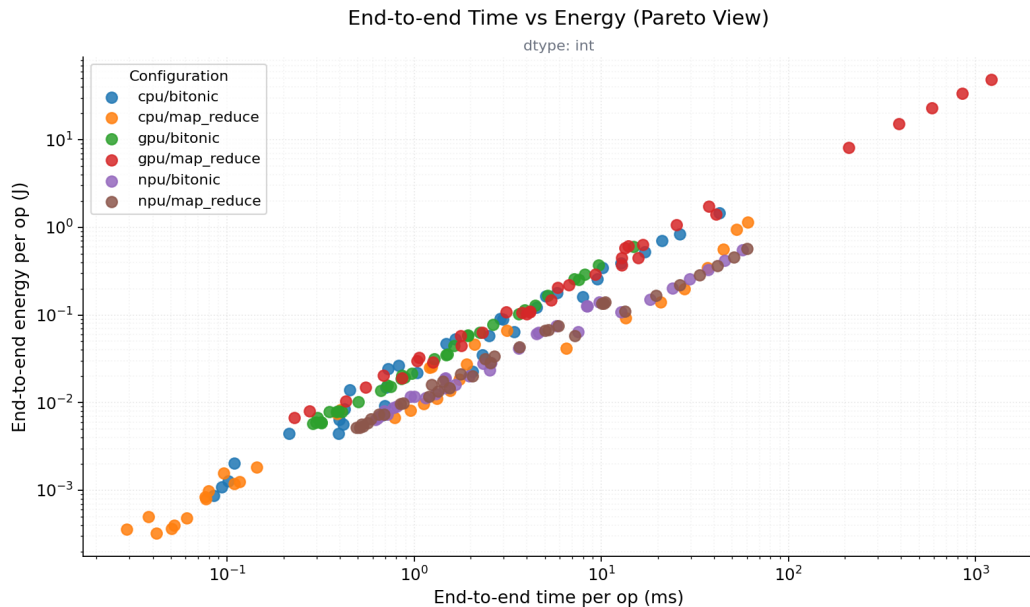
(α') Καθαρή ενέργεια.



(β') Καθαρή ισχύς.

Σχήμα 7.10: Καθαρή ενέργεια και ισχύς ανά αρχιτεκτονική και αλγόριθμο, για k ίσο με 8.

Συνδυαστικά τα σχήματα 7.10α' και 7.10β' αναδεικνύουν μια διάκριση που εύκολα παρερμηνεύεται, ιδίως για τη μονάδα νευρωνικής επεξεργασίας. Έχει τη χαμηλότερη κατανάλωση ισχύος από όλες τις υλοποιήσεις, με το map-reduce της στα 7.9 W έναντι των 15.6 W του αντίστοιχου στον επεξεργαστή και 18.6 W της μονάδας γραφικών. Καταναλώνει όμως πάνω από τετραπλάσια ενέργεια από το map-reduce του επεξεργαστή, με 0.080 J έναντι 0.019 J. Η ενέργεια είναι γινόμενο ισχύος επί χρόνο και η μονάδα χρειάζεται περίπου οκταπλάσιο χρόνο για την ίδια εργασία. Το πλεονέκτημα σαράντα εννέα τοις εκατό στην ισχύ εξανεμίζεται έτσι από το μειονέκτημα της οκταπλάσιας διάρκειας. Το πλεονέκτημα στην ισχύ δεν μεταφράζεται λοιπόν σε πλεονέκτημα στην ενέργεια. Η αρχή ότι η μετακίνηση δεδομένων κοστίζει ενεργειακά περισσότερο από τον υπολογισμό ίδιο [10] ισχύει και εδώ, αλλά λείπει η επαναχρησιμοποίηση που θα την απόσβενε. Μια αρχιτεκτονική χαμηλής κατανάλωσης αποδίδει ενεργειακά μόνο αν τελειώνει σε συγκρίσιμο χρόνο, και αυτό δεν συμβαίνει εδώ.



Σχήμα 7.11: Συνολικός χρόνος και συνολική ενέργεια κάθε μετρημένης περίπτωσης. Και τα δύο μεγέθη ελαχιστοποιούνται, οπότε οι προτιμότερες περιπτώσεις βρίσκονται προς την κάτω αριστερή γωνία.

Το σχήμα 7.11 τοποθετεί κάθε περίπτωση σε επίπεδο δύο κριτηρίων ελαχιστοποιούμενων ταυτόχρονα. Μια περίπτωση κυριαρχείται από άλλη όταν η δεύτερη δεν είναι χειρότερη σε κανένα από τα δύο και καλύτερη σε τουλάχιστον ένα, οπότε δεν υπάρχει λόγος να επιλεγεί ποτέ. Οι περιπτώσεις που δεν κυριαρχούνται από καμία άλλη σχηματίζουν το μέτωπο Pareto. Εκεί η βελτίωση του ενός κριτηρίου απαιτεί υποχώρηση στο άλλο.

Το αποτέλεσμα είναι διαφωτιστικό ακριβώς επειδή είναι αρνητικό. Οι μετρήσεις σχηματίζουν στενή διαγώνια ζώνη, με συντελεστή συσχέτισης των λογαρίθμων χρόνου και ενέργειας 0,97. Από τις 209 περιπτώσεις μόνο δύο ανήκουν στο μέτωπο, αμφότερες η ίδια υλοποίηση, το map-reduce του επεξεργαστή σε δύο γειτονικά μεγέθη. Δεν υπάρχει ουσιαστικός συμβιβασμός ανάμεσα σε ταχύτητα και ενέργεια. Η ταχύτερη διαμόρφωση είναι ταυτόχρονα και η οικονομικότερη.

Ο λόγος προκύπτει από τα δύο προηγούμενα σχήματα. Η ισχύς μεταβάλλεται μεταξύ υλοποιήσεων μόλις κατά παράγοντα 2,3, ενώ ο χρόνος κατά τέσσερις τάξεις μεγέθους. Όταν ο ένας παράγοντας του γινομένου είναι σχεδόν σταθερός, το γινόμενο ακολουθεί τον άλλον. Η ανταλλαγή μεγαλύτερης διάρκειας με χαμηλότερη ισχύ, που είναι πραγματικός σχεδιαστικός μοχλός σε άλλα συστήματα, εδώ απλώς δεν υπάρχει.

Όλες οι περιπτώσεις στο μέτωπο εκτελούνται στον επεξεργαστή. Καμία διαμόρφωση της μονάδας γραφικών ή της μονάδας νευρωνικής επεξεργασίας δεν είναι βέλτιστη κατά Pareto πουθενά στον παραμετρικό χώρο.

7.6 Επίδραση του τύπου δεδομένων

Υλοποίηση	half	int	uint	float	double
CPU διτονική	4,60	8,01	7,93	8,01	14,03
CPU map-reduce	0,39	1,23	1,22	1,22	2,35
CPU αναφορά	13,27	4,65	4,63	5,50	5,54
GPU διτονική	0,88	1,34	1,37	1,34	2,89
NPU διτονική	7,71	8,17	8,37	8,23	25,38

Πίνακας 7.2: Αλγοριθμικός χρόνος ανά τύπο δεδομένων, σε ms, για 2^{24} στοιχεία και k ίσο με 8.

Οι υλοποιήσεις της εργασίας ακολουθούν το πλάτος του τύπου, όπως αναμένεται από αλγορίθμους περιορισμένους από τη μνήμη. Η μισή ακρίβεια είναι σταθερά η ταχύτερη· η διτονική ταξινόμηση στον επεξεργαστή ολοκληρώνει σε λίγο πάνω από τον μισό χρόνο της απλής, ακριβώς επειδή διακινεί τα μισά bytes με τον ίδιο ρυθμό συγκρίσεων.

Η υλοποίηση αναφοράς συμπεριφέρεται αντίθετα, με τη μισή ακρίβεια να καταγράφει τις χαμηλότερες επιδόσεις. Η διαφορά αποκαλύπτει ότι η βιβλιοθήκη περιορίζεται πρωτίστως από το κόστος της βαθμωτής σύγκρισης (που για τη μισή ακρίβεια περιλαμβάνει μετατροπή), παρά από τη μνήμη. Η κατάταξη μεταξύ τύπων είναι λοιπόν χαρακτηριστικό της υλοποίησης, ανεξάρτητα από τους εγγενείς περιορισμούς του υλικού.

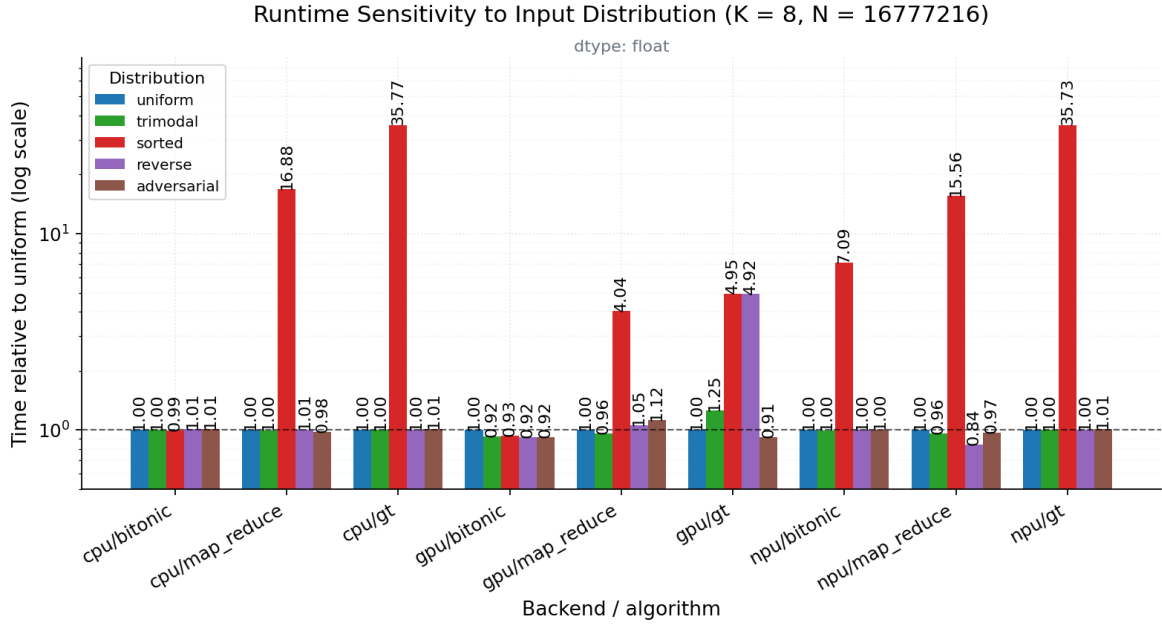
Η διπλή ακρίβεια στη μονάδα νευρωνικής επεξεργασίας ξεχωρίζει, τριπλάσιος χρόνος από κάθε άλλον τύπο. Η επιβάρυνση οφείλεται εξ ολοκλήρου στο πρόσθετο πέρασμα

διόρθωσης της ενότητας 6.4.3, απαραίτητο επειδή το κλειδί των 32 bit δεν αρκεί για τιμές των 64.

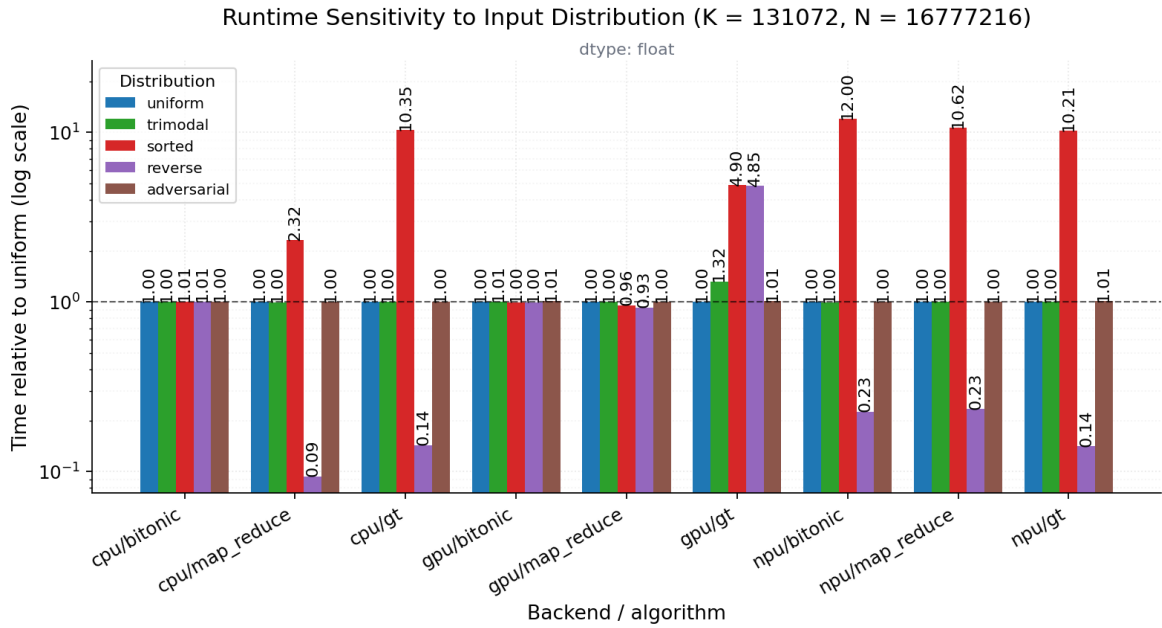
Ο συνδυασμός των δύο αντίθετων συμπεριφορών παράγει το μοναδικό σημείο όλης της εργασίας όπου η μονάδα νευρωνικής επεξεργασίας υπερτερεί της βιβλιοθήκης της. Για μισή ακρίβεια, η διτονική ταξινόμηση στη μονάδα είναι ταχύτερη από τη βιβλιοθήκη σε είκοσι εννέα από τις μετρούμενες περιπτώσεις, με τη μεγαλύτερη διαφορά να φτάνει παράγοντα δυόμισι. Το φαινόμενο εμφανίζεται ασθενέστερα και στους άλλους δύο τύπους κινητής υποδιαστολής, μα καθόλου στους ακέραιους.

Το καταγραφόμενο πλεονέκτημα πηγάζει περισσότερο από την ανεπάρκεια της βιβλιοθήκης στη διαχείριση της μισής ακρίβειας, παρά από κάποια εγγενή υπεροχή της μονάδας. Ο πυρήνας της διάταξης δουλεύει σε ακραίους 32 bit ανεξαρτήτως τύπου, μετακυλίνοντας το κόστος μετατροπής στο διανυσματικό στάδιο κωδικοποίησης του host (ενότητα 6.4.2). Η βιβλιοθήκη αντίθετα εκτελεί βαθμωτές συγκρίσεις με ενδιαμέση μετατροπή. Η εκφόρτωση συμφέρει εδώ επειδή η καθιερωμένη λύση αδυνατεί με αυτόν τον τύπο δεδομένων, ωστόσο αυτό δεν σημαίνει πως η μονάδα είναι γενικά ισχυρότερη.

7.7 Ευαισθησία στην κατανομή της εισόδου



(α) k=8.



(β) k=131072.

Σχήμα 7.12: Ευαισθησία του χρόνου στην κατανομή της εισόδου, για απλή ακρίβεια και 2^{24} στοιχεία. Κάθε στήλη δίνει τον χρόνο μιας κατανομής ως προς τον χρόνο της ομοιόμορφης για την ίδια υλοποίηση. Η μονάδα σημαίνει πλήρη ανεξαρτησία από τα δεδομένα.

Μέσω της κανονικοποίησης ως προς την ομοιόμορφη κατανομή, οι εννέα συνδυασμοί μπορούν να αντιπαρατεθούν στο ίδιο γράφημα, μολονότι οι απόλυτοι χρόνοι εκτέλεσής τους διαφέρουν κατά τάξεις μεγέθους. Στο παραπάνω σχήμα, μια υλοποίηση ανεπηρέαστη από τα δεδομένα εμφανίζεται ως πέντε ίσες στήλες μοναδιαίου ύψους. Μια

ευαίσθητη αποκλίνει προς και τις δύο κατευθύνσεις.

Η διτονική ταξινόμηση, τόσο στον επεξεργαστή όσο και στη μονάδα γραφικών, παραμένει πρακτικά ανεπηρέαστη από την κατανομή των δεδομένων και για τις δύο τιμές του k , παρουσιάζοντας μέγιστη διακύμανση της τάξης του 9%. Η σταθερότητα αυτή την καθιστά ιδανικό δείκτη ελέγχου. Οποιαδήποτε ουσιαστική μεταβολή στον χρόνο εκτέλεσής της θα οφειλόταν αποκλειστικά σε εξωτερικούς παράγοντες της μέτρησης και όχι στην εγγενή συμπεριφορά του αλγορίθμου. Η διτονική ταξινόμηση στη μονάδα νευρωνικής επεξεργασίας ξεχωρίζει εδώ επιπλέον. Σε αντίθεση με τις άλλες δύο αρχιτεκτονικές είναι ευαίσθητη στην κατανομή, με διακύμανση παράγοντα πενήντα τρία. Η ευαισθησία δεν προέρχεται από τη διάταξη, που εκτελεί σταθερό δίκτυο, αλλά από τη συγχώνευση με σωρό που εκτελεί ο *host* και κυριαρχεί στον χρόνο (ενότητα 6.4.1).

Στον αντίποδα, οι υλοποιήσεις με σωρούς εμφανίζουν σπουδαία ευαισθησία στην κατανομή, ανεξάρτητα από ποιος έγραψε τον κώδικα. Η βιβλιοθήκη αναφοράς μεταβάλλεται κατά παράγοντα εβδομήντα δύο μεταξύ αύξουσας και φθίνουσας εισόδου. Το δικό μας *map-reduce* στον επεξεργαστή κατά παράγοντα είκοσι πέντε, ίδιος μηχανισμός με την ενότητα 4.3. Η ευαισθησία στα δεδομένα είναι ιδιότητα του αλγορίθμου και είναι αναπόφευκτη για οποιαδήποτε μέθοδο επιλογής βασίζεται σε σωρό.

Η βιβλιοθήκη αναφοράς της μονάδας γραφικών παρουσιάζει ένα τρίτο, διακριτό μοτίβο. Επιβραδύνεται σχεδόν κατά τον ίδιο παράγοντα και προς τις δύο κατευθύνσεις, 4,95 για αύξουσα και 4,92 για φθίνουσα, χωρίς να ευνοεί τη μία όπως οι σωροί. Η ταξινόμηση δεν αλλάζει την κατανομή των τιμών (ενότητα 3.7.2), άρα δεν πρόκειται για την *radix-adversarial* κατανομή ψηφίων (ενότητα 2.2.4). Αντ' αυτού είναι ζήτημα θέσης. Οι υποψήφιοι που επιζούν του κρίσιμου κάδου καταλήγουν συγκεντρωμένοι στο ένα ή στο άλλο άκρο του πίνακα, κάτι που βαραίνει συμμετρικά το παράλληλο στάδιο συμπύκνωσης της βιβλιοθήκης ανεξάρτητα από ποιο άκρο.

Η αντιπαράθεση των δύο τιμών k απομονώνει την αιτία της ευαισθησίας του *map-reduce* στη μονάδα γραφικών. Ενώ η υλοποίηση είναι εξαιρετικά ευαίσθητη για $k=8$ (η ταξινομημένη είσοδος το επιβραδύνει κατά παράγοντα 4,0), παραμένει σχεδόν ανεπηρέαστη για $k=131072$ (η ίδια είσοδος το μεταβάλλει κατά μόλις 4%). Ο ίδιος κώδικας, με τα ίδια δεδομένα, αλλάζει συμπεριφορά επειδή αλλάζει εκεί ο περιοριστικός παράγοντας. Στα μεγάλα k , η επίδοση καθορίζεται πλέον από την έλλειψη παραλληλίας (ενότητα 5.3.1) και δεν ορίζεται από τον ρυθμό απόρριψης. Φυσικά, η κατανομή δεδομένων παύει να έχει σημασία.

Κεφάλαιο 8

Συμπεράσματα και μελλοντική εργασία

8.1 Συμπεράσματα

Η εργασία υλοποίησε δύο αλγορίθμους επιλογής top-k σε τρεις αρχιτεκτονικές, χρησιμοποιώντας κοινή υποδομή για την αξιολόγησή τους. Η παρούσα ενότητα συνοψίζει τα συμπεράσματα του κεφαλαίου 7, εστιάζοντας στα ευρήματα που διατηρούν την ισχύ τους ανεξαρτήτως του υποκείμενου υλικού.

Κίνηση δεδομένων. Η ανάλυση της ενότητας 7.4 έδειξε πως για μικρό k όλοι οι συνδυασμοί αλγορίθμων και αρχιτεκτονικών περιορίζονται από το εύρος ζώνης και όχι από την υπολογιστική ικανότητα των συσκευών. Η επίδοση εξαρτάται αποκλειστικά από τη μείωση του όγκου των δεδομένων που μεταφέρονται. Αυτό επιβεβαιώνεται ξεκάθαρα από τη διτονική ταξινόμηση στη μονάδα γραφικών. Αν και αξιοποιεί καλύτερα την υπολογιστική της οροφή σε σχέση με τη βιβλιοθήκη αναφοράς, τελικά υστερεί επειδή αναγκάζεται να διακινήσει πολλαπλάσια δεδομένα.

Ο λόγος n/k . Η ταχύτερη υλοποίηση για $k=8$ μετατρέπεται στη βραδύτερη για $k=131072$ ανεξαρτήτως πλατφόρμας, όπως φάνηκε στην ενότητα 7.2.2. Αντίθετα, η απόδοση του δικτύου ταξινόμησης παραμένει πολύ πιο σταθερή. Ο αλγόριθμος του σωρού αρχίζει να φιλτράρει στοιχεία μόνο αφού γεμίσει, περιορίζοντας τη διαθέσιμη παραλληλία βάσει του λόγου n/k . Παρόμοια συμπεριφορά έχει παρατηρηθεί και στη βιβλιογραφία [15], αναδεικνύοντας την ανάγκη για προσαρμοστική εναλλαγή αλγορίθμων ανάλογα με τα δεδομένα εισόδου.

Το όριο μέτρησης. Ενώ η μονάδα γραφικών υπερτερεί στον καθαρό χρόνο εκτέλεσης του πυρήνα για τα ακραία μεγέθη του πειράματος, η συνολική end-to-end μέτρηση αναδεικνύει τον επεξεργαστή ως ταχύτερο σε όλες τις περιπτώσεις. Η μετατόπιση του ορίου μέτρησης αλλάζει πλήρως την τελική κατάταξη. Συνεπώς, η αξιολόγηση μιας υλοποίησης οφείλει να συνυπολογίζει το κόστος μεταφοράς δεδομένων, διαφορετικά η σύγκριση ως προς το αν η εκφόρτωση στην κάρτα συμφέρει παραμένει ελλιπής.

Ισχύς και ενέργεια. Παρά το γεγονός ότι η μονάδα νευρωνικής επεξεργασίας καταγράφει τη χαμηλότερη ισχύ λειτουργίας, τελικά απαιτεί τη μέγιστη συνολική ενέργεια. Όπως συζητήθηκε στην ενότητα 7.5, η μικρή μείωση στην ισχύ δεν καταφέρνει να αντισταθμίσει τη ραγδαία αύξηση του χρόνου εκτέλεσης.

Για την επιλογή top-k με μικρές τιμές του k, η εκφόρτωση του υπολογισμού σε επιταχυντή αποδεικνύεται ενεργειακά και χρονικά ασύμφορη συγκριτικά με την εκτέλεση map-reduce στον επεξεργαστή. Αρχιτεκτονικές όπως οι NPU (ενότητα 2.3.3) προϋποθέτουν υψηλή τοπική επαναχρησιμοποίηση δεδομένων για να αποσβέσουν το κόστος μεταφοράς τους. Η επιλογή top-k δεν προσφέρει αυτή τη δυνατότητα, με αποτέλεσμα το μεγαλύτερο μέρος του χρόνου να αναλώνεται στην προετοιμασία των δεδομένων, αφήνοντας τη συσκευή σε αδράνεια (ενότητα 6.4.1).

Μοναδική εξαίρεση αποτελεί η διτονική ταξινόμηση στη μονάδα νευρωνικής επεξεργασίας για αριθμούς μισής ακρίβειας, όπου και ξεπερνά την αντίστοιχη υλοποίηση αναφοράς. Ωστόσο, η ανάλυση της ενότητας 7.6 αποδίδει αυτό το πλεονέκτημα στην ελλιπή βελτιστοποίηση της βιβλιοθήκης του επεξεργαστή για τον συγκεκριμένο τύπο δεδομένων.

8.2 Περιορισμοί

Η σύγκριση με την `std::partial_sort` στον επεξεργαστή ενέχει μια εγγενή ασυμμετρία. Η βιβλιοθήκη λειτουργεί αυστηρά μονονηματικά, σε αντίθεση με τις υλοποιήσεις της εργασίας που εκμεταλλεύονται την παραλληλία (ενότητα 4.5). Έτσι, η καταγεγραμμένη επιτάχυνση προκύπτει συνδυαστικά τόσο από τη βελτίωση του αλγορίθμου όσο και από τη χρήση πολλαπλών νημάτων. Η επιλογή αυτή πηγάζει από την πρόθεση να συγκριθεί η νέα υλοποίηση με τα άμεσα διαθέσιμα εργαλεία ενός προγραμματιστή, ωστόσο η διαφορά αυτή πρέπει να λαμβάνεται υπόψη στην ανάγνωση των αποτελεσμάτων.

Επιπλέον, ο όγκος δεδομένων που καταμετρήθηκε για τις βιβλιοθήκες αναφοράς συνιστά απλώς το κάτω φράγμα. Επειδή τα εσωτερικά τους περάσματα στη μνήμη δεν είναι άμεσα παρατηρήσιμα, τα αναφερόμενα ποσοστά αξιοποίησης της οροφής στην ενότητα 7.4.1 αποτελούν υποεκτιμήσεις.

Για τη μονάδα νευρωνικής επεξεργασίας δεν διατίθεται εγγενής βιβλιοθήκη top-k για άμεση σύγκριση. Η αντιπαραβολή πραγματοποιήθηκε έναντι του host, απαντώντας στο πρακτικό ερώτημα του κατά πόσο αξίζει η εκφόρτωση, χωρίς να αξιολογεί τη μέγιστη θεωρητική απόδοση της ίδιας της συσκευής. Αντίστοιχα, η ανάλυση φάσεων της ενότητας 6.4.1 προέρχεται από ξεχωριστές εκτελέσεις με ενεργοποιημένο το debug αντί από την τυπική σάρωση, οπότε τα ποσοστά της στηρίζονται σε μικρότερο δείγμα επαναλήψεων απ' ό,τι οι υπόλοιπες μετρήσεις του παραρτήματος Γ'.

Τέλος, ο πειραματικός χώρος περιορίστηκε σε ένα συγκεκριμένο σύστημα και συγκεκριμένα εύρη τιμών. Αν και οι ποιοτικές τάσεις γύρω από τον λόγο n/k και το ρόλο του ορίου μέτρησης αναμένεται να γενικεύονται σε παρόμοια συστήματα, οι ακριβείς αριθμητικοί παράγοντες παραμένουν στενά συνδεδεμένοι με τη συγκεκριμένη δοκιμαστική πλατφόρμα.

8.3 Μελλοντική εργασία

Η εκτέλεση πολλαπλών ταυτόχρονων ερωτημάτων top- k είναι η κύρια μελλοντική επέκταση, ειδικά για τη μονάδα νευρωνικής επεξεργασίας. Εφόσον το κύριο εμπόδιο εντοπίζεται στην προετοιμασία και αποστολή των δεδομένων, η ομαδοποίηση των κλήσεων θα βοηθούσε καθοριστικά στην απόσβεση αυτού του κόστους. Τέτοια επαναλαμβανόμενα ερωτήματα κυριαρχούν στις εφαρμογές της ενότητας 2.1.

Στην ίδια κατεύθυνση, η αξιοποίηση μηχανισμών άμεσης πρόσβασης της συσκευής στη μνήμη του συστήματος, αποφεύγοντας τη ρητή αντιγραφή δεδομένων από τον host, θα μπορούσε να βελτιώσει σημαντικά τη συνολική καθυστέρηση. Η υλοποίηση μιας τέτοιας δυνατότητας εξαρτάται σε μεγάλο βαθμό από τις προδιαγραφές και τις διεπαφές της εκάστοτε πλατφόρμας.

Ένας μηχανισμός δυναμικής εναλλαγής αλγορίθμου θα αξιοποιούσε άμεσα το εύρημα του λόγου n/k . Γνωρίζοντας πλέον πώς ο λόγος n/k επηρεάζει την απόδοση, μια προσαρμοστική υλοποίηση θα μπορούσε να επιλέγει αυτόματα μεταξύ σωρού και δικτύου ταξινόμησης για να διασφαλίζει ομαλή απόδοση σε όλο το φάσμα μεγεθών.

Για μεγαλύτερες τιμές του k , η επιλογή προσεγγίζει την πλήρη ταξινόμηση. Για εισόδους n που δεν χωρούν στη μνήμη της συσκευής, η διαχείριση της ροής μετατρέπεται από ευθύνη της υποδομής σε οργανικό τμήμα του ίδιου του αλγορίθμου.

Κλείνοντας, η υποδομή αξιολόγησης αυτής της εργασίας έχει γενική εφαρμογή. Ο διαχωρισμός του αλγοριθμικού χρόνου από τον χρόνο μεταφοράς, η κοινή καταγραφή ενέργειας στις διαφορετικές αρχιτεκτονικές και το μοντέλο οροφής, παρέχουν τα εργαλεία για τη μελέτη οποιουδήποτε φορτίου προορίζεται για εκφόρτωση σε ειδικό υλικό.

Παράρτημα Α΄

Ακρωνύμια

AIE AI Engine

API Application Programming Interface

AVX Advanced Vector Extensions

CPU Central Processing Unit

CUDA Compute Unified Device Architecture

D2H Device to Host

DRAM Dynamic Random Access Memory

FIFO First In First Out

GPU Graphics Processing Unit

H2D Host to Device

MLIR Multi-Level Intermediate Representation

NPU Neural Processing Unit

NVML NVIDIA Management Library

RAPL Running Average Power Limit

SIMD Single Instruction Multiple Data

SIMT Single Instruction Multiple Threads

SM Streaming Multiprocessor

TOPS Tera Operations Per Second

XRT Xilinx Runtime

Παράρτημα Β΄

Γλωσσάρι όρων

Ελληνικός όρος	Αγγλικός όρος
διτονική ακολουθία	bitonic sequence
διτονική ταξινόμηση	bitonic sort
περικομμένη διτονική ταξινόμηση	truncated bitonic sort
χρονοδιάγραμμα επιπέδων	layer schedule
συγχώνευση επιπέδων	layer fusion
δίκτυο ταξινόμησης	sorting network
σύγκριση-ανταλλαγή	compare-exchange
σωρός	heap
επιλογή με βάση την αναπαράσταση	radix selection
ψηφίδα	die
πλακίδιο	tile
πλακίδιο διασύνδεσης	shim tile
κοινή μνήμη	shared memory
απομονωτής	buffer
διπλή ενταμίευση	double buffering
άμεση προσπέλαση μνήμης	direct memory access (DMA)
νήμα	thread
διανυσματοποίηση	vectorization
βαθμωτός	scalar
ροή δεδομένων	dataflow
διοχέτευση	pipelining
υποδομή μετρήσεων	measurement harness
ταβάνι	(roofline) ceiling
υπολογιστική ένταση	operational intensity
υλοποίηση αναφοράς	reference implementation
υλοποίηση επαλήθευσης	ground truth
κίνηση δεδομένων	data traffic
εύρος ζώνης μνήμης	memory bandwidth
κατανομή εισόδου	input distribution

Πίνακας Β΄.1: Αντιστοίχιση ελληνικών και αγγλικών όρων.

Παράρτημα Γ΄

Αναπαραγωγή των μετρήσεων

Το παράρτημα αυτό περιγράφει τα βήματα αναπαραγωγής των μετρήσεων της εργασίας. Η διαδικασία περιλαμβάνει την εγκατάσταση των εργαλείων, την εκτέλεση του κώδικα, την παραγωγή των διαγραμμάτων και της αναφοράς. Οι εντολές αφορούν το σύστημα της ενότητας 3.1 και εκτελούνται από τη ρίζα του αποθετηρίου (<https://github.com/IOANNISAM/TOPIK>).

Γ΄.1 Απαιτήσεις λογισμικού

Και οι τρεις υλοποιήσεις απαιτούν μεταγλωττιστή C++ που υποστηρίζει το πρότυπο C++20, καθώς και `cmake` έκδοσης 3.18 ή νεότερης. Οι διανυσματικές εντολές δεν αποτελούν απαίτηση μεταγλώττισης. Η επιλογή ανάμεσα σε `AVX-512`, `AVX2` και βαθμωτή εκτέλεση γίνεται κατά τον χρόνο εκτέλεσης, με βάση τις δυνατότητες που αναφέρει ο επεξεργαστής. Επειδή οι μετρήσεις έγιναν σε μηχανήμα με `AVX-512`, η απουσία του οδηγεί σε διαφορετική διαδρομή κώδικα και τα αποτελέσματα δεν είναι άμεσα συγκρίσιμα.

Η μονάδα γραφικών απαιτεί την εργαλειοθήκη `CUDA`, η οποία περιλαμβάνει τον μεταγλωττιστή `nvc` και τη βιβλιοθήκη `CUB`. Οι μετρήσεις ισχύος προϋποθέτουν επιπλέον τη βιβλιοθήκη `NVML` του οδηγού.

Η μονάδα νευρωνικής επεξεργασίας απαιτεί τη στοίβα εκτέλεσης `XRT` και το πρόσθετο `XDNA`. Η μεταγλώττιση των πυρήνων χρειάζεται το `MLIR-AIE` σε περιβάλλον `Python`. Αυτό το περιβάλλον πρέπει να βρίσκεται στη ρίζα του αποθετηρίου με το όνομα `hru_venv`, καθώς εκεί αναζητείται από τα σενάρια εκτέλεσης. Ο χρήστης πρέπει να ανήκει στις ομάδες πρόσβασης της συσκευής και να έχει επαρκές όριο κλειδωμένης μνήμης, διαφορετικά η συσκευή δεν αναγνωρίζεται. Η μεταβλητή `XILINX_XRT` πρέπει να δείχνει στην εγκατάσταση της `XRT`.

Η μέτρηση ενέργειας διαβάσει τους μετρητές `RAPL` από το `/sys/class/powercap`, διαδικασία που απαιτεί δικαιώματα διαχειριστή. Σε μεμονωμένη εκτέλεση χωρίς αυτά, η μέτρηση δηλώνεται ρητά μη διαθέσιμη αντί να καταγράψει μηδενικά (ενότητα 3.3.3). Αντίθετα, η αυτοματοποιημένη σάρωση ακυρώνεται εντελώς για να αποφευχθεί η παραγωγή κενού συνόλου δεδομένων. Τέλος, η υποδομή ανάλυσης απαιτεί τις βιβλιοθήκες `Python` του αποθετηρίου. Το αντίστοιχο περιβάλλον δημιουργείται αυτόματα κατά την πρώτη εκτέλεση.

Γ'.2 Μεταγλώττιση

Η μεταγλώττιση όλων των υλοποιήσεων γίνεται με μία εντολή:

```
make build-all
```

Κάθε αρχιτεκτονική μπορεί να μεταγλωττιστεί και αυτόνομα, με τους στόχους `build-cpu`, `build-gpu` και `build-npu`, κάτι που είναι χρήσιμο σε σύστημα που δεν διαθέτει και τις τρεις μονάδες. Τα παραγόμενα εκτελέσιμα τοποθετούνται στον κατάλογο `build`, δύο ανά αρχιτεκτονική: ένα για τις μετρήσεις `top-k` και ένα για τις μετρήσεις των ταβανιών της ενότητας 3.4.

Η μεταγλώττιση για τη μονάδα νευρωνικής επεξεργασίας παράγει επιπλέον τα αρχεία διαμόρφωσης του υλικού. Επειδή η απουσία τους οδηγεί σε σιωπηλή παράλειψη των αντίστοιχων μετρήσεων, η παρουσία τους ελέγχεται ρητά:

```
make verify-artifacts
```

Ο έλεγχος εξετάζει τα δύο αρχεία διαμόρφωσης και τα τρία εκτελέσιμα `top-k`. Προϋποθέτει πλήρη μεταγλώττιση, οπότε αποτυγχάνει αυτόματα αν έχει μεταγλωττιστεί μόνο μία αρχιτεκτονική.

Γ'.3 Εκτέλεση των πειραμάτων

Ο συνιστώμενος τρόπος αναπαραγωγής είναι η συνολική εκτέλεση της διαδικασίας με μία εντολή:

```
make benchmark GPU_W=50
```

Αυτή η εντολή διασφαλίζει τη σωστή αλληλουχία βημάτων. Αν οι συνθήκες καθηλώσουν εκ των υστέρων ή χρησιμοποιηθούν παλαιά εκτελέσιμα, τα αποτελέσματα χάνουν τη συγκρισιμότητά τους. Η εντολή επιβάλλει την κατάλληλη σειρά και αποτρέπει τις παραλείψεις.

Η ακολουθία περιλαμβάνει έξι στάδια. Αρχικά διαγράφονται τα παλαιά αποτελέσματα και εκτελέσιμα. Ακολουθεί η μεταγλώττιση και ο έλεγχος των αρχείων (ενότητα Γ'.2). Στο τρίτο στάδιο καθηλώνονται οι συνθήκες του συστήματος και παρέχεται πρόσβαση στους μετρητές ενέργειας. Ακολουθούν οι τέσσερις σαρώσεις μετρήσεων, η παραγωγή των διαγραμμάτων και, τελικά, η αποκατάσταση του συστήματος.

Η διαδικασία ελέγχει πρώτα τη διαθεσιμότητα του `sudo` και του περιβάλλοντος `npu_venv`. Αν κάτι από τα δύο απουσιάζει, η εντολή τερματίζει άμεσα, χωρίς να αλλοιώσει το σύστημα ή τα δεδομένα.

Τα δικαιώματα διαχειριστή ζητούνται μία φορά και παραμένουν ενεργά. Οι συνθήκες αποκαθίστανται πάντα στο τέλος, ακόμη και αν υπάρξει σφάλμα ή διακοπή από τον χρήστη, αποτρέποντας τον εγκλωβισμό του συστήματος σε καθηλωμένη κατάσταση. Η αποτυχία μιας μεμονωμένης σάρωσης δεν σταματά τις υπόλοιπες, καταγράφεται απλώς ως προειδοποίηση.

Το όρισμα `GPU_W` ορίζει το όριο ισχύος της μονάδας γραφικών, το οποίο πρέπει να παραμείνει σταθερό (ενότητα 3.8). Η διαδικασία διαγράφει αμέσως τα υπάρχοντα αποτελέσματα, συνεπώς ο χρήστης οφείλει πρώτα να κρατήσει αντίγραφα όσων δεδομένων

θέλει να διατηρήσει.

Γ'.3.1 Εκτέλεση επιμέρους βημάτων

Για μερική ή επαναληπτική εκτέλεση, κάθε στάδιο είναι διαθέσιμο και ξεχωριστά. Οι συνθήκες του συστήματος ελέγχονται με:

```
make pin ARGS=50      # governor, boost off, GPU cap 50 W
make pin-show          # report the current state
make unpin             # restore governor, boost, GPU cap
```

Οι τέσσερις ανεξάρτητες σαρώσεις καταγράφουν τα ακατέργαστα αποτελέσματά τους σε χωριστά αρχεία:

```
make measure-cases      # timing over the parameter space
make measure-energy      # energy, requires RAPL access
make measure-dists      # input-distribution sensitivity
make measure-roofline   # bandwidth, compare-exchange roofs
```

Οι στόχοι δέχονται πρόσθετα ορίσματα μέσω της μεταβλητής ARGS, περιορίζοντας τον παραμετρικό χώρο. Αυτό διευκολύνει τον γρήγορο έλεγχο της εγκατάστασης. Στη χειροκίνητη εκτέλεση, ο χρήστης αναλαμβάνει την ευθύνη για τη σωστή σειρά και τη διατήρηση σταθερών συνθηκών.

Γ'.3.2 Παράμετροι εκτέλεσης

Ο πίνακας Γ'.1 συνοψίζει τις παραμέτρους κάθε εκτελέσιμου, τις επιτρεπτές τιμές, τις προεπιλογές και τις τιμές των σαρώσεων.

Συχνά οι προεπιλογές διαφέρουν από τις τελικές τιμές των σαρώσεων. Το εύρος τιμών ορίστηκε έως 10^9 αντί της προεπιλογής, για να αποφευχθεί η υπερβολική συσσώρευση ισοβαθμιών σε εισόδους εκατομμυρίων στοιχείων. Οι πολλές ισοβαθμίες θα ευνοούσαν τεχνητά τους αλγορίθμους που απορρίπτουν γρήγορα στοιχεία.

Το εύρος αυτό αφορά τους τέσσερις από τους πέντε τύπους. Για τον half, το εκτελέσιμο περικόπτει τα άκρα στο ± 65504 , τη μέγιστη πεπερασμένη τιμή του τύπου. Σε αυτή την περίπτωση οι ισοβαθμίες παραμένουν αναπόφευκτες, λόγω του στενού εύρους και της αδυναμίας του half να αναπαραστήσει όλους τους ακεραίους πάνω από το 2048. Η ιδιότητα αυτή πηγάζει από τον ίδιο τον τύπο και επηρεάζει άμεσα την ερμηνεία των αποτελεσμάτων.

Ο σπόρος τυχαίων αριθμών παράγεται ντετερμινιστικά από τα q και k της κάθε περίπτωσης. Έτσι διασφαλίζεται η επαναληψιμότητα χωρίς να δοθεί ποτέ η ίδια είσοδος σε διαφορετικές συνθήκες.

Η κύρια σάρωση χρησιμοποιεί τις τιμές της τέταρτης στήλης. Οι ειδικές σαρώσεις περιορίζουν τις παραμέτρους: η σάρωση κατανομών εξετάζει q από 20 έως 24, τύπους float και half, και άκρα k 8 και 131072, με δύο σπόρους ανά περίπτωση. Η σάρωση ενέργειας εξετάζει q από 16 έως 24 με τρεις επαναλήψεις, παραλείποντας τον έλεγχο ορθότητας για να μην επιβαρυνθεί άσκοπα η καταναλισκόμενη ισχύς.

Παράμετρος	Τιμές	Προεπιλογή	Στις σαρώσεις	Σημασία
q	ακέραιος 0-63	—	3-24	Μέγεθος εισόδου $n = 2^q$ · είναι η μόνη υποχρεωτική
k	θετικός ακέραιος	n	8, 256, 4096, 131072	Πλήθος ζητούμενων στοιχείων· περιορίζεται στο n
mode	min max	max	max	Επιλογή μικρότερων ή μεγαλύτερων
dtype	int uint float double half	int	και οι πέντε	Τύπος στοιχείου
algo	bitonic map_reduce gt	bitonic	και οι τρεις	Αλγόριθμος προς εκτέλεση
run	full trunc both	trunc	trunc	Παραλλαγή του διτονικού δικτύου
seed	μη αρνητικός ακέραιος	42	παράγεται από τα q, k	Σπόρος της γεννήτριας τυχαίων αριθμών
dist	uniform normal trimodal sorted reverse adversarial	uniform	uniform· πέντε στη σάρωση κατανομών	Κατανομή της εισόδου
min / max	ακέραιοι	0 / 1000	0 / 10^9	Εύρος τιμών· απαιτείται $\max \geq \min$ · περικόπτεται στο εύρος του τύπου
threads	θετικός ακέραιος	0 (αυτόματο)	8	Πλήθος νημάτων· η NPU το αγνοεί
verify	true false	false	true	Έλεγχος ως προς την υλοποίηση επαλήθευσης
debug	true false	true	false	Αναλυτική έξοδος διαγνωστικών

Πίνακας Γ'.1: Οι παράμετροι εκτέλεσης και οι προεπιλογές τους.

Γ'.4 Παραγωγή των διαγραμμάτων

Η υποδομή ανάλυσης διαβάξει τα ακατέργαστα αποτελέσματα, τα μετατρέπει σε πίνακες CSV και παράγει τα διαγράμματα:

```
make plot ARGS="--plot all"
```

Το όρισμα `-plot` επιλέγει τα διαγράμματα και το `-input` τη σάρωση (η προεπιλογή είναι η σάρωση χρόνου). Επειδή τα υπόλοιπα διαγράμματα χρησιμοποιούν δικά τους αρχεία, συνιστώνται οι συνδυαστικοί στόχοι, οι οποίοι εκτελούν άμεσα τη μέτρηση και την παραγωγή:

```
make benchmark-cases
make benchmark-energy
make benchmark-dists
make benchmark-roofline
```

Τα αρχεία οργανώνονται ανά τύπο δεδομένων και κατηγορία. Διαγράμματα που αφορούν αποκλειστικά το υλικό (π.χ. κλίμακα εύρους ζώνης) αποθηκεύονται σε ξεχωριστό κατάλογο, καθώς δεν εξαρτώνται από τον τύπο.

Δύο παράμετροι καθορίζουν την απεικόνιση. Η `-energy-metric` επιλέγει μεταξύ μεικτής ή καθαρής ενέργειας. Οι σαρώσεις της εργασίας χρησιμοποιούν την καθαρή (ενότητα 3.3.3). Η παράμετρος `-error-bars` ορίζει τη ζώνη διασποράς (τυπική απόκλιση, δεκατημόρια ή καμία). Η ζώνη σχεδιάζεται μόνο αν υπάρχουν τουλάχιστον δύο δείγματα ανά σημείο, δηλαδή στη σάρωση κατανομών (δύο σπόροι) και ενέργειας (τρεις επαναλήψεις). Στην αυτοματοποιημένη εκτέλεση, οι ζώνες απενεργοποιούνται πλήρως.

Bibliography

- [1] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [2] N. Sismanis, N. Pitsianis, and X. Sun, “Parallel search of k-nearest neighbors with synchronous operations,” in *2012 IEEE Conference on High Performance Extreme Computing (HPEC)*, Waltham, MA, USA, 2012, pp. 1–6.
- [3] J. Zhang, A. Naruse, X. Li, and Y. Wang, “Parallel top-K algorithms on GPU: A comprehensive study and new methods,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’23)*, Denver, CO, USA, 2023.
- [4] K. E. Batcher, “Sorting networks and their applications,” in *Proceedings of the AFIPS Spring Joint Computer Conference*, 1968, pp. 307–314.
- [5] A. Shanbhag, H. Pirk, and S. Madden, “Efficient top-K query processing on massively parallel hardware,” in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, Houston, TX, USA, 2018, pp. 1557–1570.
- [6] D. A. Palmer and M. Florea, “Neural processing unit,” U.S. Patent US8655815B2, assigned to the University of California, San Diego, 2014, application US13/011,727, filed 21 January 2011; granted 18 February 2014.
- [7] H. Esmailzadeh, A. Sampson, L. Ceze, and D. Burger, “Neural acceleration for general-purpose approximate programs,” in *45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2012, pp. 449–460.
- [8] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” in *19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 269–284.
- [9] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *44th International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12.
- [10] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *43rd International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [11] AMD, “Versal adaptive SoC: The industry’s first adaptive compute acceleration platform,” AMD, White Paper, 2021, <https://www.amd.com/>

content/dam/amd/en/documents/products/adaptive-socs-and-fpgas/versal/adaptive-soc-paper.pdf.

- [12] A. Rico, S. Pareek, J. Cabezas, J. Noguera *et al.*, “AMD XDNA NPU in Ryzen AI processors,” *IEEE Micro*, vol. 44, no. 6, 2024.
- [13] AMD/Xilinx, “MLIR-AIE: An MLIR-based toolchain for AMD AI engine-enabled devices,” <https://github.com/Xilinx/mlir-aie>, 2024.
- [14] NVIDIA, “CCCL: CUDA core compute libraries,” <https://github.com/NVIDIA/cccl>, 2024.
- [15] C. Lin, M. Yang, C. Wang, and C. Huang, “Adaptive top-K selection: Eliminating performance cliffs across all K values on AMD GPUs,” AMD ROCm Blogs, Feb. 2026, <https://rocm.blogs.amd.com/software-tools-optimization/adaptive-topk/README.html>.
- [16] Y. Samaga, V. Yerram, S. R. Babbula, P. Jain, and P. Netrapalli, “A faster generalized two-stage approximate top-K,” *Transactions on Machine Learning Research*, 2026, <https://openreview.net/forum?id=izqZ1Crpjz>.
- [17] AMD, “Bitonic SIMD sorting on AI engine for float datatypes,” Vitis Tutorials: AI Engine Development, 2024.1, 2024, <https://docs.amd.com/r/2024.1-English/Vitis-Tutorials-AI-Engine-Development/Bitonic-SIMD-Sorting-on-AI-Engine-for-float-Datatypes>.
- [18] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [19] Intel, “Intel intrinsics guide,” <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>, 2024.
- [20] AMD, “IRON: A close-to-metal programming framework for AMD NPUs,” <https://github.com/amd/IRON>, 2024.
- [21] —, “AI engine kernel and graph programming guide (UG1079): Casting and datatype conversion,” 2024, <https://docs.amd.com/r/en-US/ug1079-ai-engine-kernel-coding/>.